



SOFTWARE FRAMEWORKS – AN OVERVIEW

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

CONTENTS

- Preliminaries
- Programming vs Software Engineering
- Generic functionalities in Software Development
- Frameworks: what are they?
- How does Frameworks help with the Software Engineering Approach?
- Framework vs. Libraries
- Why use frameworks?
- Advantages of using frameworks
- Limitations of using frameworks

PRELIMINARIES

- What is this course?
 - **PAF (IT3030)** introduces the fundamentals of full stack web development with an emphasis on application of good software engineering practices and Industry standards.
- Learning Outcomes of this course?
 - After completing this module, you will,
 - Know the basics of Software Frameworks
 - Apply Industry standard software development best practices
 - Understand the REST architectural style
 - Develop Java based REST web services (Back-end)
 - Develop JavaScript based Interactive web applications (Front-end)
- Module Outline
- Assessments

PROGRAMMING VS SOFTWARE ENGINEERING

- What is Programming?
 - Programming simply means writing Code to create an application.
- What is Software Engineering?
 - Software engineering is the end-to-end process of applying good engineering principles and software development practices to produce quality software.
- Programming vs Software Engineering
 - Software Engineering is **NOT** just coding. Coding is just one part of a large process.
 - **Q: What other practices come under software engineering?**

PROGRAMMING VS SOFTWARE ENGINEERING

- What other practices come under software engineering?
 - Analyzing user requirements and designing the software to meet those requirements.
 - Communicating with all the stakeholders to resolve issues and update them on the progress.
 - Choosing the software architectures, patterns, algorithms, coding techniques, technologies used to realize the software design based on sound design considerations.
 - Quality Assurance (QA) of the developed software to ensure that the software meets the client requirements and acceptable reliability, performance and security considerations.
 - Deploying/ releasing the software for actual use.
 - Maintaining the developed software as needed after release/ deployment.

GENERIC FUNCTIONALITIES IN SOFTWARE DEVELOPMENT

- Generic functionalities in software development
 - Most (web) applications have a lot of features in common.
 - E.g.: User Authentication, routing, database connectivity, scheduled jobs etc.
 - There is little point in rewriting all the functionalities (including the common ones) from scratch every time new software is developed.
 - What if the generic functionalities can be collected and made reusable?

FRAMEWORKS:WHAT ARE THEY?

- A framework is an integrated set of software artifacts (such as classes, objects, and components) that collaborate to provide a reusable architecture for a family of related applications [1].
- A framework enables a coding environment that contains low-level libraries to address conventional coding issues. The objective of a framework is to deliver faster development of an application. This includes everything we need to build large-scale applications, such as templates based on best practices [4].
- A framework is a foundation for developing software applications. Software engineers and developers use a framework as a template to create websites and applications. Developers do this by adding code to a framework, then personalizing it for their specific purpose [5].
- A software framework is an abstraction in which software, providing generic functionality, can be selectively changed (**extended**) by additional user-written code, thus providing application-specific software [6].

HOW DO FRAMEWORKS HELP WITH THE SOFTWARE ENGINEERING APPROACH?

- How do Frameworks help with the Software Engineering Approach?
 - Architectural Patterns (e.g.: MVC, MVVM)
 - Design Patterns
 - Inversion of Control
 - Standards/ Best Practices (Naming conventions, Coding standards, Directory structures etc.)
 - Tooling Support (Package managers, Dependency managers, Boilerplate code generation)
 - Testing Support

FRAMEWORK VS. LIBRARIES

- Certain features make a framework different from other library forms, including the following [7]:
 - **Default Behavior:** Before customization, a framework behaves in a manner specific to the user's action.
 - **Inversion of Control:** Unlike other libraries, the global flow of control within a framework is employed by the framework rather than the caller.
 - **Extensibility:** A user can extend the framework by selectively replacing default code with user code.
 - **Non-modifiable Framework Code:** A user can extend the framework, but not modify the code.

WHY USE FRAMEWORKS?

- The designers of software frameworks aim to facilitate software developments by allowing designers and programmers to **devote their time to meeting software requirements rather than dealing with the more standard low-level details of providing a working system, thereby reducing overall development time** [6].
- The aim of frameworks is to provide a common structure so that developers don't have to redo it from scratch and can reuse the code provided. In this way, frameworks allow us to cut out much of the work and save a lot of time.
- To summarize: there's **no need to reinvent the wheel**.

ADVANTAGES OF USING FRAMEWORKS

- Improved coding, easy code reusability and ease of debugging code compared with from-scratch developments.
- Community support.
- Accelerated development (if the programmer is familiar with the framework).
- Often provides caching and optimized processes for resource intensive tasks.
- Enables faster methods for development with less code (boilerplate code etc.).
- Better compliance to accepted security standards.

LIMITATIONS OF USING FRAMEWORKS

- Learning the Framework and not the programming language prevent programmers from gaining an in-depth understanding of the programming language.
- Users of a framework are forced to respect the limitations and conventions imposed by its design.
- Options to tweak functionalities are limited.
- A framework will come with everything required to satisfy a wide range of use cases, not all these features will be used for a given project.
- The right framework for the application should be chosen, or else performance and user experience may be impacted negatively.
- We must be up-to-date with new/deprecated features in every version.

SOME EXAMPLES

- Spring/ Spring Boot (Java)
- Express.js (with Node.js)
- Laravel (PHP)
- .NET/ .NET Core (C# etc.)
- Django (Python)
- Angular (Typescript)
- Vue.js (JavaScript)



SUMMARY

- Programming vs Software Engineering
- Generic functionalities in Software Development
- Frameworks: what are they?
- How do Frameworks help with the Software Engineering Approach?
- Framework vs. Libraries
- Why use frameworks?
- Advantages and limitations of using frameworks
- Some examples

REFERENCES

1. [Software Engineer vs. Programmer: What's the Difference?](#)
2. [Frameworks: Why They Are Important and How to Apply Them Effectively](#)
3. [What is a Web Framework, and Why Should I use one?](#)
4. [The Difference Between a Framework and a Library](#)
5. [What Is a Framework? \(Definition and Types of Frameworks\)](#)
6. [Software framework](#)
7. [Techopedia Explains Software Framework](#)



THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK



VERSION CONTROLLING & WORKFLOWS WITH GIT - I

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

LEARNING OUTCOMES

- After completing this lecture, you will be able to,
 - Describe the basic concepts of Version Controlling.

CONTENTS

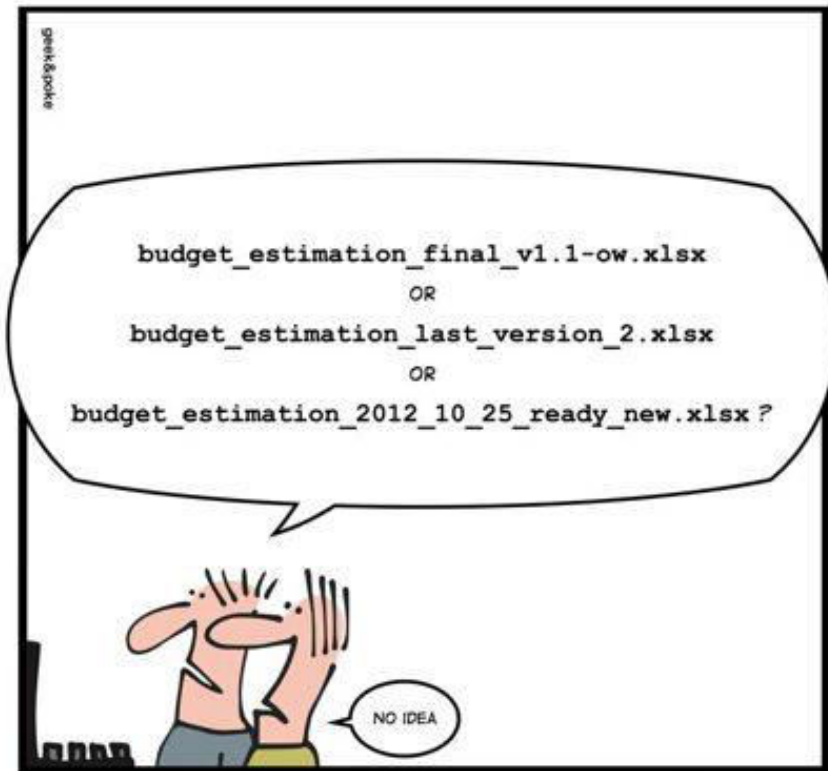
- Version Controlling: What?
- What you know about Version Controlling
- Version Controlling: Scenario Discussion
- A bit of history on Version Controlling Systems (VCS)
- Git: the de-facto standard today in VCS
- Git Basic Concepts
- Conclusion

VERSION CONTROLLING:WHAT?

- “Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later.” [1].

WHAT YOU KNOW ABOUT VERSION CONTROLLING

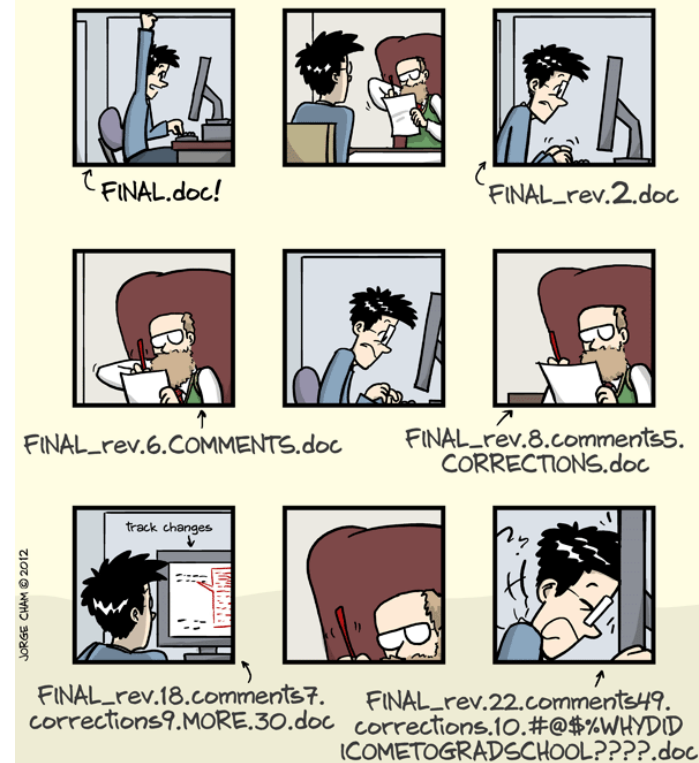
SIMPLY EXPLAINED



VERSION CONTROL

Source: <https://www.datamation.com/trends/tech-comics-version-control/>

"FINAL".doc



Source: <https://swcarpentry.github.io/hg-novice/01-basics/>

WHAT YOU KNOW ABOUT VERSION CONTROLLING

- Q: What methods of Version Controlling are you familiar with?
 - Keeping multiple copies of files with different names
 - Google Drive/ OneDrive
 - Undo/ Redo Buffer
- Now, let's see how one student named Amith does version controlling.

VERSION CONTROLLING: SCENARIO DISCUSSION

- Suppose Amith is working on one file called numbers.py for an assignment.
- Amith has worked on the file for several days now, and has saved multiple copies of the file, i.e., numbers1.py, numbers2.py, numbers3.py and numbers.py.
- numbers.py is the latest version of the file. Whenever a new update is to be made to the file, first the existing numbers.py is renamed to numbersX.py ($X = \{1, 2, 3, 4 \dots\}$) and then the numbers.py is saved with the most recent change.
- Amith needs to submit the assignment now, but at the last moment he discovers that the numbers.py (most recent version) throws a serious error.

VERSION CONTROLLING: SCENARIO DISCUSSION

- **Q1:** What can Amith do?
- **A:** Amith can submit the one before (e.g.: numbers10.py).
- **Q2:** What if the most recent version (numbers.py) has some good changes too, which Amith thinks needs to be in the submitted file?
- **A:**
 - Compare numbers10.py and numbers.py and Copy the good parts.
 - Paste the copied code as fit in the appropriate places in numbers10.py.
 - Replace the numbers.py with this modified file.

VERSION CONTROLLING: SCENARIO DISCUSSION

- The assignment was submitted. Now, Amith needs to keep this numbers.py and all its previous versions for future submissions too.
- However, Amith has a nagging feeling about what will happen if the HDD of his laptop fails.
- **Q3:** What can Amith do address this very valid concern?
- **A:** Copy all the versions of numbers.py to a Cloud storage location (OneDrive/ Google Drive).

VERSION CONTROLLING: SCENARIO DISCUSSION

- What are the pros of above approach?
 - All the versions are safely stored.
 - The files can be accessed from anywhere now.
- The cons?
 - Considering accessing from anywhere: If Amith develops two versions of numbers.py (both named the same), one at the computer lab in his university (then uploaded to the cloud) and another numbers.py separately on his laptop, Amith runs the risk of replacing the version on laptop with the other if he uploads the first file without the name being changed when his laptop syncs with the cloud storage.

VERSION CONTROLLING: SCENARIO DISCUSSION

- All in all, Amith now has a working Version Controlling System.
- His system can,
 - **Revert** to a past version.
 - **Compare** two file versions.
 - **Push** changes done from anywhere to a secure remote location on the cloud.
 - **Pull** changes from the cloud to anywhere.
 - **Merge** different versions of files.

VERSION CONTROLLING: SCENARIO DISCUSSION

- Now, Binesh joins Amith as the future assignments are to be done as pair work.
- It is more important than ever to follow a strict discipline when using Amith's system of version controlling as there are other people involved (than Amith).
- However, just focusing on naming the files correctly (to avoid overwrites) is not enough.
- Just seeing the files will not give each developer a high-level idea about what each other is doing.

VERSION CONTROLLING: SCENARIO DISCUSSION

- Solution?
 - A log file!
- How?
 - The log file should contain information on **who** did **what** **when** so everybody would be aware of what is happening.



Log - Notepad

File	Edit	Format	View	Help
EntryNo DateTime Author Change				
+-----+-----+-----+-----+				
01	08/02/23	Amith	Added numbers.py file and all previous versions to OneDrive.	
02	09/02/23	Amith	Added comments on Add() method on numbers.py.	
03	10/02/23	Binesh	Added new file sequence.py to OneDrive folder.	

VERSION CONTROLLING: SCENARIO DISCUSSION

- Does Amith's method solve the problem of version controlling he had earlier?
 - Yes.
- Does that mean it is an easy system to follow?
 - Not really. There are several specific rules the users of the system must follow in order to make it work.
 - As the number of users of this system increase, so does the chance of someone making a mistake and ruining the system for everybody.
- Should he follow this approach then?
 - Yes, it's a good system. But the management of the system should not be given to the users. It should be automated.
- Are there such automated systems already?
 - Yes! That's what version controlling systems like GIT, SVN and Mercurial do.

A BIT OF HISTORY ON VERSION CONTROLLING SYSTEMS (VCS)

- The first proper Version Control System (VCS), named *Source Code Control System (SCCS)* was released in 1973 by Bell Labs.
- It was a first-generation VCS, which are now collectively known as **Local Version Control Systems**.
- Local VCS were intended to track changes for individual files and checked-out files could only be edited locally by one user at a time.
- Facilitating collaboration among large teams was problematic.

A BIT OF HISTORY ON VERSION CONTROLLING SYSTEMS (VCS)

- To address the problem of collaboration, a new type of VCS were developed.
- This second generation of VCS are collectively known as **Centralized Version Control Systems**.
- Centralized VCS allowed users to refer files stored on a centralized repository through a network and allowed multiple users to use the files concurrently.
- Follows a *Client-Server* approach.
- However, they would all have to commit their code to the centralized repository, which would require network access.
- A central authority is required to maintain the Centralized repository.

A BIT OF HISTORY ON VERSION CONTROLLING SYSTEMS (VCS)

- With the advent of open-source software came the third generation of VCS. They are collectively known as **Distributed Version Control Systems**.
- The earlier Centralized VCS required a central authority to maintain the centralized repository. No such authorities are usually found in open-source projects.
- Instead, there are just contributors who may develop features which may or may not be relevant to the main open-source project.
- Therefore, Distributed VCS have *peer-to-peer (P2P)* approach to version controlling.
- Distributed VCS are widely used today.
- De-facto standard in Distributed VCS now?
 - **Git!**

GIT: THE DE-FACTO STANDARD TODAY IN VCS

- Created by **Linus Torvalds** (creator of Linux) in 2005.
- Pronounced as *Ghi-T*. Not *Ji-T* (A "git" is a cranky old man. According to online sources, Linus meant himself).
- Originally designed to do version control on Linux kernel.
- Git supports non-linear development (thousands of parallel branches) and is fully distributed.

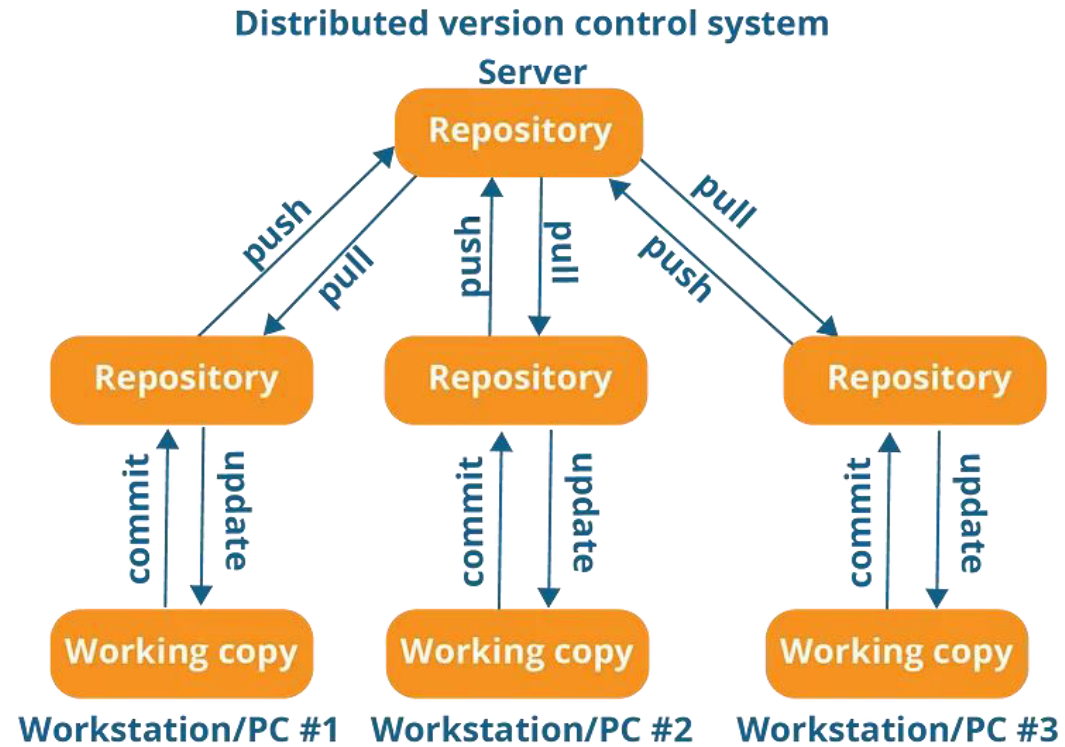
GIT BASIC CONCEPTS

- As mentioned, git does not necessarily need a Centralized Repository to work (It can work in somewhat centralized manner too, which we will see).
- Every user maintains a **local repository** of their own.
- When starting out they will obtain this local repository,
 - from another user or,
 - they may **clone** it from a central location or,
 - They may even initialize a brand-new repository from scratch.
- This repository will contain what others have done originally, what the user has done themselves and even work of others as selected by the user.
- They can **commit** and **update** their local repository without any interference from any others.

GIT CONCEPTS

- Git has two types of repositories.
 - Local repositories – which is owned by each user.
 - Remote repository – a centralized location accessible to all users. A single source of truth.
 - The users will **push** their new updates to this repository so those can be shared with others as well.
 - The users will **pull** new updates from others from this repository.
- Why do we need Remote repositories?
 - Scenario Discussion: The problem with Distributed Version Controlling. Which project is the “right” project when multiple versions of the project exist?
 - Solution? Remote repositories as a single source of truth!
- Git != GitHub (Git is not Github)!

GIT CONCEPTS



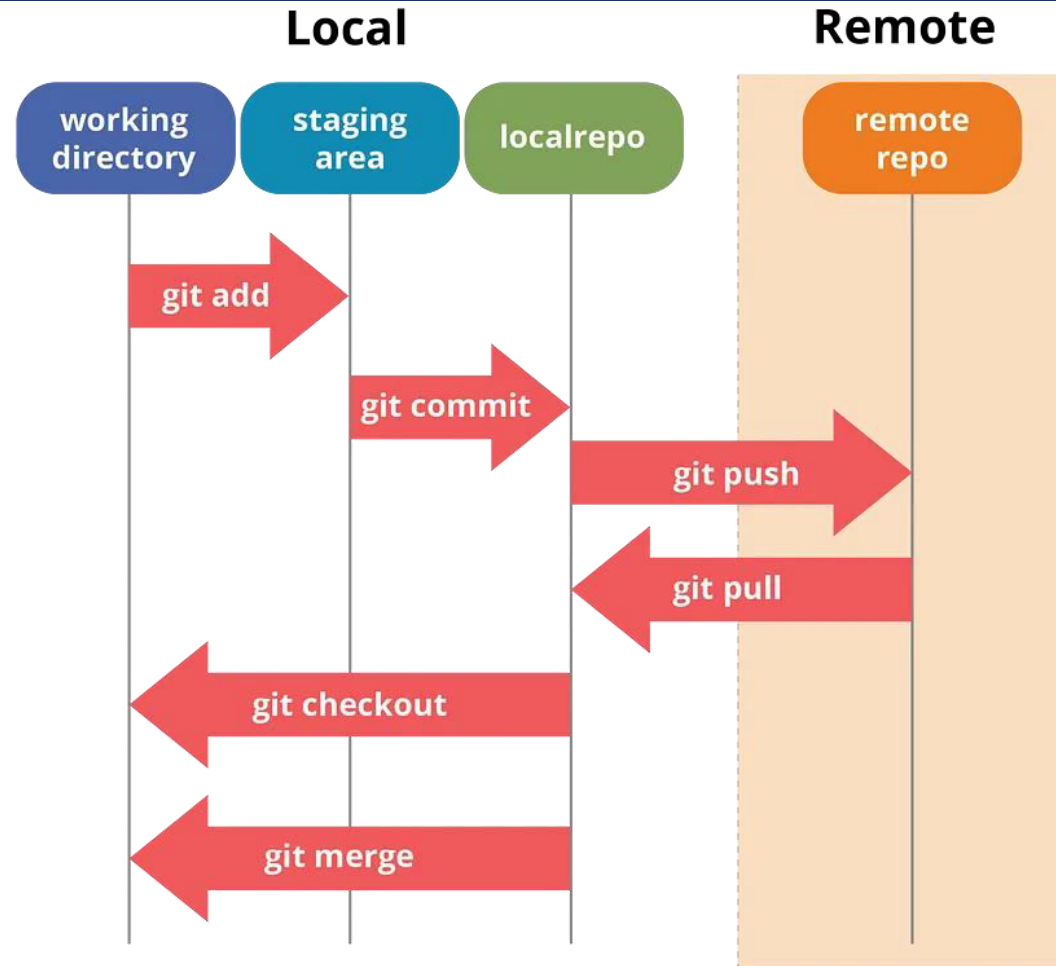
Source: <https://medium.com/@sahoosunilkumar/how-does-git-works-5cc8444ea383>

GIT CONCEPTS

- Git works on **Commits**.
- A Commit is very simply a **snapshot of all changes in the local repository** (more correctly, the working directory).
- Each Commit has a unique Commit ID.
- A Commit ID is a unique SHA-1 hash that is created whenever a new commit is recorded.
- The most recent commit is called the **Head** (more correctly, it refers to the currently checked-out branch's latest commit).
- For now, we will only deal with **main/ master Branch**.
- We will extensively deal with Commit IDs for many operations on git (more on this later).

GIT CONCEPTS

1. **Add** a new file to the local Repo.
2. Do the required changes to the file.
3. **Commit** the file to the local repo.
4. **Pull** the changes from the remote repo.
5. **Push** the local changes to the remote repo.



SCENARIO REVISITED

- Refer back to Amith's version controlling method in the beginning. It could,
 - **Revert** to a past version.
 - **Compare & Merge** two different versions of files.
 - **Pull** changes from a remote location to anywhere.
 - **Push** changes done from anywhere to a secure remote location.
 - Use the **Log file** to keep track of all the events.
- All these actions can be easily accomplished using Git.
 - Revert – git revert (this reverts a commit that was made to the repository).
 - Compare & Merge – Automatically handled by git. No user intervention required most of the time (unless a conflict occurs).
 - Push and Pull – Update the **Upstream** and get updates from upstream.
 - Git has many ways of automatically keeping track of all the events (e.g.: git status, log, blame etc.)

NEXT WEEK

- Branching
- Best Practices
- Pull Requests
- Git Workflows

REFERENCES

1. [Pro Git - Scott Chacon and Ben Straub \[Free eBook\]](#)
2. [Dangit, Git!?! \[Git quick reference\]](#)
3. [Reading 5:Version Control \[MIT Lecture\]](#)
4. [Distributed Version Control \[Wikipedia\]](#)
5. [History of Version Control Systems](#)



THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK



VERSION CONTROLLING & WORKFLOWS WITH GIT - II

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

LEARNING OUTCOMES

- After completing this lecture, you will be able to,
 - Apply the basic concepts of Git for version controlling
 - Apply an efficient branching strategy for a team
 - Describe what a git workflow is and why a workflow is important.
 - Apply a simple git workflow to your own projects.

CONTENTS

- Recap from the previous week
- Git branching
 - Workflows (Branching strategies)
 - GitFlow
 - GitHub Flow
 - Pull Requests (PRs)
 - Best practices for branching
- Git general best practices
- Additional topics
- Summary

FROM THE PREVIOUS WEEK

- Version Controlling: Scenario Discussion
- A bit of history on Version Controlling Systems (VCS)
- Git Basic Concepts
 - Repositories – Local and Remote
 - Each user has their own local repository
 - This local repository can be obtained by,
 - Copying from another user (P2P)
 - **Cloning** it from a remote repository (on GitHub, BitBucket etc.)
 - Initializing their own repository.
 - Commits
 - Basic git operations (git add, commit, push, pull)

GIT BRANCHING: THE BASICS

- Think of a tree. What is a branch in a tree?
 - An offshoot of the trunk of a tree.
- What is a branch in Git?
 - The default branch in Git is the ***master/******main*** branch (It is like the trunk of a tree).
 - Very simply put, a branch is a detour you make from the main path of development.

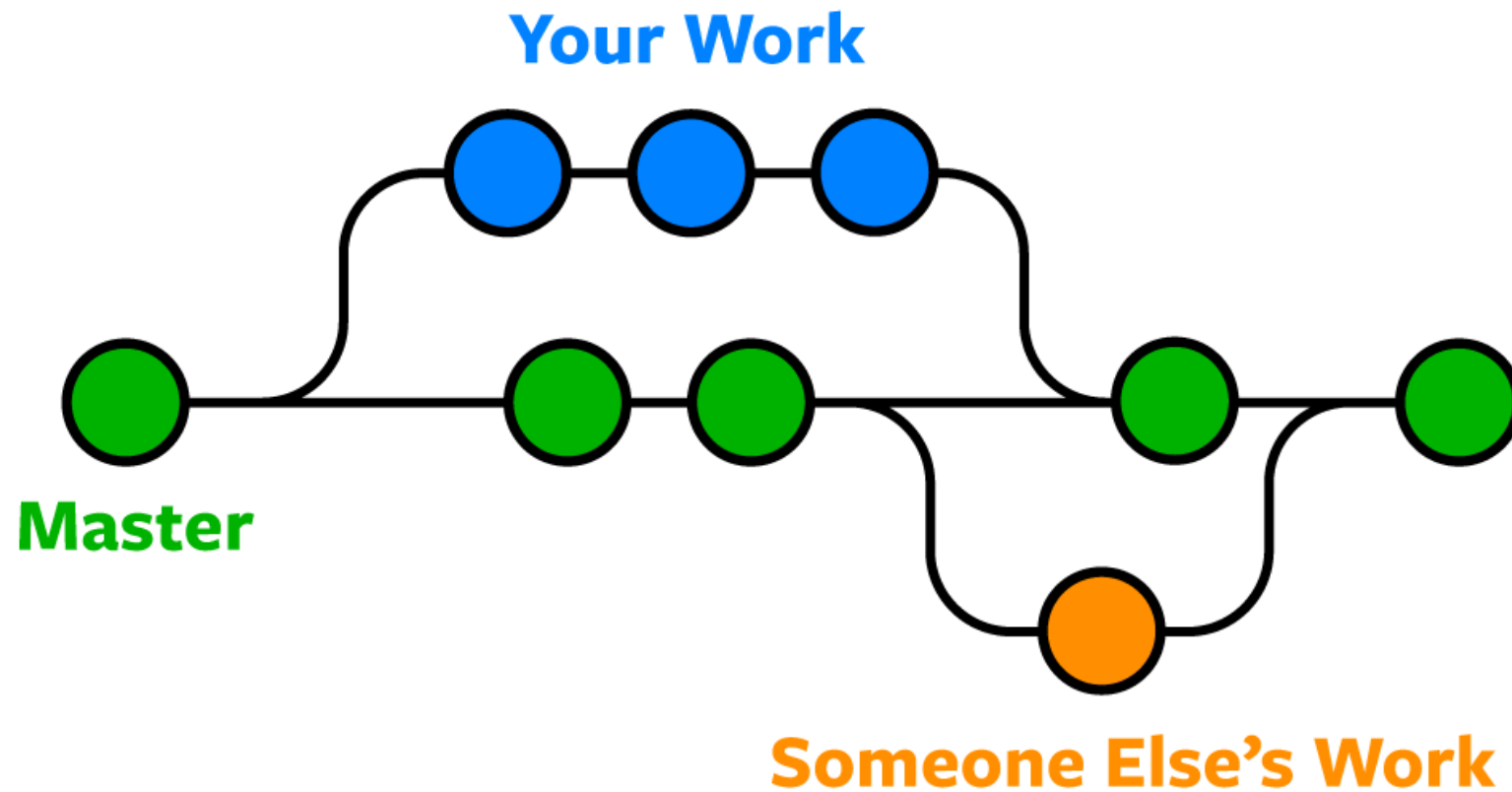


Source: <https://www.rainforest-alliance.org/wp-content/uploads/2021/06/kapok-tree-profile-1.jpg>

GIT BRANCHING: THE BASICS

- Why is branching needed?
 - Branching lets developers work independently inside a single code repository.
 - E.g., In product X, Amith works on feature A, while Binesh works on feature B.
 - If both have their code in just their local repositories, then no problem.
 - Now, it is a good practice to push your changes to the remote repository regularly.
 - What sort of problems can be expected when they do that?
 - **Merge Conflicts!**
 - Any solution to the above problem?
 - Yes, let each of them work on their own branch.

GIT BRANCHING: THE BASICS



Example on git branching. Source: <https://www.nobledesktop.com/learn/git/git-branches>

GIT BRANCHING: THE BASICS

- Why is branching needed? (continued)
 - Allows a developer to switch among different tasks easily.
 - Suppose while Amith is working on feature A, he gets a request to attend to a major issue in the product X.
 - If Amith is working on his local master branch, he will now have to,
 - Undo all the changes he did.
 - Fix the issue.
 - Restart work on feature A.

GIT BRANCHING: THE BASICS

- If Amith had been working on a separate branch even if he was on his local repository, he could have,
 - Stopped the work on the feature and switch back to master/ main.
 - Created new branch for the “**hotfix**” and work on it.
 - Pushed the hotfix branch to the remote repo (and merge to remote master).
 - Switched back to his feature branch and continue where he left off.

GIT BRANCHING:THE BASICS

- Referring to the previous scenario,
 - Using git branches during development time is clearly more efficient rather than not using any.
 - Using branches is a must for collaborations with multiple developers.
 - Whatever you do on a branch is isolated from the rest of the code – great when you want to experiment!
 - It is not a bad idea to use branching for individual projects too.
- Branching on Git is a very lightweight operation. Therefore, use it whenever possible.

HOW DO WE CREATE A BRANCH ON GIT?

- Very simple.
 - `git branch <branch_name>` - just creates the branch based on your current branch.
 - `git checkout <branch_name>` - checks out the branch.

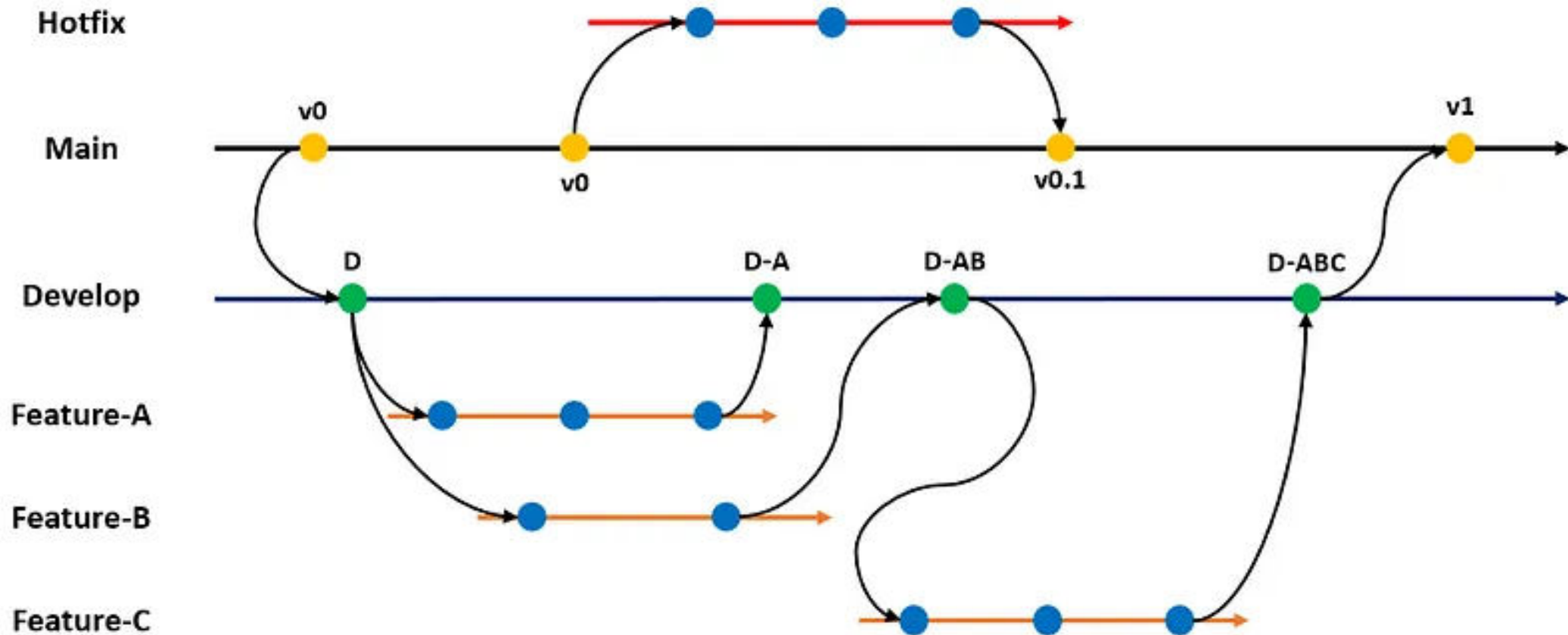
OR

- `git checkout -b <branch_name>` - creates the branch, and switches to the new branch (checks it out) from your current branch immediately.

WORKFLOWS (BRANCHING STRATEGIES)

- Git is very flexible when it comes to how-to branch. There is no one standard method, but some well accepted how-to branch 'recipes' are there.
- They are called workflows or branching strategies.
 - A Git workflow is a recipe or recommendation for how to use Git to accomplish work in a consistent and productive manner.
- Some popular workflows are,
 - **GitFlow**
 - **GitHub Flow**
 - GitLab Flow
 - Trunk Based Development (goes with CI/ CD)

GITFLOW WORKFLOW



GITFLOW WORKFLOW

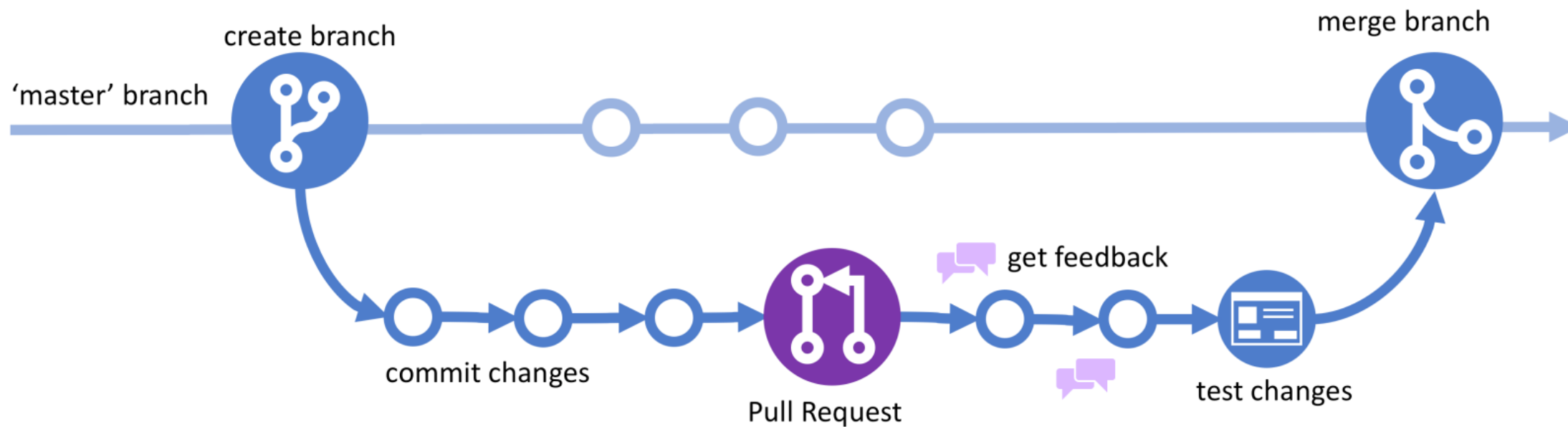
- GitFlow is very comprehensive.
 - [\[YouTube\] GitFlow Workflow](#)
 - Can become unnecessarily complex for small projects.
- It is more suited to Enterprise grade software development efforts.
 - Requires enforcement of the practices for the proper functioning of the workflow.
 - For most use cases, it is overkill.

GITHUB WORKFLOW

- A simpler alternative is preferable for most use cases.
 - One such workflow is **GitHub Flow** which can be used by anybody.
 - [\[YouTube\] GitHub Flow](#)

THE GITHUB FLOW

GitHub Flow



GITHUB FLOW

- This is a GitHub based simple workflow.
 1. Create a new feature branch from the main (master) branch.
 2. Make the necessary changes to the feature branch – continue until you think your feature is ready.
 3. Once you think the feature is done, ask for feedback from the other developers – create a Pull Request (PR) for this.
 4. Address their review comments received on the Pull Request (PR).
 5. Merge your Pull Request (PR) to the main branch.
 6. Delete your feature branch.

Refer the official documentation [here](#).

PULL REQUESTS (PR)

- A Pull Request (PR) is used to propose a new change to a repository (usually to the main branch).
- Usually, the proposed changes are in a feature branch, which ensures that the main branch will only contain code which is tested and working.
- The other collaborators will have to verify that the proposed change (PR) is OK first.
- If the PR is OK, they will accept it. Or else, they will suggest changes until it is acceptable.
- Once the PR is accepted, the changes in the feature branch are merged automatically to the main branch, bringing the proposed change to the main branch.
- Find the official documentation on Creating PRs [here](#).

GIT BRANCHING: BEST PRACTICES

1. Create a branch for each feature. Do not work on multiple features in a single branch.
2. Ideally, only one person should work on one feature branch.
3. Feature branches should be short-lived. They **should not exist** once the feature is complete.
4. Delete the local and remote feature branch after merging to the master branch (see point 3).

GIT BRANCHING: BEST PRACTICES

5. Merge early and often to avoid merge conflicts.
6. Keep the master branch clean. Only *production ready* code should be in the master branch. It is better to **avoid** working directly on the master branch.
7. Use a good naming convention to name your feature branches. Everyone should stick to the same naming convention.
 - feature/<username>/task-name-in-lowercase-spaces-replaced-with-dashes
 - feature/vishan.j/awesome-feature-x-to-our-app

GIT GENERAL BEST PRACTICES

1. Make small, incremental changes.

- Don't make huge changes in one go. If you have a big feature to add, break it down to smaller features and add one at a time.

2. Keep commits atomic.

- It is better if each commit is focused on one part of the feature. This will minimize the damage if you must revert a commit.

3. Commit often.

- Commit your changes often. Even if the work is not done.

4. Pull the changes in origin/main branch before you create a new branch from your local/main.

- This ensures that the new feature branch is created with all the latest changes in master. This reduces the possibility of merge conflicts later.

GIT GENERAL BEST PRACTICES

5. Push local feature branch commits to Remote often.
 - Even if it is an unfinished feature, push it. That's good for safekeeping.
6. Use branches.
 - See the section on branching for a comprehensive set of reasons.
7. Write good commit messages.
 - Make the lives of other Software Engineers, DevOps engineers, QA engineers easier.
8. Select one branching strategy (Git Workflow) and stick with it.
 - Select one that suits you and use it well.

SUMMARY

- Git branching
 - The basics
 - How do we create a branch on git?
 - Workflows (Branching strategies)
 - GitFlow
 - The GitHub Flow
 - Pull Requests (PRs)
 - Best practices for branching
- Git general best practices

ADDITIONAL TOPICS

Read on these topics to be up to date on some current trends.

- Continuous Integration and Continuous Deployment (CI/ CD).
- Compare Trunk based development workflow with GitFlow workflow.
 - Understand when to use which.
- Read on Pull Requests.
- Read on Git Rebase and see why it's a better alternative to Git Merge.

REFERENCES

1. [Pro Git - Scott Chacon and Ben Straub \[Free eBook\]](#)
2. [Dangit, Git!?! \[Git quick reference\]](#)
3. [Branch in Git](#)
4. [What Are the Best Git Branching Strategies](#)
5. [10 Git Branching Strategy Best Practices](#)
6. [Trunk-based Development vs. Git Flow](#)
7. [How to Use Git and Git Workflows – a Practical Guide](#)



THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK

SASINI.H@SLIIT.LK



WEB APPLICATION ARCHITECTURE – AN OVERVIEW

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

LEARNING OUTCOMES

- After completing this lecture, you will be able to,
 - Describe the main components of three tier architecture for web development.
 - Identify some suitable technologies to develop the presentation layer and the application layer of a web application.
 - Apply suitable architectural components to design the presentation layer and the application layer of a Three tier web application.

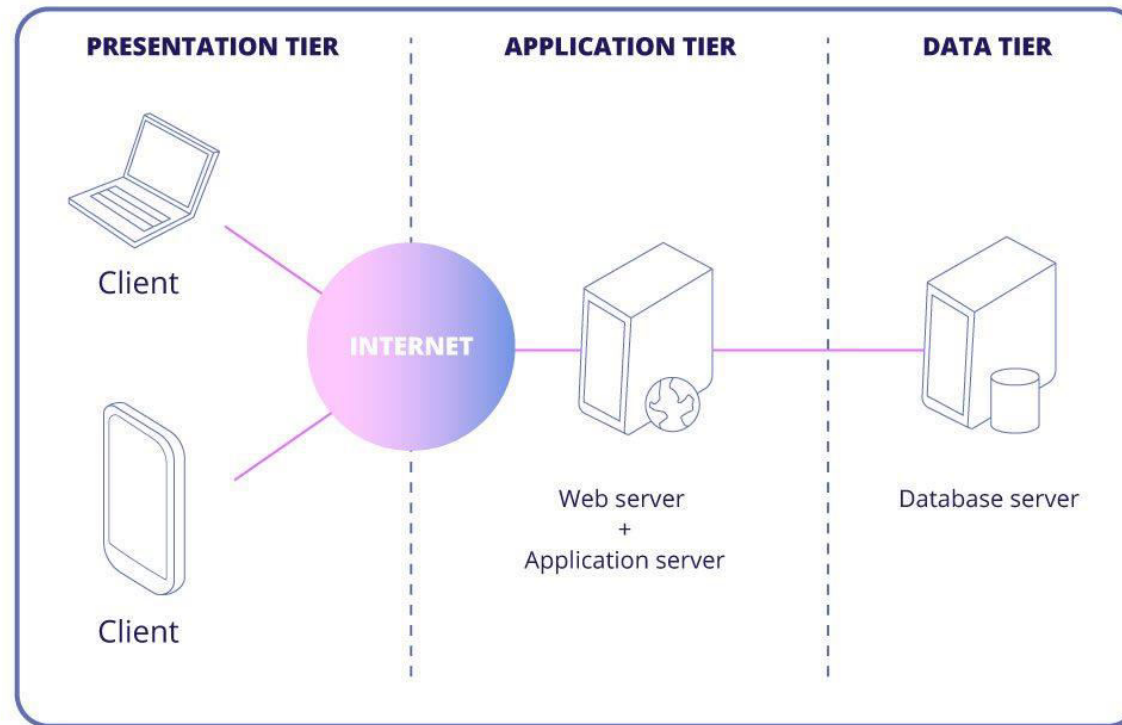
CONTENTS

- Website vs Web Application
- Three Tier/ Three Layer Architecture
- Presentation Layer
 - Some technologies
 - Architecting the Presentation Layer
- Application Layer
 - Some technologies
 - Application Programming Interface (API)
 - Architecting the Application Layer
 - Big Ball of Mud: an architectural anti-pattern
- Other Helpful Technologies

WEBSITE VS. WEB APPLICATION

- What is the difference between a Website and a Web Application?
 - A Website provides static content which can be consumed by the end users.
 - A **Web Application** is designed for **interaction** with the end users.
- Our focus is on **Web Applications**.

THREE TIER ARCHITECTURE



Source: <https://mobidev.biz/blog/web-application-architecture-types>

THREETIER ARCHITECTURE

- Modern web applications are engineered based on this architecture.
- Has three tiers.
 - Presentation tier/ layer
 - Application tier/ layer
 - Database tier/ layer
- The most popular form of n-tier (multitier) architecture.

THREETIER ARCHITECTURE

- Three tier architecture is a good choice for the development of web applications.
- Reasons?
 - Each tier/ layer runs on its own infrastructure.
 - Each tier/ layer can be developed independently/ in parallel by different teams.
 - Allows updating and scaling up/ down each tier independently without impacting the other tiers.
 - Better security when compared with two tier architecture (due to the isolation of data and logic from the presentation tier).

PRESENTATION LAYER

- Also called the client side or the frontend.
- Technologies used to implement include,
 - HTML/ CSS/ JavaScript
 - React.js
 - Angular
 - Vue.js
 - Some legacy technologies
 - jQuery and AJAX

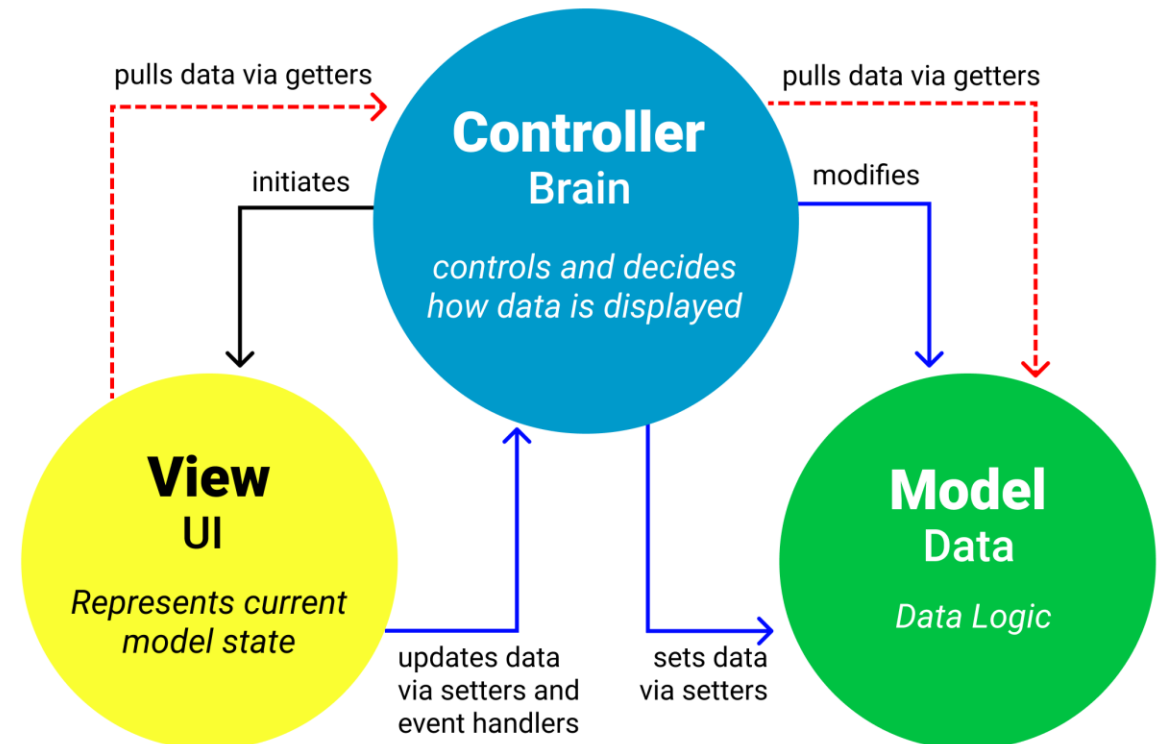
ARCHITECTING THE PRESENTATION LAYER

- Some well known examples
 - Model-view-controller (MVC)
 - Model-view-viewmodel (MVVM)
- Some more examples
 - Model-view-presenter (MVP)
 - Component Architecture
 - Micro frontends

ARCHITECTING THE PRESENTATION LAYER: MVC

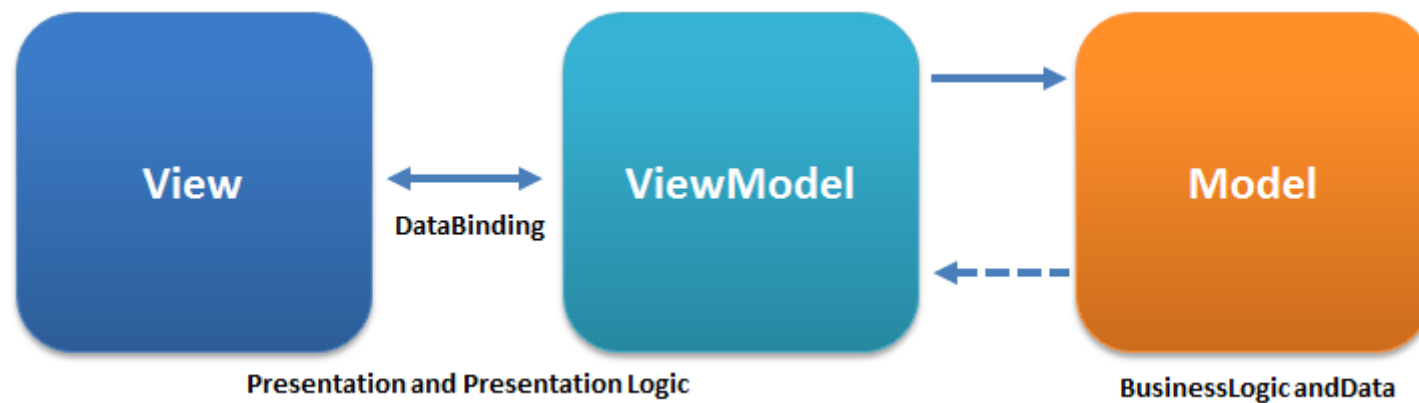
- [Youtube] MVC Explained

MVC Architecture Pattern



Source: <https://www.freecodecamp.org/news/the-model-view-controller-pattern-mvc-architecture-and-frameworks-explained/>

ARCHITECTING THE PRESENTATION LAYER: MVVM



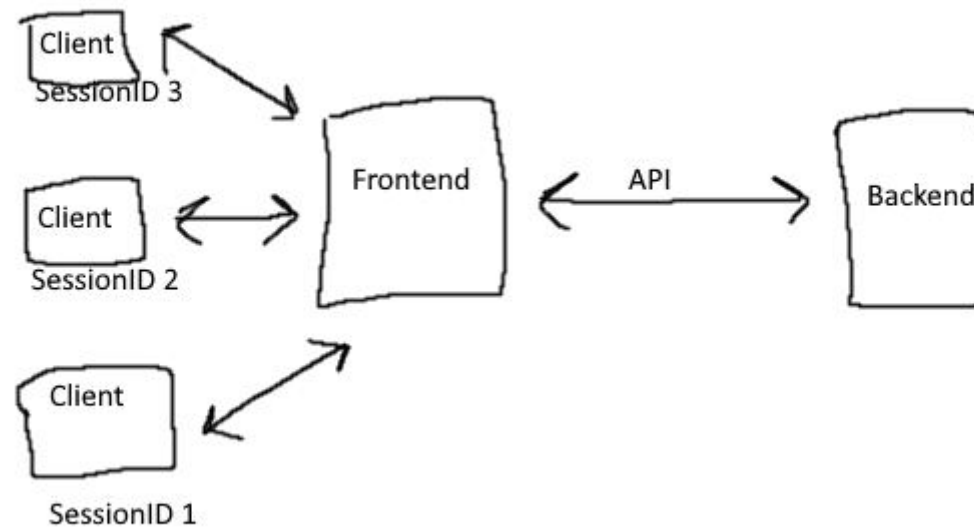
Source: <https://en.wikipedia.org/wiki/Model%E2%80%93view%E2%80%93viewmodel>

APPLICATION LAYER

- Also called the server side or the backend.
- Technologies used to implement include but not limited to,
 - Programming languages and frameworks
 - Java (frameworks: Spring/ Spring Boot)
 - Node.js (frameworks: express.js/ Koa)
 - Python (frameworks: Django/ Flask)
 - PHP (frameworks: Laravel, Symphony)
 - .Net Framework
 - Virtual Machines (VMs)
 - Serverless Computing
 - Containers

APPLICATION LAYER

- How does the Backend and the frontend communicate?



Source: <https://stackoverflow.com/q/71033687>

APPLICATION LAYER: APIS

- Application Programming Interface (API)!
- What is it?
 - A software interface which facilitates communication between two or more applications.
 - Abstracts away the complexities of application integrations.
 - A contract of services offered.

APPLICATION LAYER: APIS

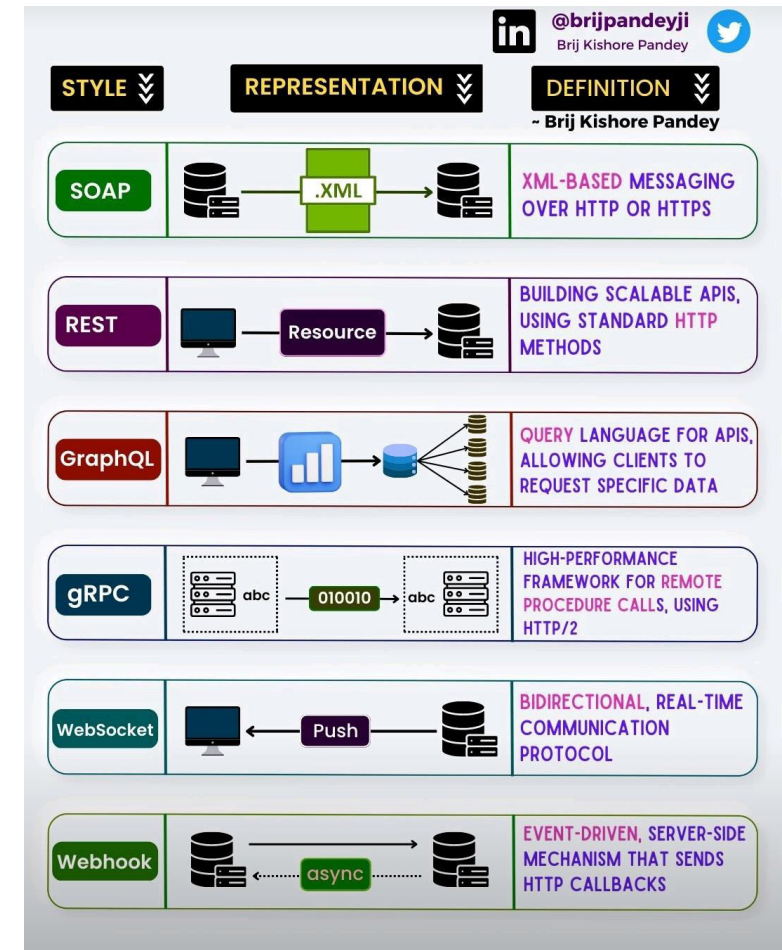
■ Benefits of APIs?

- developers don't have to create everything from scratch - can use existing functions exposed as an API resulting in more productivity.
- APIs significantly reduce development costs by reducing development efforts.
- Improves collaboration and connectivity across the ecosystem.

APPLICATION LAYER: APIS

- Different types of APIs?
 - Representational State Transfer API (REST API)
 - Simple Object Access Protocol API (SOAP API)
 - Remote Procedure Calls (RPC)
 - WebSockets
 - GraphQL APIs

More on some of these in a separate lecture.

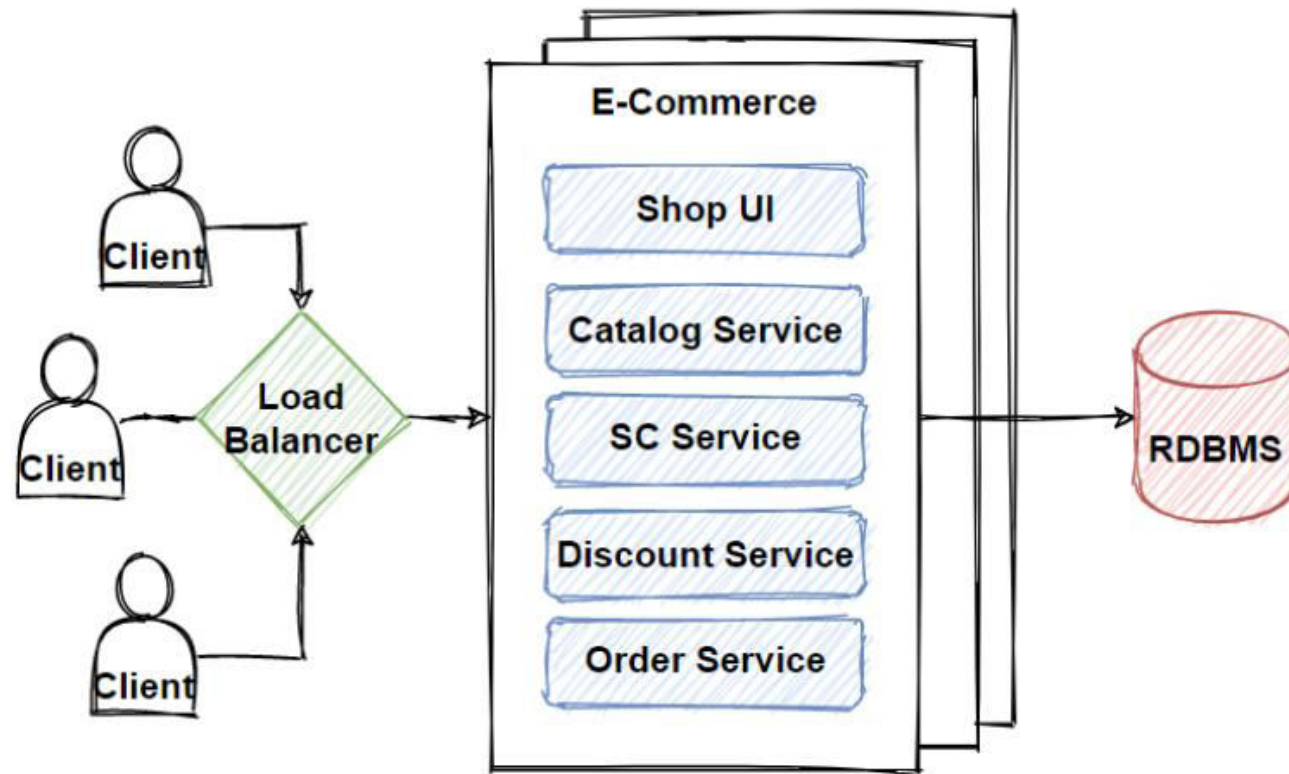


ARCHITECTING THE APPLICATION LAYER

- Some well known examples
 - Monolithic Architecture
 - Microservices Architecture
 - Service Oriented Architecture (SOA)
 - [\[Read\] What is SOA?](#)

ARCHITECTING THE APPLICATION LAYER: MONOLITHIC

Monolithic Architecture



Source: <https://medium.com/design-microservices-architecture-with-patterns/monolithic-architecture-is-still-worth-at-2021-98bfc112dc24>

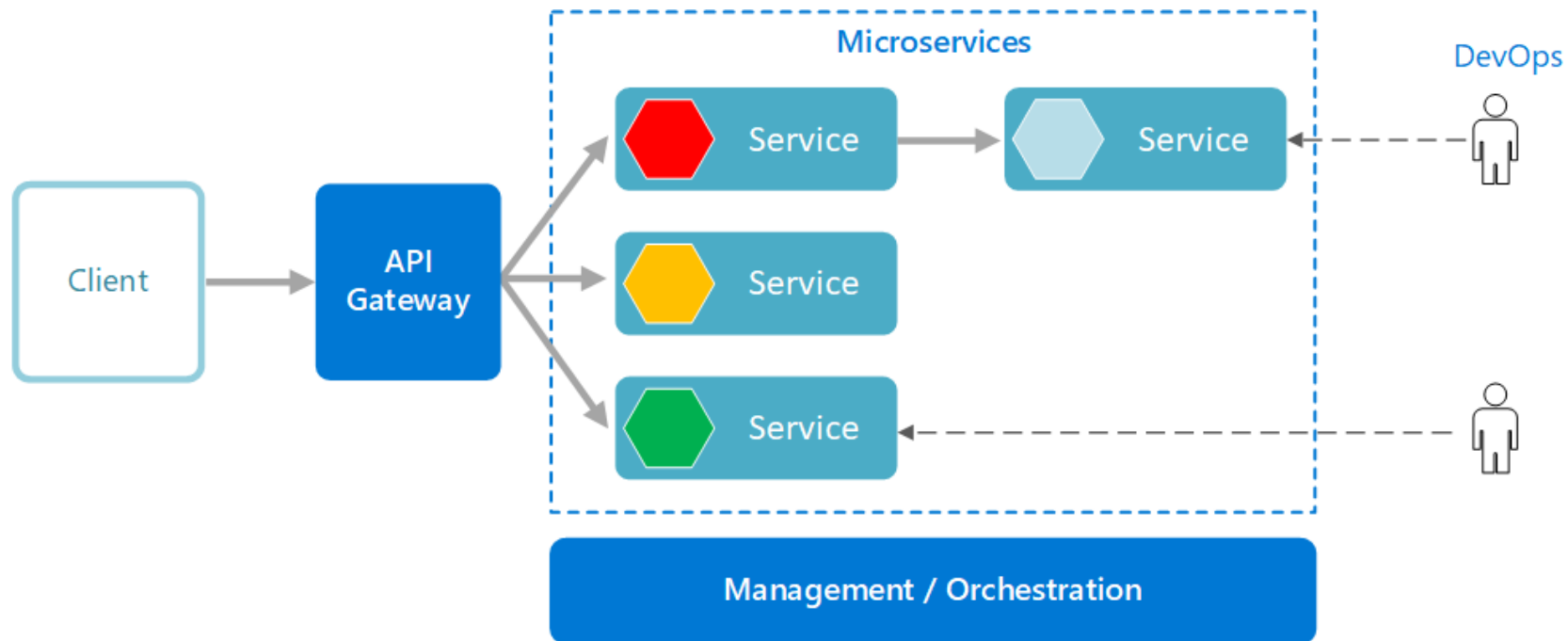
ARCHITECTING THE APPLICATION LAYER: MONOLITHIC

- Drawbacks in Monolithic Architecture?
 - Slower development speed due to **local complexity**.
 - Can't scale individual components.
 - Reliability – if there's an error in any module, it could affect the entire application's availability.
 - Barrier to technology adoption – any changes in the framework or language affects the entire application, making changes often expensive and time-consuming.
 - Lack of flexibility – a monolith is constrained by the technologies already used in the monolith.
 - A small change to a monolithic application requires the redeployment of the entire monolith.

ARCHITECTING THE APPLICATION LAYER: MICROSERVICES

- “Microservices architecture is an approach in which a single application is composed of many loosely coupled and independently deployable smaller services.” (Source – IBM)

ARCHITECTING THE APPLICATION LAYER: MICROSERVICES



Source: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>

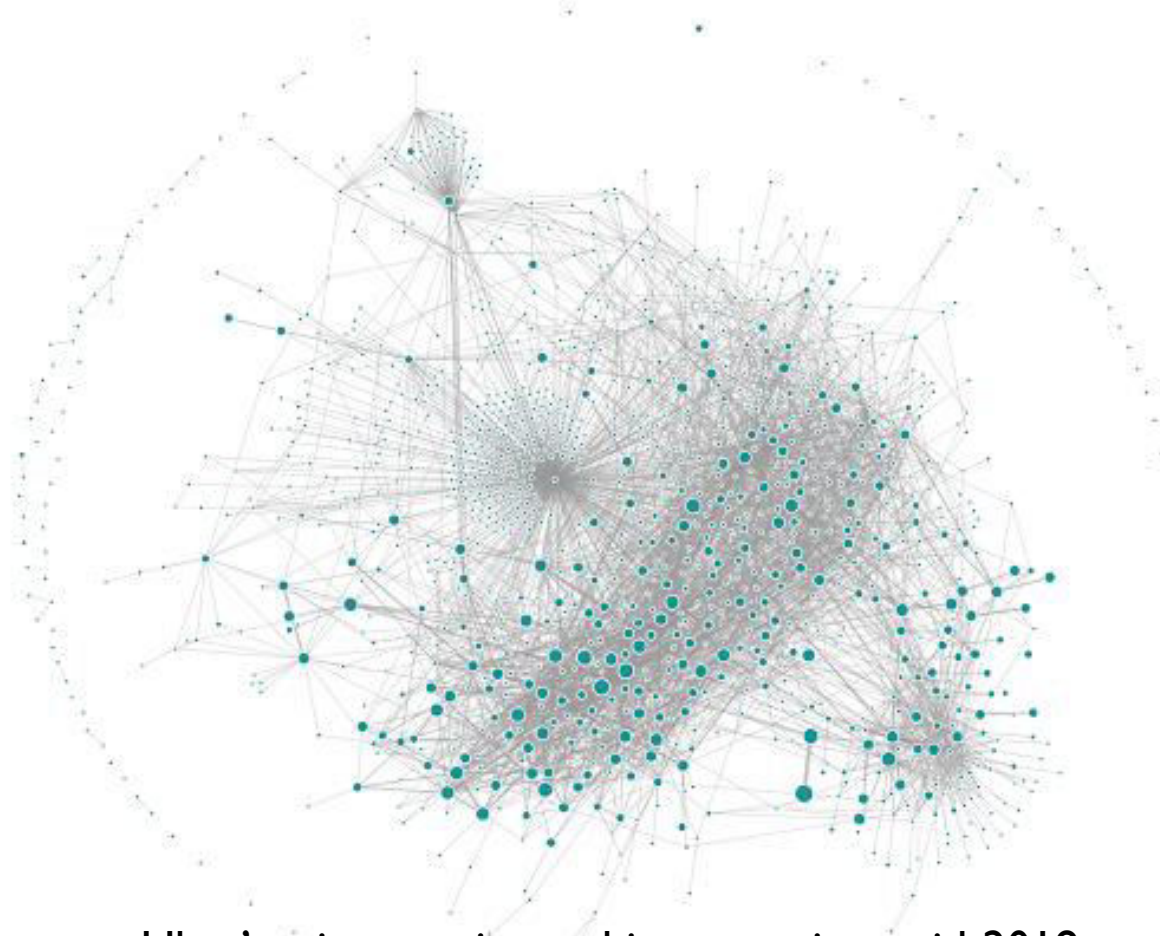
ARCHITECTING THE APPLICATION LAYER: MICROSERVICES

- Some benefits of Microservices
 - Microservices are **small, independent, and loosely coupled** (therefore, a single team can write/ maintain a single service).
 - Services can be **deployed independently** (can update an existing service without rebuilding/ redeploying the entire application).
 - Can **scale** each service independently.
 - High **reliability**.
 - Supports **polyglot programming**.

ARCHITECTING THE APPLICATION LAYER: MICROSERVICES

- Is everything great with Microservices?

ARCHITECTING THE APPLICATION LAYER: MICROSERVICES



Uber's microservice architecture circa mid-2018

Source: <https://www.uber.com/en-LK/blog/microservice-architecture/>

ARCHITECTING THE APPLICATION LAYER: MICROSERVICES

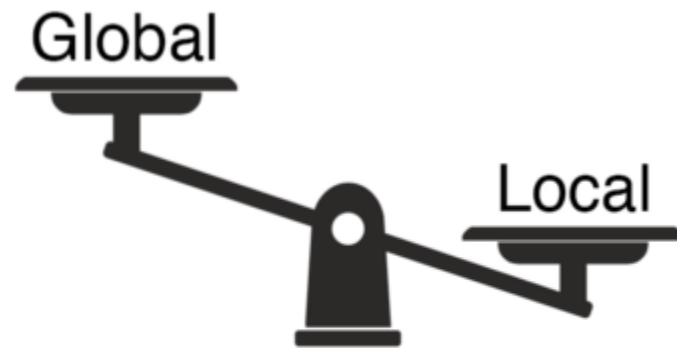
- **High global complexity** is the biggest problem when it comes to Microservices.
 - *Local complexity* depends on the **implementation of a service**; *global complexity* is defined by the **interactions and dependencies** between the services.
- Managing complexity is a constant challenge in Microservices.

ARCHITECTING THE APPLICATION LAYER: MICROSERVICES

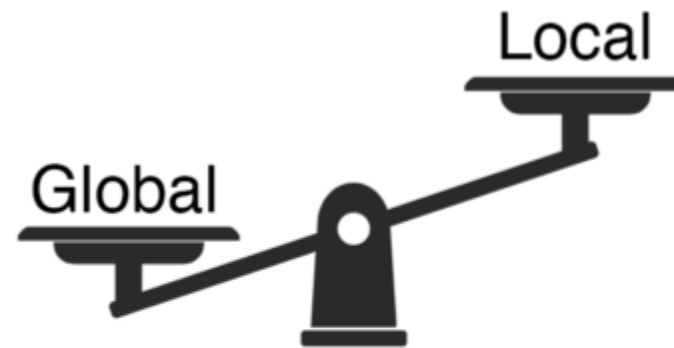
- What are some other downsides of Microservices?
 - High infrastructure costs
 - Debugging challenges
 - A lot more...



BIG BALL OF MUD:AN ARCHITECTURAL ANTI-PATTERN



Big Ball of Mud



Distributed
Big Ball of Mud

Source: <https://www.doit.com/untangling-microservices-or-balancing-complexity-in-distributed-systems/>

OTHER HELPFUL TECHNOLOGIES

- API Gateways (authentication, throttling, orchestration, caching, statistics etc.)
- Content Delivery Networks (CDN)
- Caching Tools (e.g., Redis)
- Load Balancers
- Message Queues
- Cloud Computing
 - Cloud Storage
 - Virtual Machines (VMs)
 - Many more resources...

SOME INTERESTING TOPICS

- Component Based Architecture
- Micro Frontends – Microservices for the frontend
- BFF: Backend For Frontend also known as BIF: Backend In Frontend

REFERENCES

1. [Web application vs. website: finally answered](#)
2. [What is three-tier architecture?](#)
3. [SOAP APIs vs REST APIs](#)
4. [Micro Frontends: What are They and When to Use Them?](#)
5. [Model-view-presenter](#)
6. [An In-Depth Guide to Component-Based Architecture: Features, Benefits, and more](#)
7. [The Complete Guide To BFF \(Backend For Frontend\)](#)
8. [Microservices vs. monolithic architecture](#)
9. [Untangling Microservices or Balancing Complexity in Distributed Systems](#)
10. [Big Ball of Mud - The Daily Software Anti-Pattern](#)



THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK



REST APIS

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

LEARNING OUTCOMES

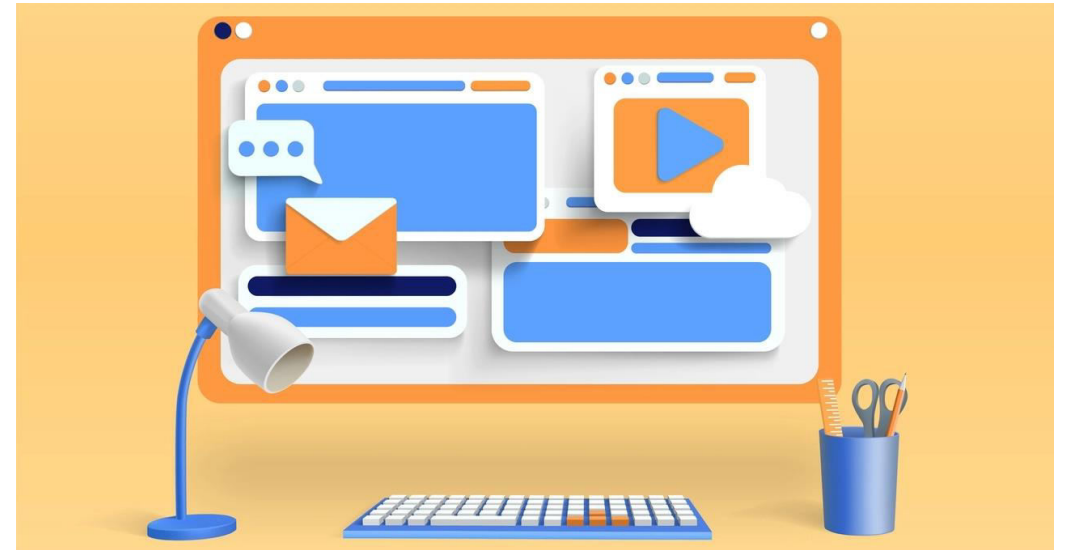
- After completing this lecture, you will be able to,
 - Describe the six REST architectural constraints.
 - Apply the knowledge gained in this lecture in designing proper REST APIs.
 - Evaluate if an API is a REST API or not.
 - Evaluate a given API to identify how much it conforms to the REST constraints using the Richardson Maturity Model.

CONTENTS

- Consuming web resources - Human vs. programs
- Application Programming Interface (API)
 - Examples
- REST
 - Six Architectural Constraints of REST
 - True REST vs. RPC (Remote Procedure Calls)
 - Richardson Maturity Model
- Some Facilitators of REST
 - HTTP, JSON, HAL
- Best Practices

CONSUMING WEB RESOURCES - HUMAN VS. PROGRAMS

- How do you consume web resources?
 - Web browsers
 - Web applications
 - Mobile applications etc.



Source: <https://illuminatingfacts.com/a-timeline-of-the-rise-and-fall-of-popular-web-browsers/>

CONSUMING WEB RESOURCES - HUMAN VS. PROGRAMS

- How do programs consume web resources?
 - Web APIs!
 - API stands for **Application Programming Interface**



```
... item = el->FirstChildElement();  
GroupDesc::ElementDesc elDesc;  
  
std::string sp_name = item->Attribute("sp_name");  
std::string spritename = item->Attribute("spritename");  
  
float x = boost::lexical_cast<float>(item->Attribute("x"));  
float y = boost::lexical_cast<float>(item->Attribute("y"));  
float offset = boost::lexical_cast<float>(item->Attribute("offset"));  
unsigned layer = 50; // default  
if (item->Attribute("layer") != NULL)  
{  
    layer = boost::lexical_cast<unsigned>(item->Attribute("layer"));  
}  
  
elDesc.name_ = sp_name;  
elDesc.spriteName_ = spritename;  
elDesc.x_ = x;
```

Source: <https://stileex.xyz/en/difference-computer-program-software/>

APPLICATION PROGRAMMING INTERFACE (API)

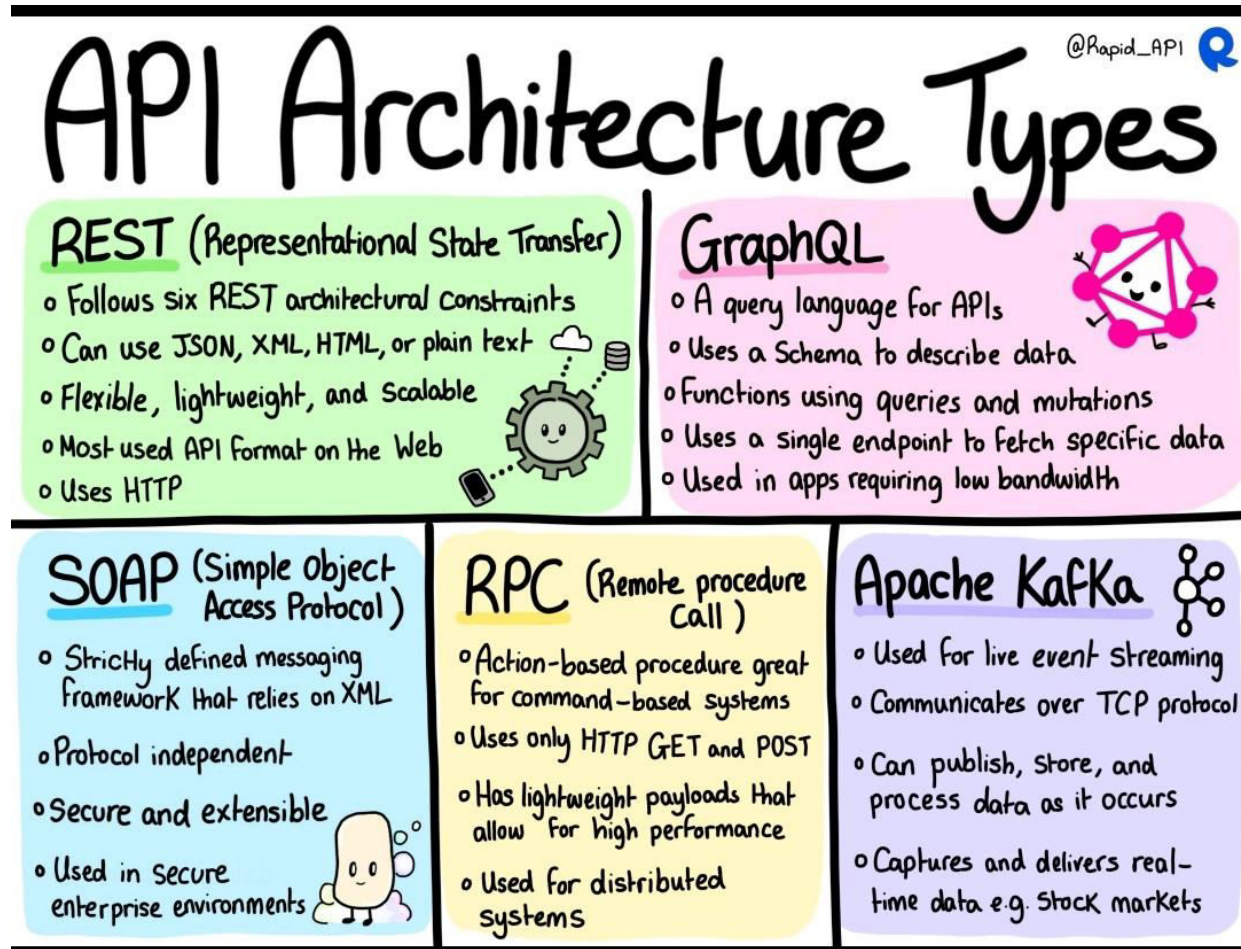
- What is an API?
 - A software interface which facilitates communication between two or more applications.
 - Abstracts away the complexities of application integrations.
 - A contract of services offered.

APPLICATION PROGRAMMING INTERFACE (API)

■ Examples:

- CORBA Implementations (Common Object Request Broker Architecture)
 - SOAP (Simple Object Access Protocol)
 - RPC (Remote Procedure Calls)
 - **REST** (Representational State Transfer)
- 
- Mostly legacy technologies

APPLICATION PROGRAMMING INTERFACE (API)



REST (REPRESENTATIONAL STATE TRANSFER)

- What is REST?
 - an **architectural style** for creating web services.
 - Introduced by Roy Fielding in 2000 in his now famous Doctoral dissertation.
 - Arguably the most popular approach to creating Web APIs now.



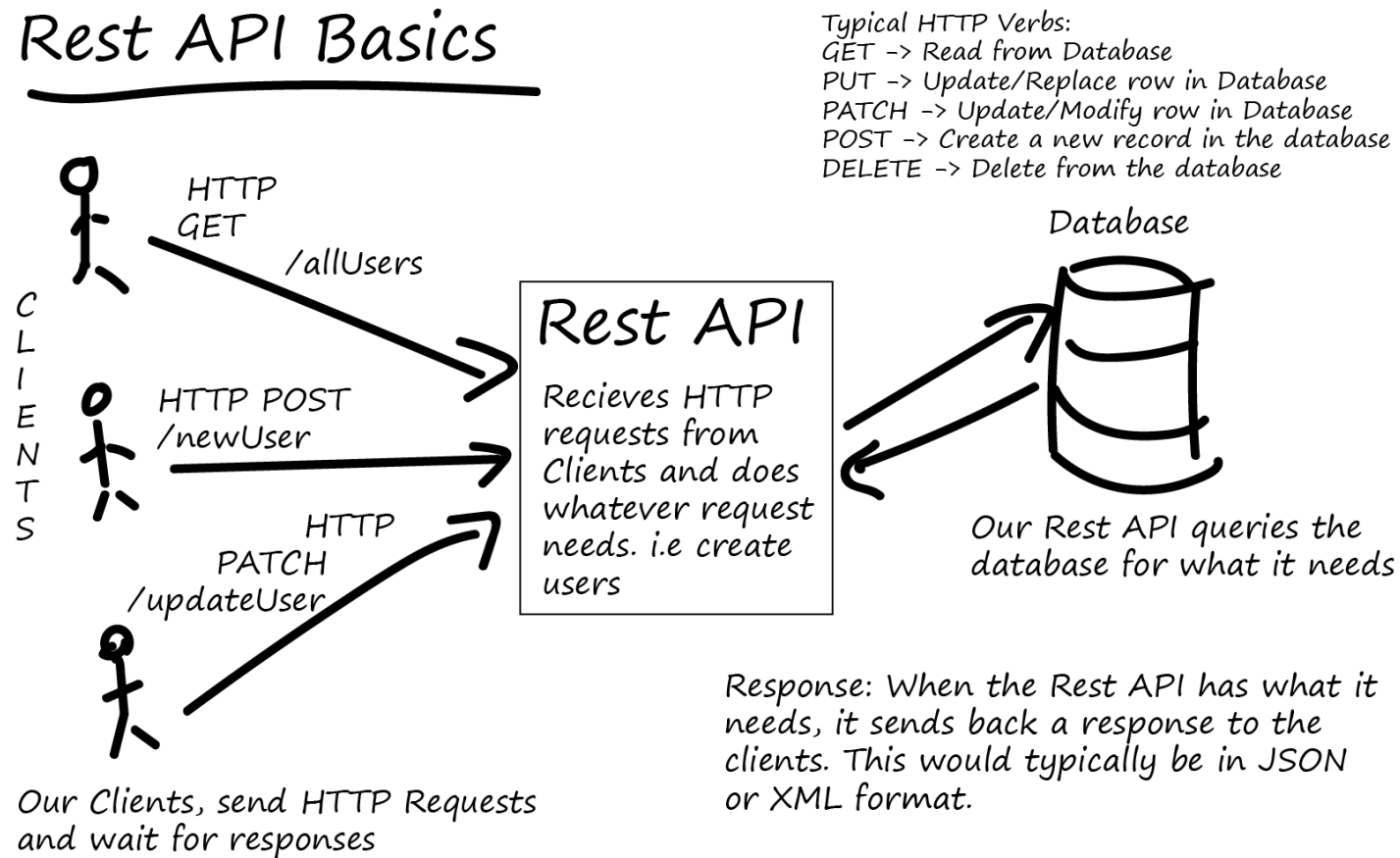
Source: https://medium.com/@liams_o/15-fundamental-tips-on-rest-api-design-9a05bcd42920

REST (REPRESENTATIONAL STATE TRANSFER)

- Why is REST Popular?
 - Just an **architectural style** (not a technology or a full-blown protocol like SOAP)
 - If the API aligns with the six REST constraints, it is a REST API
 - No strict rules, very flexible
 - Simple to use and learn compared with previously used approaches (e.g., SOAP/ CORBA etc.)
 - Runs on HTTP Protocol

REST (REPRESENTATIONAL STATE TRANSFER)

Rest API Basics



REST (REPRESENTATIONAL STATE TRANSFER)

- An example
 - A proper REST response to a GET call to <http://localhost:8080/employees/1> REST endpoint.

```
{
  "id": 1,
  "firstName": "Bilbo",
  "lastName": "Baggins",
  "role": "burglar",
  "name": "Bilbo Baggins",
  "_links": {
    "self": {
      "href": "http://localhost:8080/employees/1"
    },
    "employees": {
      "href": "http://localhost:8080/employees"
    }
  }
}
```

Source: <https://spring.io/guides/tutorials/rest/>

REST (REPRESENTATIONAL STATE TRANSFER)

■ Design Principles

- REST is designed to facilitate **strong decoupling** between the client and the server to facilitate internet-scale usage.
- Follows a Request/ Response style of communication.
- Some Terms:
 - Client – the consumer of services via the API
 - Server – the provider of services and the API that exposes these services
 - Resource – any content that the server responds with to a client request

REST (REPRESENTATIONAL STATE TRANSFER)

- Design Principles (contd.)
 - When a **client** requests for a **resource**, the **server** will respond with
 - the **representation** of the resource in its current **state** (Usually JSON or XML)
 - and relevant **hypermedia links (URIs)** that can be used to change the **state** of the resource

REST: SIX ARCHITECTURAL CONSTRAINTS

- Client-server architecture
- Stateless
- Cacheable
- Uniform Interface
- Layered system
- Code on demand (Optional)

Let's examine each constraint.

REST: SIX ARCHITECTURAL CONSTRAINTS

- Client-server architecture
 - Maintain the clear separation of the client and the server,
 - To promote portability of the user interface
 - To promote low coupling between the client and the server
 - To improve scalability of the server components
 - To allow clients and servers grow independently

REST: SIX ARCHITECTURAL CONSTRAINTS

■ Stateless

- Communication between clients and server must be stateless in nature.
- Each request from client to server must contain all the information necessary to understand the request and cannot take advantage of any stored context on the server.
- Session state is kept entirely on the client.
- Improves visibility, reliability, and scalability.

REST: SIX ARCHITECTURAL CONSTRAINTS

■ Cacheable

- Cache constraints require that the data within a response to a request be implicitly or explicitly labeled as cacheable or non-cacheable.
- If a response is cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests.
- Caching can eliminate requirement for some interactions thereby improving efficiency, scalability, and user-perceived performance.

REST: SIX ARCHITECTURAL CONSTRAINTS

- Uniform Interface
 - Implementations are decoupled from the services they provide, which encourages independent evolvability.
 - Continued in the next slide...

REST: SIX ARCHITECTURAL CONSTRAINTS

■ Uniform Interface

- Has four sub-constraints.
 - **Self-descriptive messages** - Each message going back and forth between the client and the server should contain enough information to process the message.
 - **Identification of resources** - Individual resources are identified in requests/ responses. This is done through URIs (Uniform Resource Identifiers). The resources themselves on the server are conceptually separate from the representations that are returned to the client.
 - **Resource manipulation through representations** - When a client holds a representation of a resource, including any metadata attached, it has enough information to modify or delete the resource's state.
 - **Hypermedia as the engine of application state (HATEOAS)** – continued in the next slide...

REST: SIX ARCHITECTURAL CONSTRAINTS

- Uniform Interface (continued...)
 - **Hypermedia as the engine of application state (HATEOAS)** - (Hey-tee-oh-s)
 - Having accessed an initial URI for the REST application a REST client should then be able to use server-provided links dynamically to discover all the available resources it needs.
 - As access proceeds, the server responds with text that includes hyperlinks to other resources that are currently available (Dynamic API).
 - There is **no need** for the client to be hard-coded with information regarding the structure of the server.
 - This is akin to a human user having to know only the main URL of a website. The rest of the URLs are discovered when consuming the site.

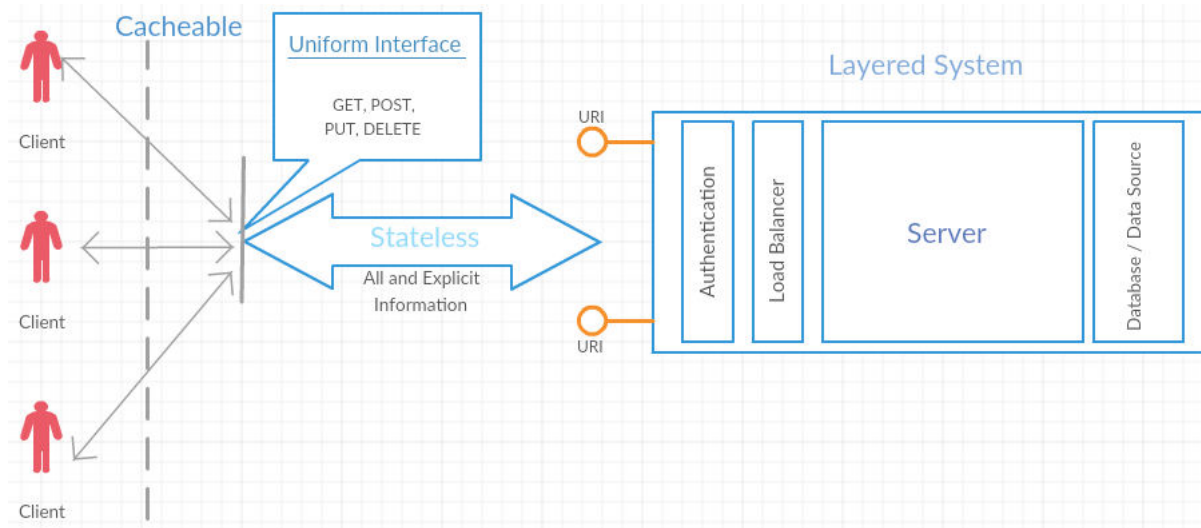
REST: SIX ARCHITECTURAL CONSTRAINTS

■ Layered system

- The layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior such that each component cannot "see" beyond the immediate layer with which they are interacting.
- The layered system constraints enable network-based intermediaries such as proxies and gateways to be transparently deployed between a client and server using the Web's uniform interface.
 - A network-based intermediary will intercept client-server communication for a specific purpose. Network-based intermediaries are commonly used for enforcement of security, response caching, and load balancing.

REST: SIX ARCHITECTURAL CONSTRAINTS

- Layered system (continued...)



Source: REST API Design Rulebook (Mark Masse, Oreilly, 2011)

REST: SIX ARCHITECTURAL CONSTRAINTS

- Code on demand (Optional)
 - REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts.
 - This simplifies clients by reducing the number of features required to be pre-implemented.
 - This is the **only optional constraint** in REST.

TRUE REST VS. RPC (REMOTE PROCEDURE CALLS)

- Pretty URLs like /employees/10 aren't REST.
- Merely using HTTP methods such as GET, POST etc. is not REST.
- Having all the CRUD operations laid out isn't REST.
- In such an API there is no way to know **how to interact with the service** without explicit help from the developers.
 - HATEOAS constraint is missing!

TRUE REST VS. RPC (REMOTE PROCEDURE CALLS)

The response to a GET call to <http://localhost:8080/employees/1> endpoint.

```
{
  "id": 1,
  "name": "Bilbo Baggins",
  "role": "burglar"
}
```

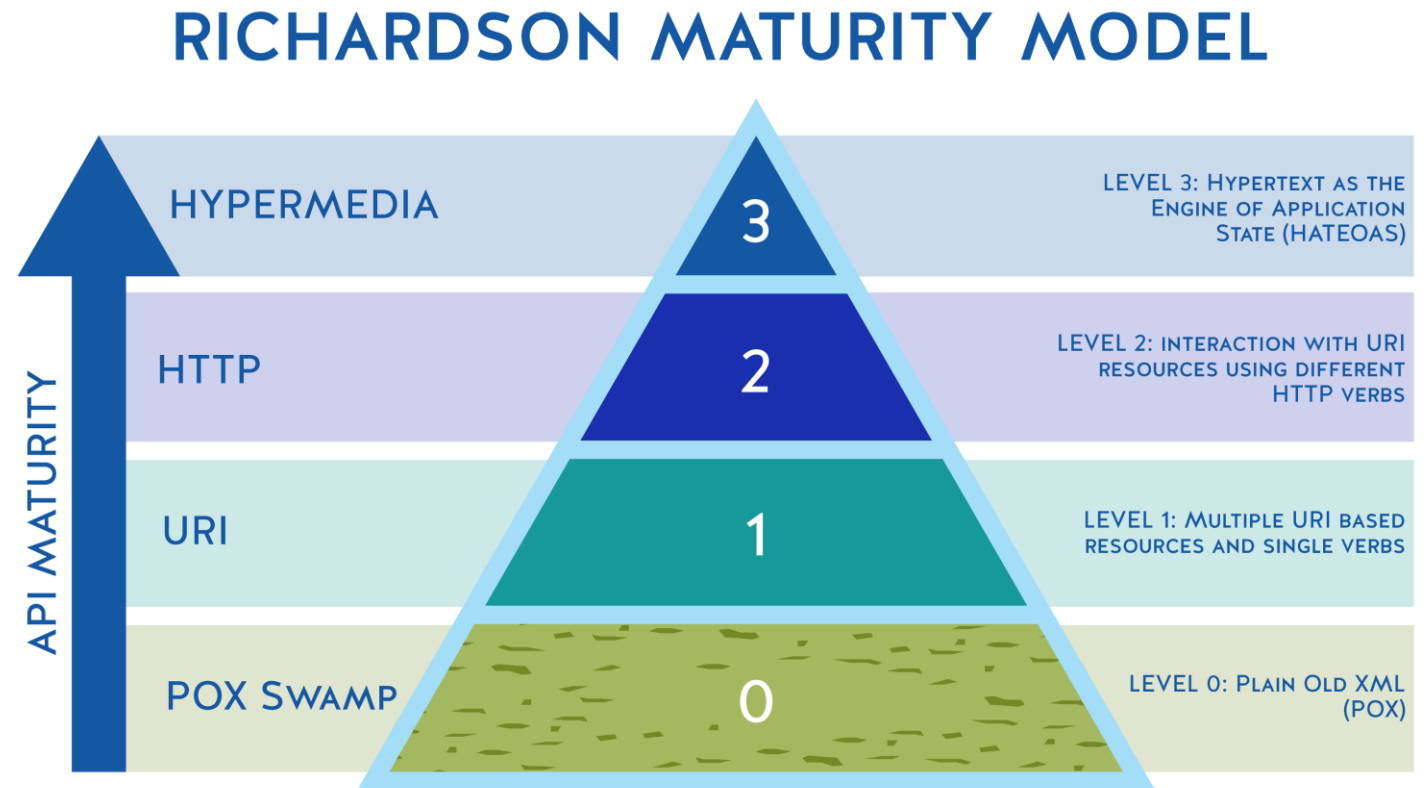
Just a JSON entity. No hyperlinks – Results in RPC

```
{
  "id": 1,
  "name": "Bilbo Baggins",
  "role": "burglar",
  "_links": {
    "self": {
      "href": "http://localhost:8080/employees/1"
    },
    "employees": {
      "href": "http://localhost:8080/employees"
    }
  }
}
```

RESTful representation of the same entity

RICHARDSON MATURITY MODEL

- It is a four-level scale that indicates extent of API conformity to the REST constraints.
- Proposed by Leonard Richardson.



RICHARDSON MATURITY MODEL

- Level-0: Swamp of POX
 - Usually exposes just one URI for the entire application. Uses HTTP POST for all actions, even for data fetch. SOAP or XML-RPC-based applications come under this level. POX stands for Plain Old XML. Could be JSON too.
- Level-1: Resource-Based Address/URI
 - Introduces resources and allows to make requests to individual URIs (still all typically POST) for separate actions instead of exposing one universal endpoint (API).
- Level-2: HTTP Verbs (Methods)
 - Fully utilize all HTTP commands or verbs such as GET, POST, PUT, and DELETE.
- Level-3: HATEOAS
 - Full maturity. Utilizes URI, HTTP Methods and HATEOAS.

SOME FACILITATORS OF REST

■ HTTP

- HTTP Methods – REST APIs utilize these to indicate the type of operation being performed.
 - GET – Reading a resource
 - POST – Writing a resource
 - PUT – Updating a resource
 - DELETE – Deleting a resource
- HTTP Status Codes – REST APIs utilize these to denote the status of a response to a request.
 - 2XX – Successful responses range (e.g., 200 OK)
 - 4XX – Client errors range (e.g., 404 NOT FOUND)
 - 5XX – Server errors range (e.g., 500 INTERNAL SERVER ERROR)
 - List of HTTP status codes: [HTTP response status codes](#)

SOME FACILITATORS OF REST

- HTTP (continued...)
 - HTTP Headers – These are used to pass along additional information/ meta data between the client and the server.
 - Content-type
 - Cache-control
 - Authorization
 - List of HTTP Headers: [HTTP headers](#)
- It is important that proper HTTP methods, Headers and Status Codes are used in REST requests/ responses.

SOME FACILITATORS OF REST

- JSON (JavaScript Object Notation) – A lightweight human readable format for storing and transporting data as key-value pairs.
- HAL (Hypertext Application Language) – An open specification that provides a structure to represent RESTful resources

```
{
  "id": 1,
  "firstName": "Bilbo",
  "lastName": "Baggins",
  "role": "burglar",
  "name": "Bilbo Baggins",
  "_links": {
    "self": {
      "href": "http://localhost:8080/employees/1"
    },
    "employees": {
      "href": "http://localhost:8080/employees"
    }
  }
}
```

Source: <https://spring.io/guides/tutorials/rest/>

SOME BEST PRACTICES

- Use correct HTTP methods, headers and status codes according to the operation being performed.
- Accept and respond with JSON.
- Always use proper authentication and authorization mechanisms (e.g., OAuth, JWT etc.)
- Give meaningful names to the URIs.
 - E.g.,
 - <https://abcd.com/users/>
 - <https://abcd.com/users/2>
- Keep the API consistent.

MORE BEST PRACTICES

- Self study activity
 - Read and follow this list of best practices from Microsoft: [RESTful web API design](#)
 - Make sure you apply these in your assignment as well!
- More hands-on experience on REST APIs during the practical session.

SUMMARY

- Consuming web resources - Human vs. programs
- Application Programming Interface (API)
 - Examples
- REST
 - Six Architectural Constraints of REST
 - True REST vs. RPC (Remote Procedure Calls)
 - Richardson Maturity Model
- Some Facilitators of REST
 - HTTP, JSON, HAL
- Best Practices

REFERENCES

1. An Introduction to Representational State Transfer (REST)
2. R.T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures", University of California, Irvine, 2000
3. Mark Masse, "REST API Design Rulebook", O'Reilly Media, Inc., 2011
4. Representational state transfer
5. Richardson Maturity Model
6. RESTful web API design



THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK



REST APIS - AUTHENTICATION AND AUTHORIZATION

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

LEARNING OUTCOMES

- After completing this lecture, you will be able to,
 - Know the difference between authentication and authorization.
 - Apply different authentication and authorization mechanisms for REST APIs as appropriate to given scenarios.

CONTENTS

- Authentication vs. Authorization
- Importance of Authentication and Authorization
- Some methods for Authentication and Authorization in REST APIs
 - HTTP Basic Authentication
 - API Keys
 - JSON Web Tokens (JWT)
 - OAuth 2.0
 - OpenID Connect (OIDC)
- Summary

AUTHENTICATION VS. AUTHORIZATION

- Should everyone have free access to your resources?
 - No!
 - Only known users should.
- How do we identify if a user is a known user?
 - **Authentication!**
 - [YouTube] Session vs Token Authentication

AUTHENTICATION VS. AUTHORIZATION

- Should all known users be allowed to access all resources?
 - No!
 - They should only have access to what we explicitly want them to access.
- How do we control what they can access?
 - **Authorization!**

AUTHENTICATION VS. AUTHORIZATION



Authorization

What you can do



Authentication

Who you are

Source: <https://blog.restcase.com/4-most-used-rest-api-authentication-methods/>

AUTHENTICATION VS. AUTHORIZATION

- **Authentication** is when an entity proves an **identity**.
 - Authentication proves that you are who you say you are.
- **Authorization** is when an entity proves a **right to access**.
 - Authorization proves you have the right to make a request.
- In summary:
 - Authentication: Refers to proving correct identity.
 - Authorization: Refers to allowing a certain action.

AUTHENTICATION VS. AUTHORIZATION

Authentication

Determines whether users are who they claim to be

Challenges the user to validate credentials (for example, through passwords, answers to security questions, or facial recognition)

Usually done before authorization

Generally, transmits info through an ID Token

Generally governed by the OpenID Connect (OIDC) protocol

Example: Employees in a company are required to authenticate through the network before accessing their company email

Authorization

Determines what users can and cannot access

Verifies whether access is allowed through policies and rules

Usually done after successful authentication

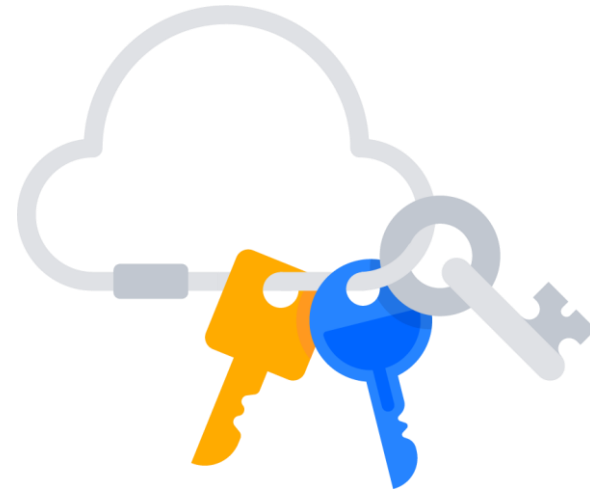
Generally, transmits info through an Access Token

Generally governed by the OAuth 2.0 framework

Example: After an employee successfully authenticates, the system determines what information the employees are allowed to access

SOME METHODS FOR AUTH IN REST APIS

- HTTP Basic Authentication
- API Keys
- JSON Web Tokens (JWT)
- OAuth 2.0
 - OpenID Connect (OIDC)



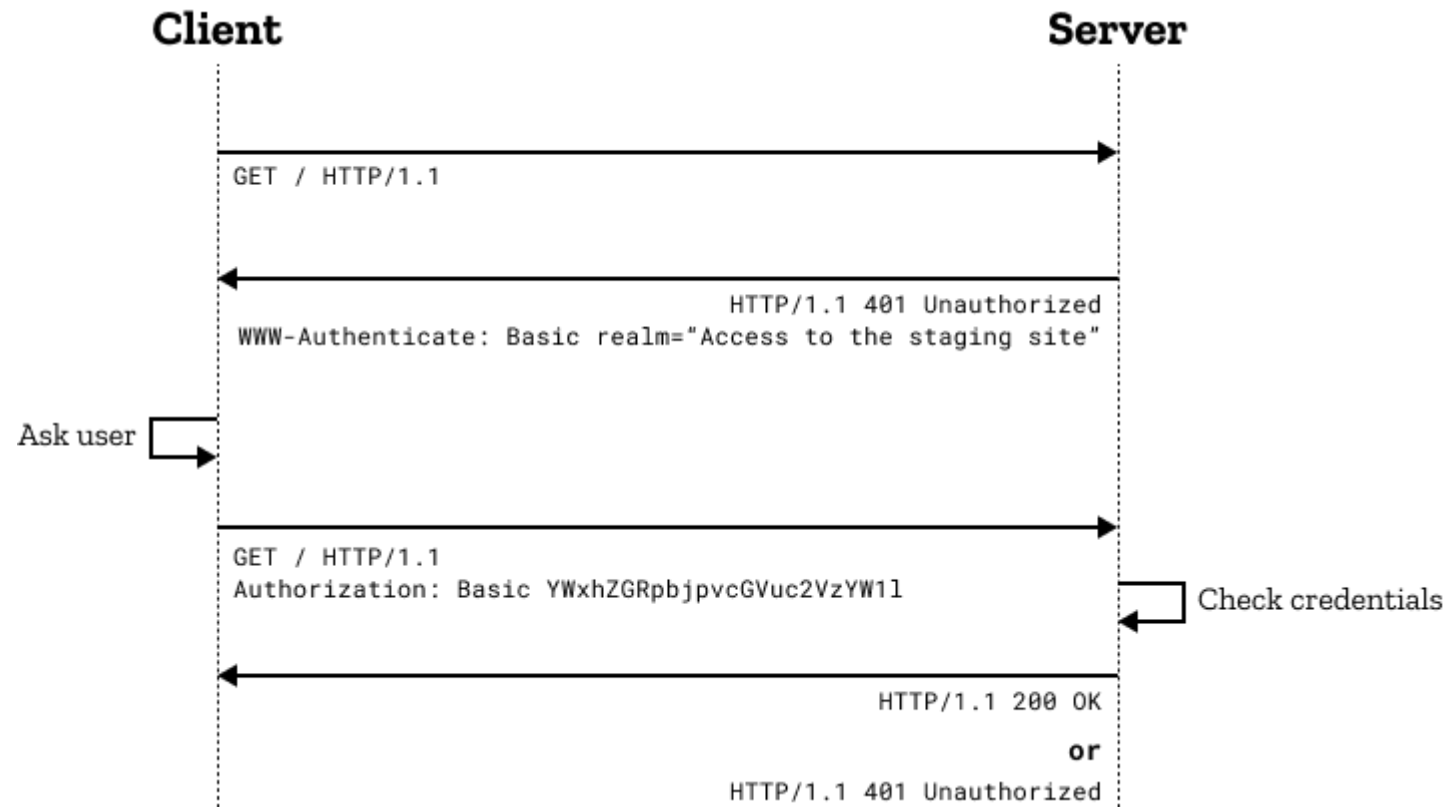
HTTP BASIC AUTHENTICATION

- Most basic form of authentication in APIs.
- The client sends the username and the password of the user encoded in Base64 encoding technique to the server.
- The credentials are sent in the HTTP header with every request.
 - *Authorization: Basic <username:password>*

HTTP BASIC AUTHENTICATION

- Very simple to use as it does not require cookies, session identifiers, login pages.
- Lightweight.
 - Good for use-cases like IoT.
- Rarely recommended due to its inherent security vulnerabilities.
 - For better security use with HTTPS over SSL/ TLS.

HTTP BASIC AUTHENTICATION



Source: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication>

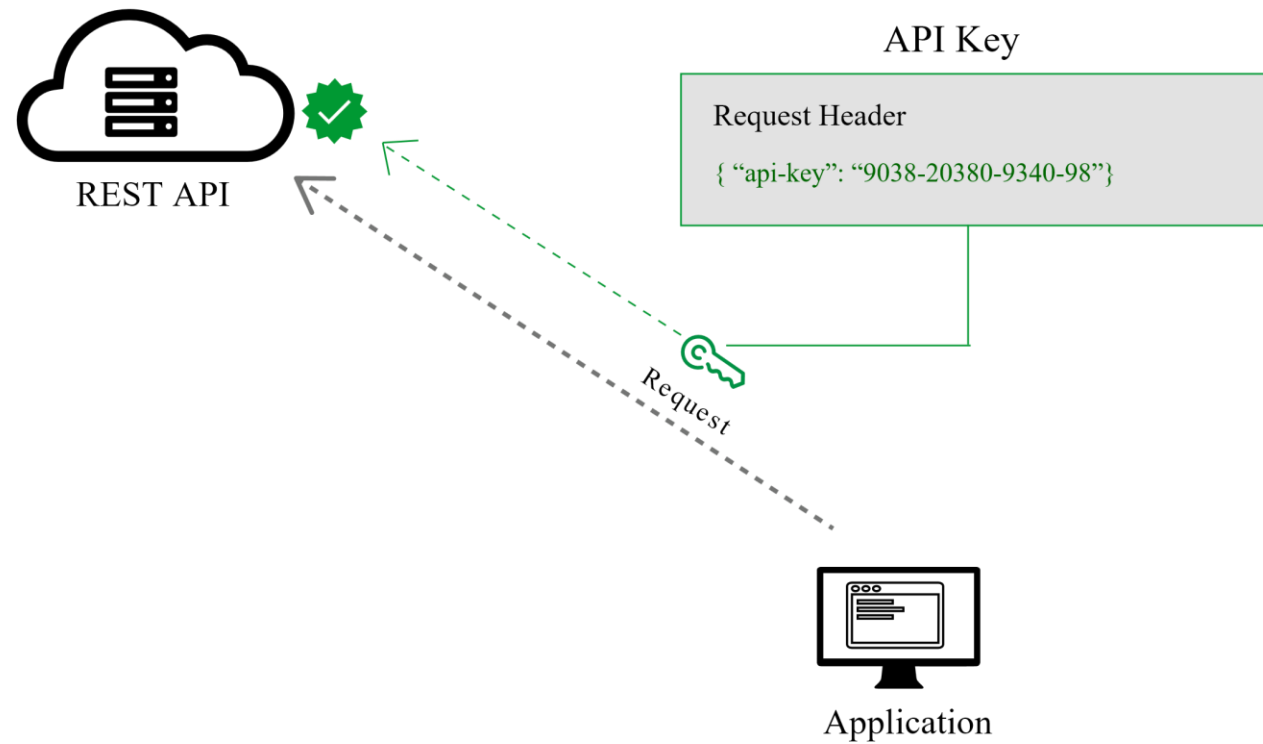
API KEYS

- A **unique generated key value** is assigned for each first-time user.
- Thereafter, the user sends all requests with the key as,
 - a query parameter or
 - in the HTTP header (the standard way)
- Simple, fast and convenient.

API KEYS

- A method of authentication, not authorization.
- No expiration.
 - Once the key is stolen, it may be used indefinitely unless revoked.
- As with Basic HTTP authentication, not recommended to be used as is.
 - For better security use with HTTPS over SSL/ TLS.

API KEYS

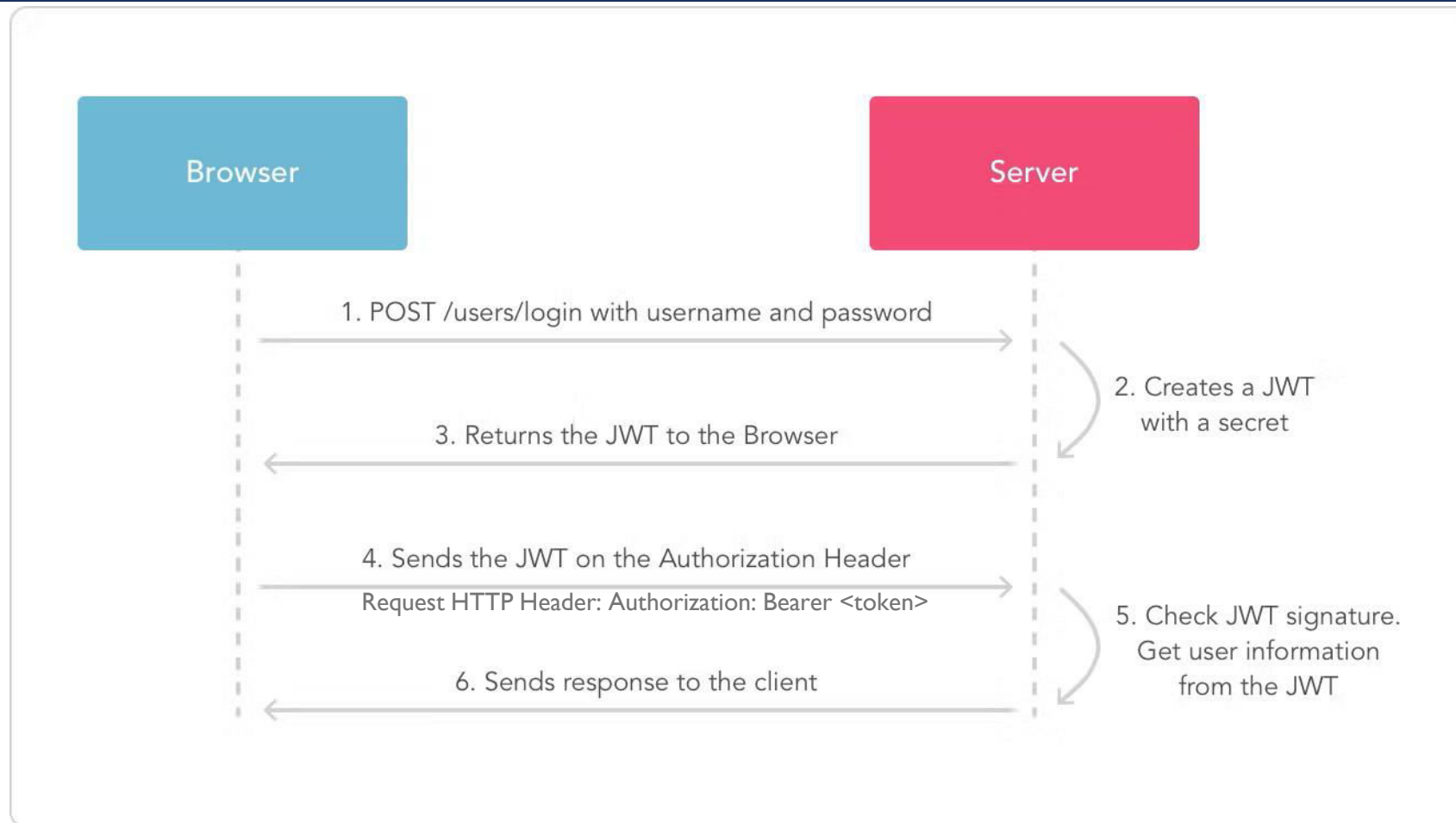


Source: <https://blog.restcase.com/4-most-used-rest-api-authentication-methods/>

JSON WEB TOKENS (JWT)

- A compact and a self-contained means to securely exchange information between parties as a JSON object.
- Digitally signed, therefore can be verified and trusted.
 - Using secrets
 - Using public/ private key pairs
- Used for user **authorization** after authentication.
- Very popular!

JSON WEB TOKENS (JWT)



JSON WEB TOKENS (JWT)

- JWT Decoder – example JWT demo
- JWT guarantees that data is not tampered because of the signature.
- The contents of a JWT can be seen by anyone that intercepts the token.
 - It is just serialized, not **encrypted**.
 - It is recommended to **use HTTPS** to avoid this problem.
- JWT is pronounced as “Jot”.
- JWT is good for implementing stateless APIs.

OAUTH 2.0

Have you seen this before?

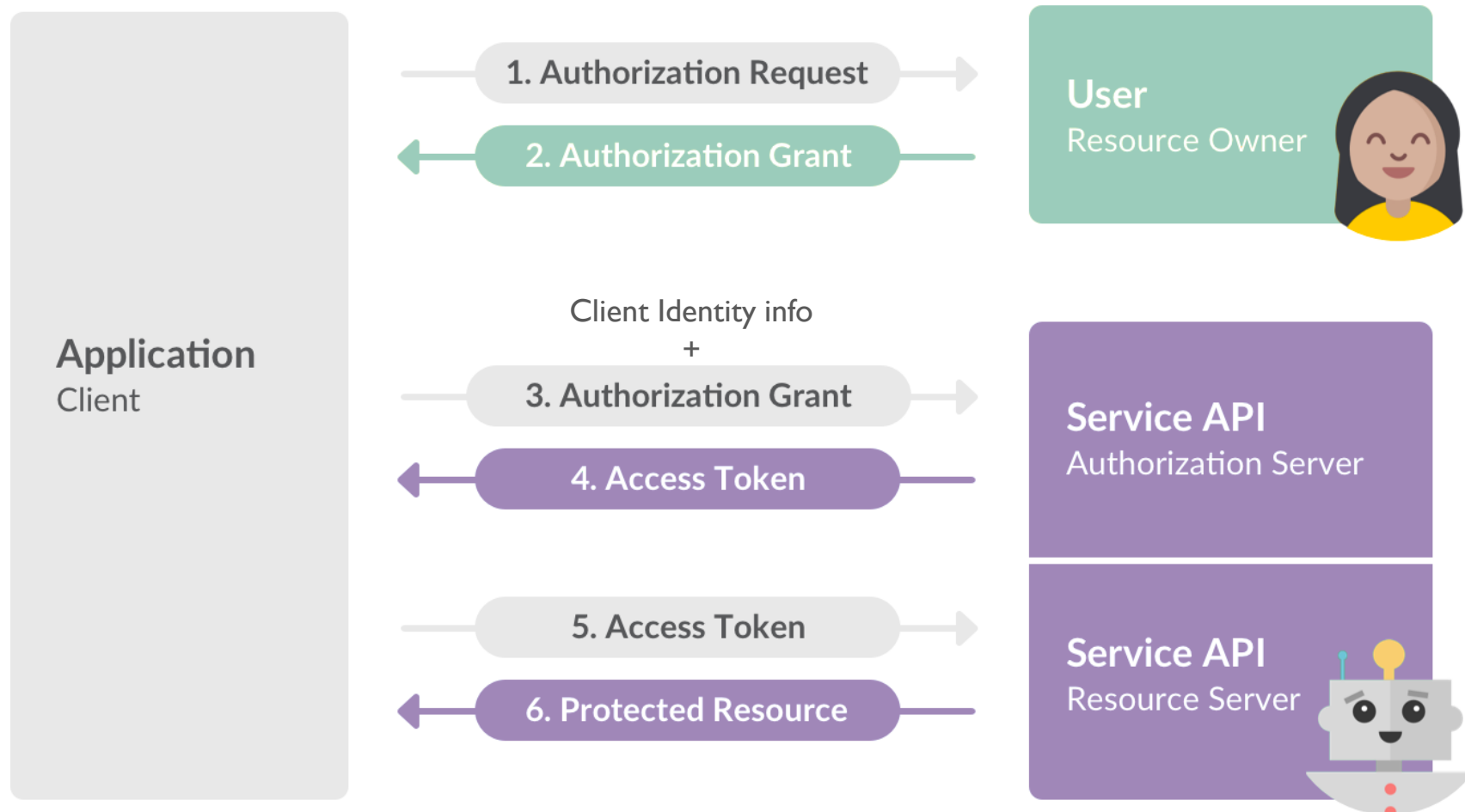
The image shows a login interface with the title "Login Into Your Account". It features two input fields: "User Name" and "Password", each with a small icon on the right (a person for the name and a lock for the password). Below these fields is a green "Sign In" button. To the right of the password field is a vertical line with the word "or" in the middle. To the right of this line are three social media login buttons, each with a logo and the text "Sign In with [Platform]": a blue Facebook button, a light blue Twitter button, and a red Google+ button. These three social media buttons are enclosed in a red rectangular border.

Source: <https://lo-victoria.com/introduction-to-rest-api-authentication-methods>

OAUTH 2.0

- It is an authorization framework/ protocol.
- OAuth addresses a very specific problem.
 - How can the **resource owner** give a **client, scoped access** to a **protected resource**?
- If an implementation also conforms to OpenID Connect specification, it will also support authentication.
 - This is how social authentication works!
- OAuth works by,
 - **Delegating user authentication** to a service that hosts a user account
 - **Authorizing third-party applications** to access that user account

OAUTH 2.0



OAUTH 2.0

■ OAuth Roles

- **Resource Owner:** The resource owner is the user who authorizes an application to access their account.
- **Client:** The client is the application that wants to access the user's account.
- **Authorization Server:** The authorization server verifies the identity of the user then issues access tokens to the client application.
- **Resource Server:** The resource server hosts the protected resources.

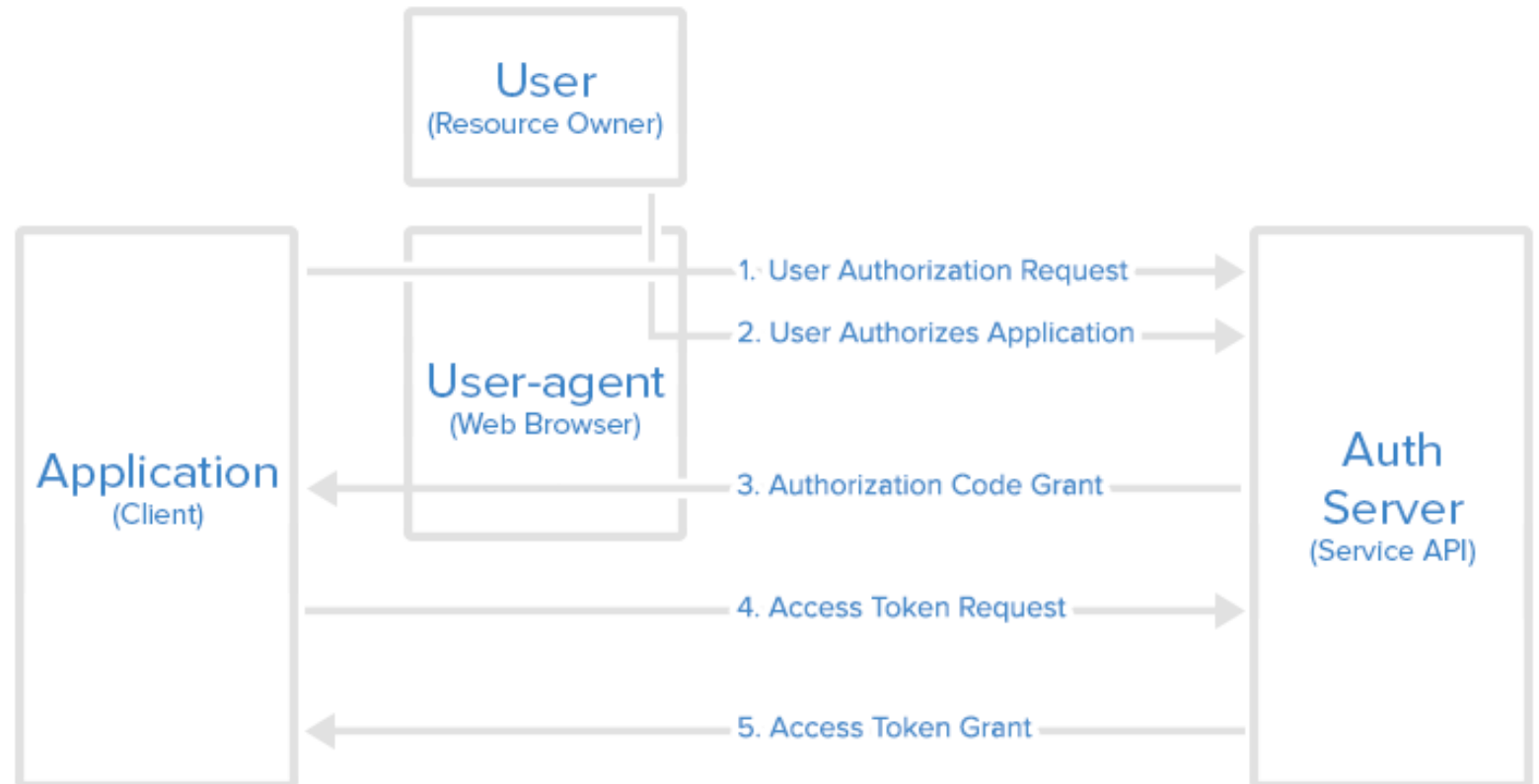
OAUTH 2.0

- An OAuth 2.0 flow (grant type) is a way of retrieving an Access Token.
- OAuth 2.0 supports different authorization flows (also known as grant types).
 - Authorization Code Flow – Widely used for server-side and mobile web applications.
 - PKCE
 - Client Credentials flow etc.

OAUTH 2.0

Authorization Code Flow

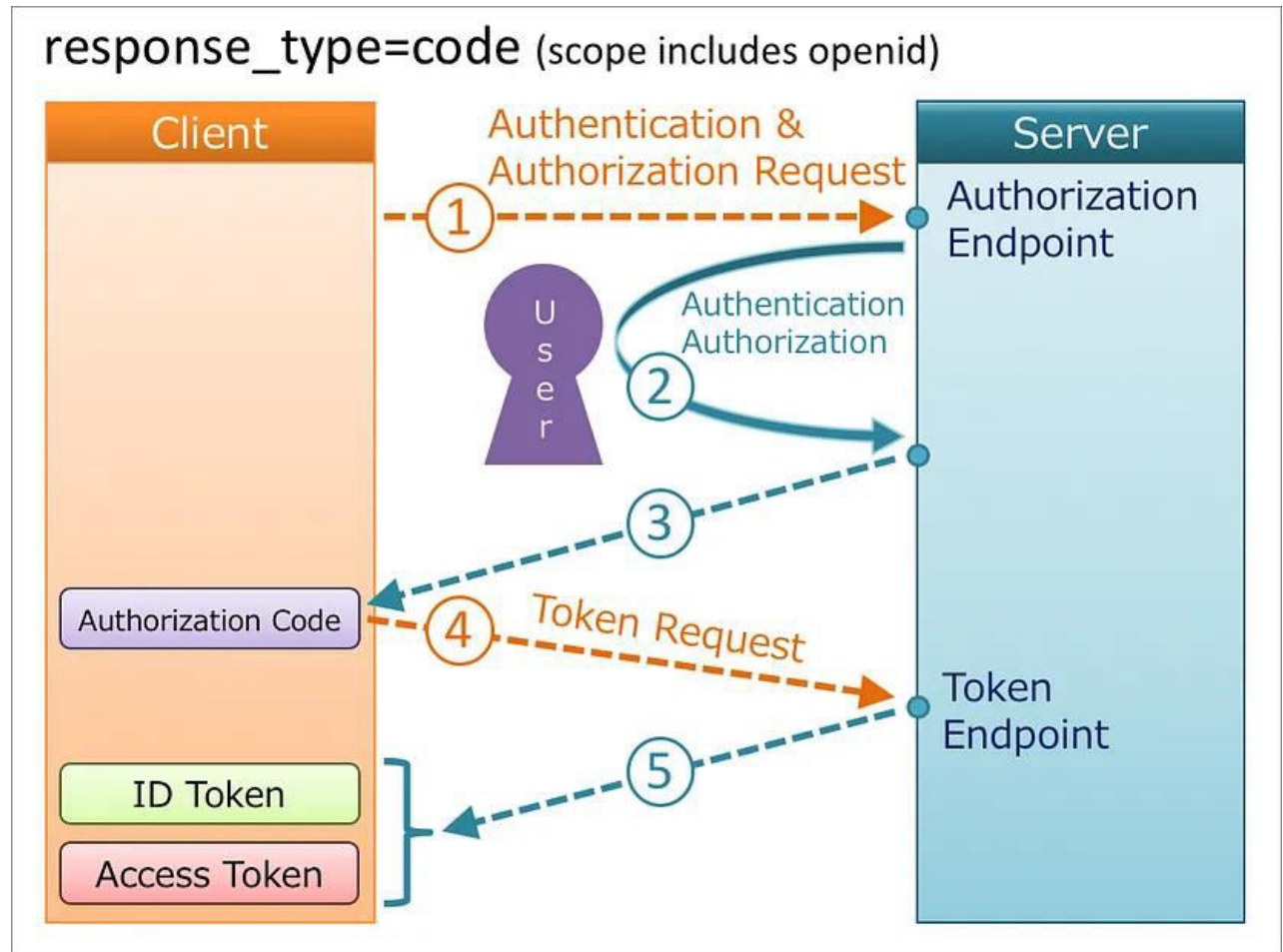
Authorization Code Flow



Source: <https://www.digitalocean.com/community/tutorials/an-introduction-to-oauth-2>

OPENID CONNECT

Authorization Code Flow + with Authentication



Source: <https://darutk.medium.com/diagrams-of-all-the-openid-connect-flows-6968e3990660>

OAUTH 2.0

- Why use OAuth 2.0?
 - It enables an application to obtain limited access (scopes) to a user's data without the user having to share their password.
- Why use OpenID Connect (which is an extension of OAuth 2.0)?
 - Accelerate Sign Up Process at Your Favorite Websites
 - No need to maintain multiple usernames/ passwords.
 - Minimize password security risks.

OAUTH 2.0

- [\[YouTube\] OAuth 2.0: An Overview](#)

SELF-STUDY ACTIVITY

- There is much more in OAuth 2.0.
 - Read this [introduction to OAuth 2.0](#) to get a more detailed understanding.
- Spring Boot and OAuth 2.0
 - Check out [these official tutorials from Spring](#).

SUMMARY

- Authentication vs. Authorization
- Importance of Authentication and Authorization
- Some methods for Authentication and Authorization in REST APIs
 - HTTP Basic Authentication
 - API Keys
 - JSON Web Tokens (JWT)
 - OAuth 2.0
 - OpenID Connect (OIDC)

REFERENCES

1. [3 Common Methods of API Authentication Explained](#)
2. [4 Most Used REST API Authentication Methods](#)
3. [How does HTTPS actually work?](#)
4. [HTTP authentication](#)
5. [API Keys](#)
6. [Introduction to JSON Web Tokens](#)
7. [Authorization Code Flow](#)



THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK



FRONTEND DEVELOPMENT OVERVIEW

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

LEARNING OUTCOMES

- After completing this lecture, you will be able to,
 - Describe the main concepts associated with frontend web development.
 - Apply the knowledge gained in developing more robust frontend web applications.
 - Apply the learned concepts to create more secure web applications.

CONTENTS

- What is frontend development?
- Why is frontend development important?
- HTML, CSS, JS
- Document Object Model (DOM)
- State in Web apps
 - State Management
- Web Application Security
 - OWASP Top 10 Security Risks & Vulnerabilities
 - HTTP Security Headers
- Wrap up
- Summary

WHAT IS FRONTEND DEVELOPMENT?

- Frontend development is the development of visual and interactive elements of a website that users interact with directly.
- Also known as Client-side development.

WHY IS FRONTEND DEVELOPMENT IMPORTANT?

■ Backend vs. Frontend

- Frontend web development focuses on **creating a good look and feel for the user.**
- Backend web development focuses on engineering the web application's structure, logic and data.

WHY IS FRONTEND DEVELOPMENT IMPORTANT?

- Frontend development can be considered under two aspects.
 - **Designing** a good look and feel
 - HCI Design, UI/ UX etc.
 - **Engineering** the designed good look and feel
 - Frontend technologies
 - E.g., HTML, CSS, JS, Different frameworks, libraries etc.

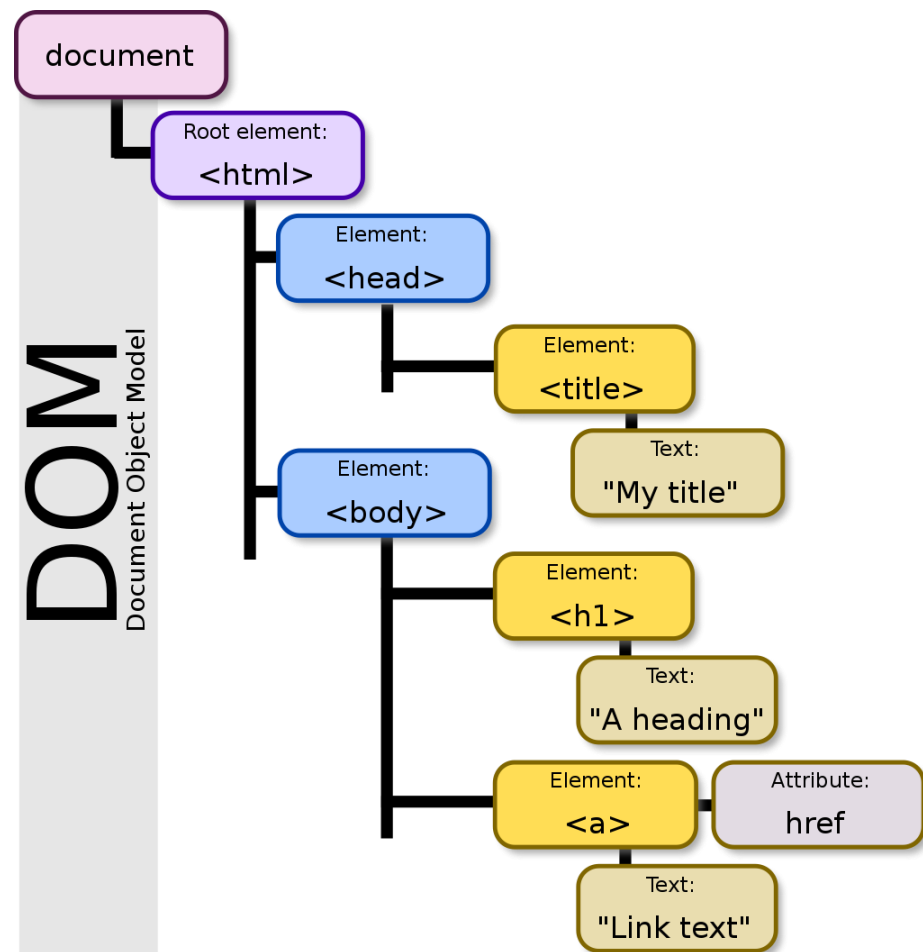
HTML, CSS, JS

- Frontend development is done with a combination of technologies in which,
 - **HTML** provides the structure
 - **CSS** provides the styling and layout
 - **JavaScript** provides the dynamic behavior and interactivity

DOCUMENT OBJECT MODEL (DOM)

- What do we actually manipulate with these technologies?
 - Document Object Model (DOM)!

DOCUMENT OBJECT MODEL (DOM)

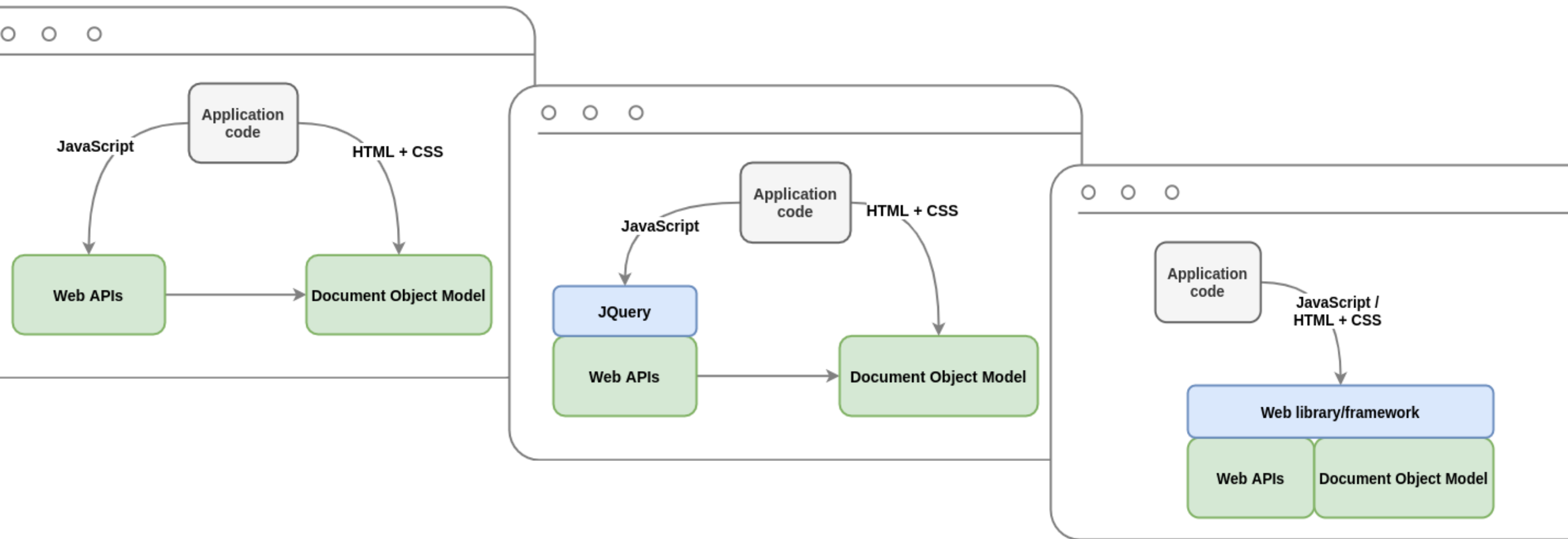


Source: https://en.wikipedia.org/wiki/Document_Object_Model

DOCUMENT OBJECT MODEL (DOM)

- Document Object Model (DOM) – An object-oriented representation of the HTML structure
- DOM is not a JS functionality.
 - JS is the mostly used language to manipulate it, but other languages can also be used.
- It is just a Web API.
- [YouTube] The DOM in 4 minutes

EVOLUTION OF FRONTEND DEVELOPMENT



Source: <https://www.epineda.net/the-evolution-of-front-end-development/amp/>

REACT JS

- Developed and maintained by Facebook and Instagram.
- A JavaScript library for creating user interfaces.
- Serves as the view of MVC architecture.
- Suitable for large web application which use data and change over the time without reloading the entire page.
- React Native caters developing mobile application for various platforms and React VR caters developing VR applications.
- Aims at Speed, Simplicity and Scalability.

REACT : COMPONENTS

- Components are one of the core concepts of React.
- They are the foundation upon which you build user interfaces (UI)
- UI is built from small units like buttons, text, and images.
- React lets you combine them into reusable, nestable components.
- From web sites to phone apps, everything on the screen can be broken down into components.
- <https://react.dev/learn/thinking-in-react>

Amazing scientists



Hedy Lamarr's Todos



- Invent new traffic lights
- Rehearse a movie scene
- Improve spectrum technology

NOTABLE FEATURES

One-Way data flow.

- Single source of truth - Data is originated and kept in one place, data is immutable.
- Data flow from the parent component to the child component.
- Action flow from child component to parent component.

Virtual DOM

- DOM manipulation is cost instead react create a virtual DOM tree and compare it with the rendered tree and update only what is changed.

JSX

- React JS language for defining user interfaces.
- HTML/XML like syntax.
- Prevents cross-site scripting attacks by converting expressions to strings.

PROJECT STRUCTURE

```
/my-react-app
|-- /node_modules
|-- /public
|-- /src
    |-- /components // React components, representing the 'View' in MVC
        |-- /shared // Shared components like buttons, modals, etc.
        |-- /layout // Layout components like headers, footers, etc.
        |-- /pages // Page components, corresponding to different views/screens
    |-- /models // Models in MVC, representing the data layer
        |-- user.js
        |-- product.js
        // other data models
    |-- /controllers // Controllers in MVC, orchestrating the app's data flow
        |-- UserController.js
        |-- ProductController.js
        // other controllers
    |-- /services // Services for handling side effects, API calls, etc.
        |-- ApiService.js
        |-- UserService.js
        |-- ProductService.js
        // other services
    |-- /utils // Utility functions and helpers
    |-- /hooks // Custom React hooks, if any
    |-- /store // If using Redux or a similar state management, to handle shared state
        |-- /actions
        |-- /reducers
    |-- App.js // Main React component
    |-- index.js // Entry point for React application
|-- package.json // Node package configuration
|-- README.md
```

STATE IN WEB APPS

- **State** refers to the current condition or data of the application at a given point in time.
- State is important in web applications because it affects the behavior and appearance of the application.
- [\[YouTube\] What is "State" in Programming?](#)

STATE MANAGEMENT IN WEB APPS

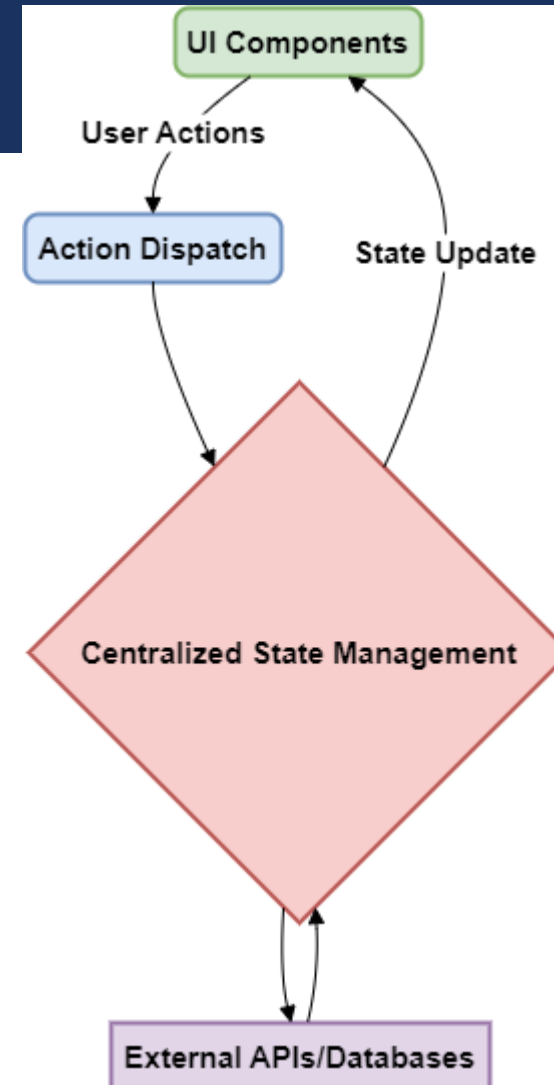
- **State management** is a concept in web development that refers to the management and manipulation of data or information within an application.
- In simple terms, it involves **controlling the data flow** and ensuring that the application's **data is consistent** and **up-to-date**.

STATE MANAGEMENT IN WEB APPS

- Proper management of state is important for the performance and reliability of web applications.
- Poorly managed state can result in slow application performance, data inconsistencies, or security vulnerabilities.

STATE MANAGEMENT IN WEB APPS

- How is State managed in web apps?
 - Local state management
 - Global state management
 - Redux



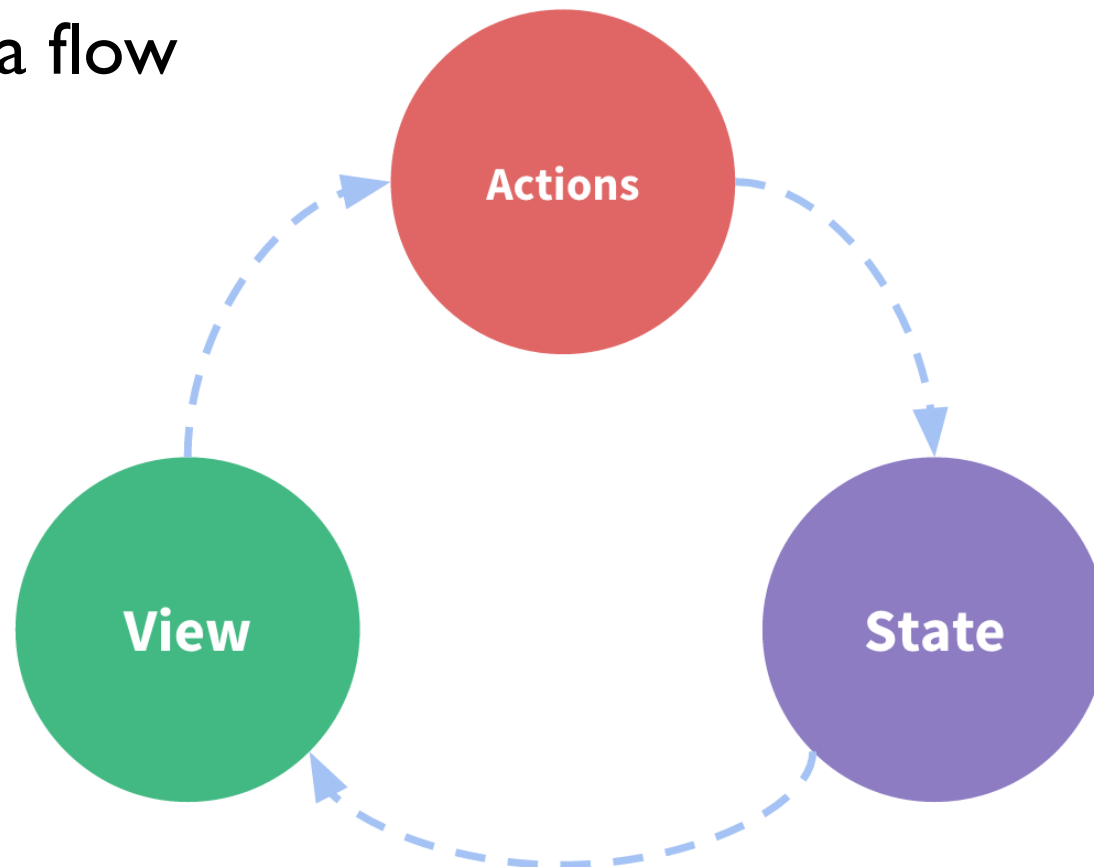
STATE MANAGEMENT IN WEB APPS

■ Redux

- Redux is a pattern and library for managing and updating application state.
- Redux helps you manage "global" state - state that is needed across many parts of your application.
- It serves as a centralized store for state that needs to be used across your entire application, with rules ensuring that the state can only be updated in a predictable fashion.
- Redux is commonly used with React.
- Read more on Redux [here](#).

STATE MANAGEMENT IN WEB APPS

Redux one-way data flow



Source: <https://redux.js.org/tutorials/essentials/part-1-overview-concepts>

STATE MANAGEMENT IN WEB APPS

- Redux is not always needed.
 - **Do not use Redux until** you have problems with vanilla React.
- When to use Redux?
 - You have **large amounts of application state** that are needed in many places in the app.
 - The app state is updated frequently over time.
 - The logic to update that state may be complex.
 - The app has a medium or large-sized codebase and might be worked on by many people.

PROPS AND STATE

- Props: Props are used to pass data from a parent component to a child component.
- State: State is used to manage data that changes over time in a component.

EVENT HANDLERS

- In React, event handlers are functions that are called in response to user actions, such as clicks, keystrokes, or form submissions.
- React event handlers are typically defined as methods of a component class or as arrow functions within a functional component.
- In JSX, event handlers are specified as attributes of elements, using a naming convention where the event name is prefixed with `on`, followed by the name of the event in camelCase.
- Event handlers can receive an event object as a parameter, which contains information about the event, such as the target element, the mouse position, or the key that was pressed.

HOOKS

- React Hooks are a way to use state and other React features in functional components, without the need for class components.
- The most common Hooks are `useState`, `useEffect`, `useContext`, and `useReducer`. Each Hook has a specific purpose and usage.
- The `useState` Hook allows you to add state to a functional component. It returns an array with two values: the current state value and a function to update the state.
- The `useEffect` Hook allows you to perform side effects in a functional component, such as fetching data from an API or updating the DOM. It takes a function as its argument and runs it after every render.
- The `useContext` Hook allows you to access data from a React context in a functional component. It takes a context object as its argument and returns the current context value.
- The `useReducer` Hook is an alternative to `useState` that allows you to manage more complex state updates. It takes a reducer function and an initial state value as its arguments and returns the current state value and a dispatch function to update the state.
- Hooks should only be used at the top level of a functional component or another custom Hook. They should not be used inside loops or conditionals.

MANAGING STATE | REACTING TO INPUTS

- As your application grows, it helps to be more intentional about how your state is organized and how the data flows between your components.
- Redundant or duplicate state is a common source of bugs.
- React uses a declarative way to manipulate the UI.
- Instead of manipulating individual pieces of the UI directly, you describe the different states that your component can be in, and switch between them in response to the user input.
- This is similar to how designers think about the UI.

MANAGING STATE | STATE STRUCTURE

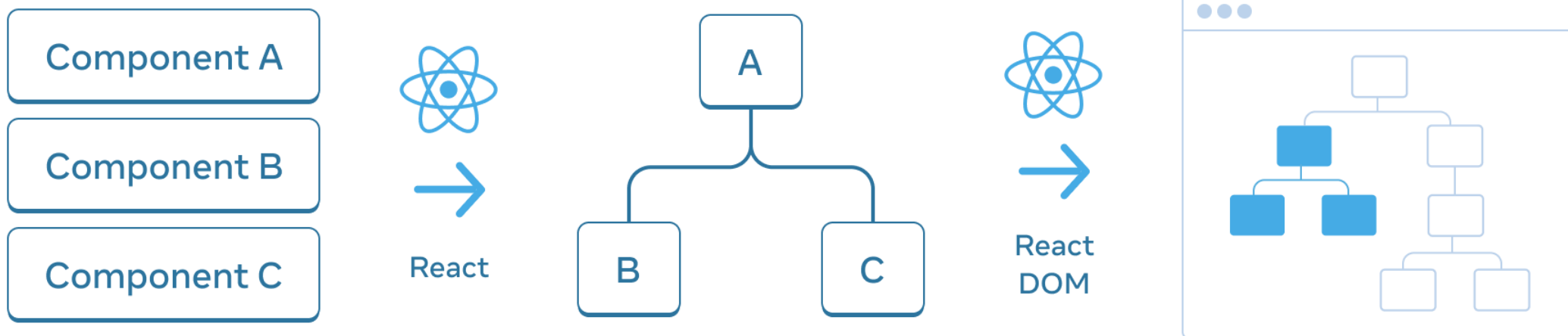
- Structuring state well can make a difference between a component that is pleasant to modify and debug, and one that is a constant source of bugs.
- **Group related state.** If you always update two or more state variables at the same time, consider merging them into a single state variable.
- **Avoid contradictions in state.** When the state is structured in a way that several pieces of state may contradict and “disagree” with each other, you leave room for mistakes. Try to avoid this.
- **Avoid redundant state.** If you can calculate some information from the component’s props or its existing state variables during rendering, you should not put that information into that component’s state.
- **Avoid duplication in state.** When the same data is duplicated between multiple state variables, or within nested objects, it is difficult to keep them in sync. Reduce duplication when you can.
- **Avoid deeply nested state.** Deeply hierarchical state is not very convenient to update. When possible, prefer to structure state in a flat way.

MANAGING STATE | STATE SHARING

- Sometimes, you want the state of two components to always change together.
- Remove state from both and move it to the closest parent component
- Then pass it down to them via props
- This is known as lifting state up, and it's one of the most common things you will do writing React code.

MANAGING STATE | PRESERVING & RESETTNG STATE

- State is isolated between components.
- React keeps track of which state belongs to which component based on their place in the UI tree.
- You can control when to preserve state and when to reset it between re-renders.

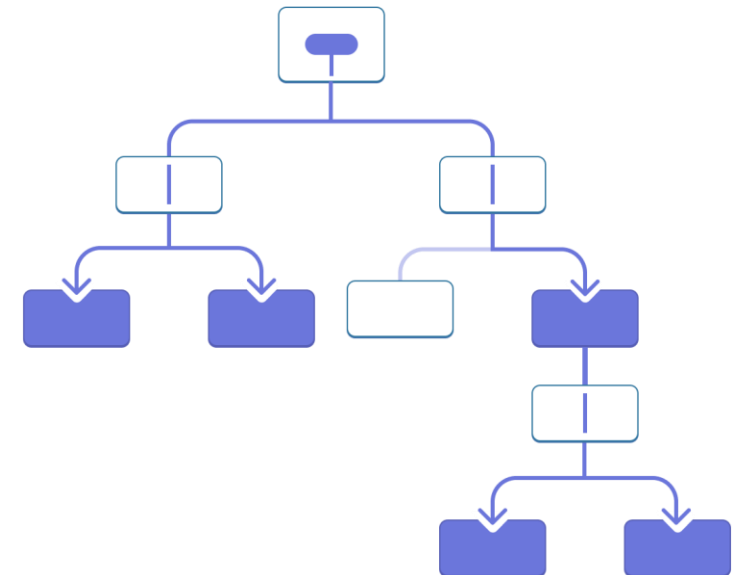
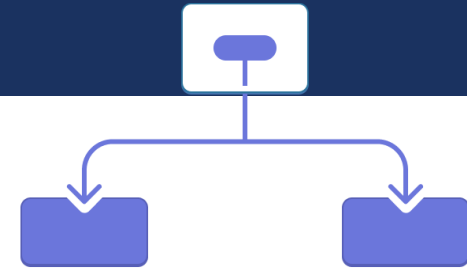


MANAGING STATE | USING REDUCER

- Components with many state updates spread across many event handlers can get overwhelming
- For these cases, you can consolidate all the state update logic outside your component in a single function, called a reducer.
- Although reducers can “reduce” the amount of code inside your component, they are named after the `reduce()` operation that you can perform on arrays.

MANAGING STATE | WITH CONTEXT

- Usually, you will pass information from a parent component to a child component via props.
- But passing props can become verbose and inconvenient if you must pass them through many components in the middle, or if many components in your app need the same information.
- Context lets the parent component make some information available to any component in the tree below it no matter how deep without passing it explicitly through props.



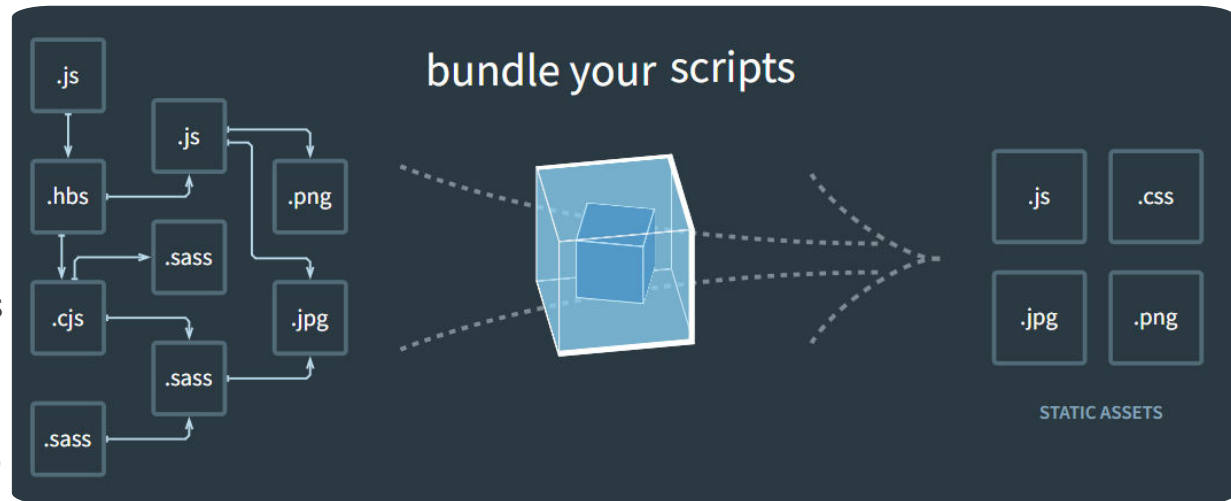
BUNDLING

- In web development, bundling refers to combining multiple files into a single file.
- Bundling is typically done with JavaScript or CSS files.
- Bundling reduces the number of HTTP requests needed to fetch all the required resources for a web page.
- This can improve the page's performance by reducing network latency.
- Bundling tools like Webpack or Parcel are used to create the bundle file.
- These tools analyze dependencies between files to create the bundle.
- Optimization techniques like minification and tree shaking may also be used as part of the bundling process.



WEBPACK

- Webpack is an open-source bundling tool and module bundler used in web development.
- It is used to bundle JavaScript files and other assets like CSS, images, and fonts for web applications.
- Webpack analyzes the dependencies between modules in the application and generates a single optimized bundle file.
- It includes a built-in development server that allows developers to preview their applications in a browser and see live changes.
- Webpack offers features like code splitting, lazy loading, tree shaking, and hot module replacement to optimize and speed up performance.
- It is highly configurable and can be customized according to project requirements.
- Webpack is widely used in modern web development frameworks like React, Vue, and Angular.
- <https://webpack.js.org/>



PARCEL

- Parcel is an open-source web application bundler and build tool, similar to Webpack.
- It is designed to be zero-config, meaning that it requires minimal setup and configuration to get started.
- Parcel can bundle a variety of web assets, including JavaScript, CSS, HTML, images, and more.
- It automatically handles file transformations, such as transpiling ES6 code to ES5 or compressing images.
- Parcel can use multiple cores to build and bundle projects in parallel, which can reduce build times.
- It includes a built-in development server that allows developers to see their changes in real-time as they make modifications to their code.
- Parcel has gained popularity among developers due to its ease of use and simplicity.
- It has been adopted by popular frameworks such as React, Vue, and Angular.
- <https://parceljs.org/>



SETTING UP REACT

- Official Documentation recommend to use a framework for react
- Frameworks provide features that most apps and sites eventually need, including routing, data fetching, and generating HTML.
- Popular Frameworks
 - Next.js
 - Remix
 - Gatsby
 - Expo (for native apps)

VITE

- Vite is a build tool and development server for modern web applications.
- It was created by Evan You, the creator of Vue.js.
- Vite is known for its fast startup time.
- It uses a development server that leverages modern browser features like native ES modules and HTTP/2 to quickly serve application code.
- Vite provides a build tool that includes features like code splitting and tree shaking to optimize the final application bundle for deployment.
- Vite is primarily used for developing front-end applications built with frameworks like React or Vue.js.



WEB APPLICATION SECURITY

- OWASP, or the Open Web Application Security Project, is a nonprofit organization focused on software security.
- OWASP Top 10 is a list of the 10 most common web application security risks.

WEB APPLICATION SECURITY

- OWASP Top 10 Vulnerabilities 2021
 1. Broken Access Control
 2. Cryptographic Failures
 3. Injection
 4. Insecure Design
 5. Security Misconfiguration
 6. Vulnerable and Outdated Components
 7. Identification and Authentication Failures
 8. Software and Data Integrity Failures
 9. Security Logging and Monitoring Failures
 10. Server-Side Request Forgery

WEB APPLICATION SECURITY

- Testing an application for the OWASP vulnerabilities is a **crucial step** in application development.
- [YouTube] What is The OWASP Top 10? | A Radware Minute
- [Mandatory] Read OWASP Top Security Risks & Vulnerabilities 2021.

WEB APPLICATION SECURITY

■ HTTP Security Headers

- HTTP security headers are a subset of HTTP headers that is related specifically to security.
- Setting suitable headers in your web applications and web server settings is an easy way to greatly improve the resilience of your web application against many common attacks.
- Read this link on [HTTP Security Headers](#).
- Some additional information can be found [here](#) too.

WRAP UP

- See [this link](#) for a step-by-step guide on becoming a frontend developer.
- Here's a nice overview on Frontend development: [\[YouTube\] Frontend web development - a complete overview](#)

SUMMARY

- What is frontend development?
- Why is frontend development important?
- HTML, CSS, JS
- Document Object Model (DOM)
- State in Web apps
 - State Management
- Web Application Security
 - OWASP Top 10 Security Risks & Vulnerabilities
 - HTTP Security Headers

REFERENCES

1. [What is the Document Object Model?](#)
2. [Introduction to the DOM](#)
3. [The Best React State Management Tools for Enterprise Applications](#)
4. [Redux Essentials, Part I: Redux Overview and Concepts](#)
5. [OWASP Top 10 Vulnerabilities](#)



THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK



DEVOPS AND CI/ CD - AN INTRODUCTION

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

LEARNING OUTCOMES

- After completing this lecture, you will be able to,
 - Describe the failures in the traditional software deployment methods, and the reasons for the rise of DevOps.
 - Describe the main components of DevOps.
 - Describe what CI/ CD is and how its application is beneficial in modern software development.
 - Describe the importance of Test Automation in DevOps.
 - Apply the concepts learnt in solving real-world problems.

CONTENTS

- Deploying an app
 - Typical Deployment Scenario
 - Failures
- DevOps
 - Core DevOps principles
 - Continuous Integration (CI)
 - Continuous Delivery and Continuous Deployment (CD)
 - Infrastructure Automation
 - Software Testing and Test Automation
- Wrap Up
- Summary

JOURNEY SO FAR

- What was discussed so far,
 - Software Frameworks
 - Version controlling with Git
 - Git Workflows
 - Web application architectures
 - REST APIs
 - An overview on Frontend development

DEPLOYING AN APP

- We have discussed how we engineer a web application so far.
- However, engineering the application is not enough.
- It must be **deployed!**
 - Why? So that actual users can use the application.

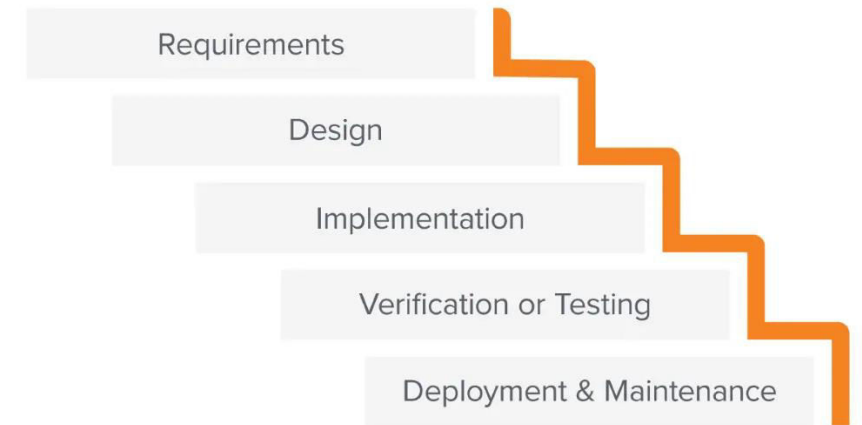
DEPLOYING AN APP

- Application Deployment, also known as Software Deployment, is the process of installing, configuring, updating, and enabling one application or suite of applications that make a software system **available for use**.
(Source)

TYPICAL DEPLOYMENT SCENARIO

- How were deployments handled traditionally?
 - Waterfall model was the most common SDLC model used back in the day.
 - SDLC - Software Development Life Cycle
 - In Waterfall model, each stage is carefully planned and executed sequentially by a dedicated team.
 - E.g.: Business stakeholders, Business Analysts, Architects, Developers, Quality Assurance, IT Operations

The Waterfall Method



Source: <https://business.adobe.com/blog/basics/waterfall>

TYPICAL DEPLOYMENT SCENARIO

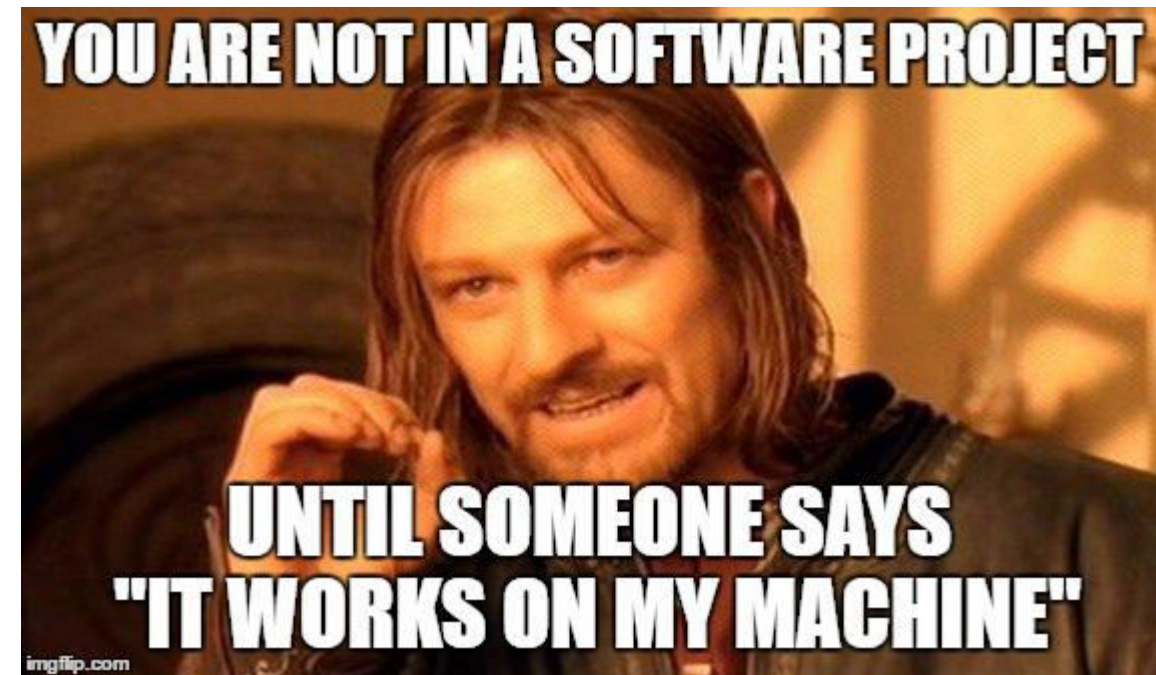
- A release would be carried out every few months.
- First, the developers would develop the solution.
- Then the QA team would start release testing the application leading to the release.

TYPICAL DEPLOYMENT SCENARIO

- The QA team would then discover bugs.
 - QA team would frantically report the bugs.
 - Developers would frantically fix the bugs.
- Now the QA team has to retest the fixes.
 - This is on top of the functionality remaining to be tested!
- Typically, the developers and/ or QA teams would get overwhelmed with the workload as the release date becomes closer.

TYPICAL DEPLOYMENT SCENARIO

- Then the Developers, QA and IT Operations teams would recommend to push ahead the release date given the problems.
 - Business stakeholders would not agree!
 - They dictate that release be carried out as planned as they have their own strategic goals to meet.
- IT Ops would also run into various problems deploying the application.
 - Famous example: **It works on my machine!**



Source: <https://elbruno.com/you-are-not-in-a-software-project-until-someone-says-it-works-on-my-machine/>

TYPICAL DEPLOYMENT SCENARIO

- Finally, the release would be carried out as planned even with the issues.
 - It will be rolled back in the next couple of hours after doing everything to make the application work fails.
 - Redeploy again with several components disabled.
 - Fixes will be carried out for some time after the release to stabilize the product.
 - The product is actually usable only after few days/ weeks after the release.
- **Rinse and repeat** for the next release!

TYPICAL DEPLOYMENT SCENARIO: FAILURES

- Each team (Dev/ QA/ IT Ops) worked separately and had competing objectives (“siloed”).
- Each team had their own goals.
 - Resulted in botched releases and unhappy customers.
- Lots of firefighting/ finger pointing - Very stressful environment!



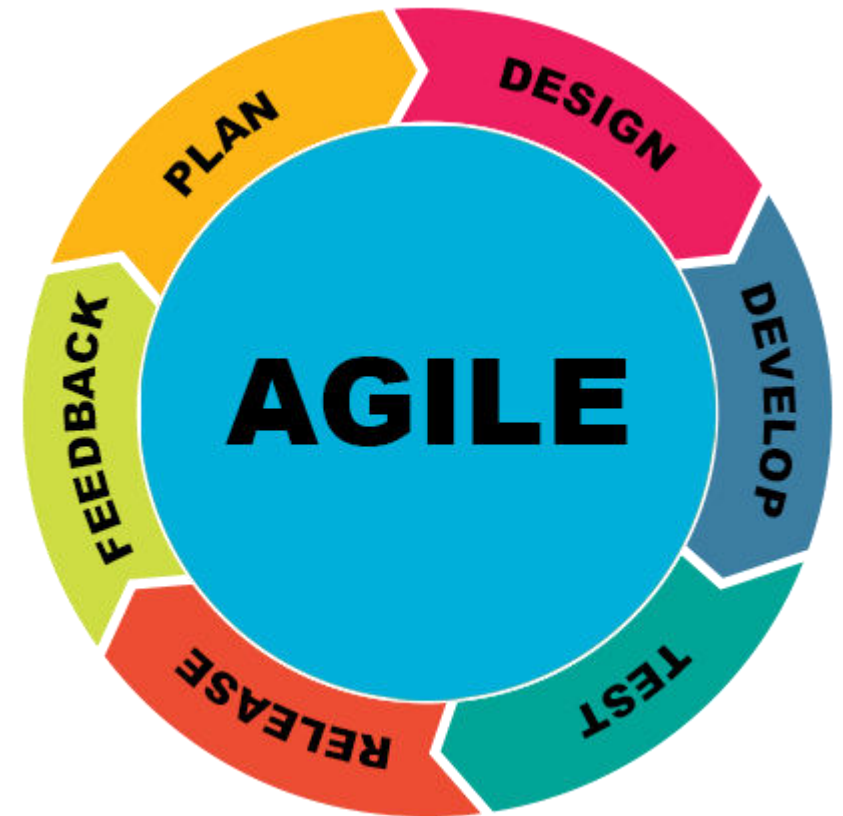
Source: <https://resources.biginterview.com/behavioral-interviews/behavioral-interview-questions-conflict/>

DEVOPS

- Somewhere between 2007-2009 different communities started raising concerns about the level of dysfunction in doing releases this manner.
- There had to be a better way of doing things!
 - Gradually a movement appeared against the traditional mindset of keeping different stakeholders separated in different silos.
- This is the origin of “**DevOps**”.
- [\[YouTube\] What is DevOps?](#)

DEVOPS

- DevOps stands for “Development-Operations”.
- The concept behind DevOps is simple.
 - Bring the Developers, QA, IT Operations (IT Ops) teams together to work collaboratively on planning, building, testing and releasing software with shared responsibility for successful business outcomes.
 - Based on **Agile approach** to software development!
 - DevOps is not just a method for Software Development. It's a **culture**.

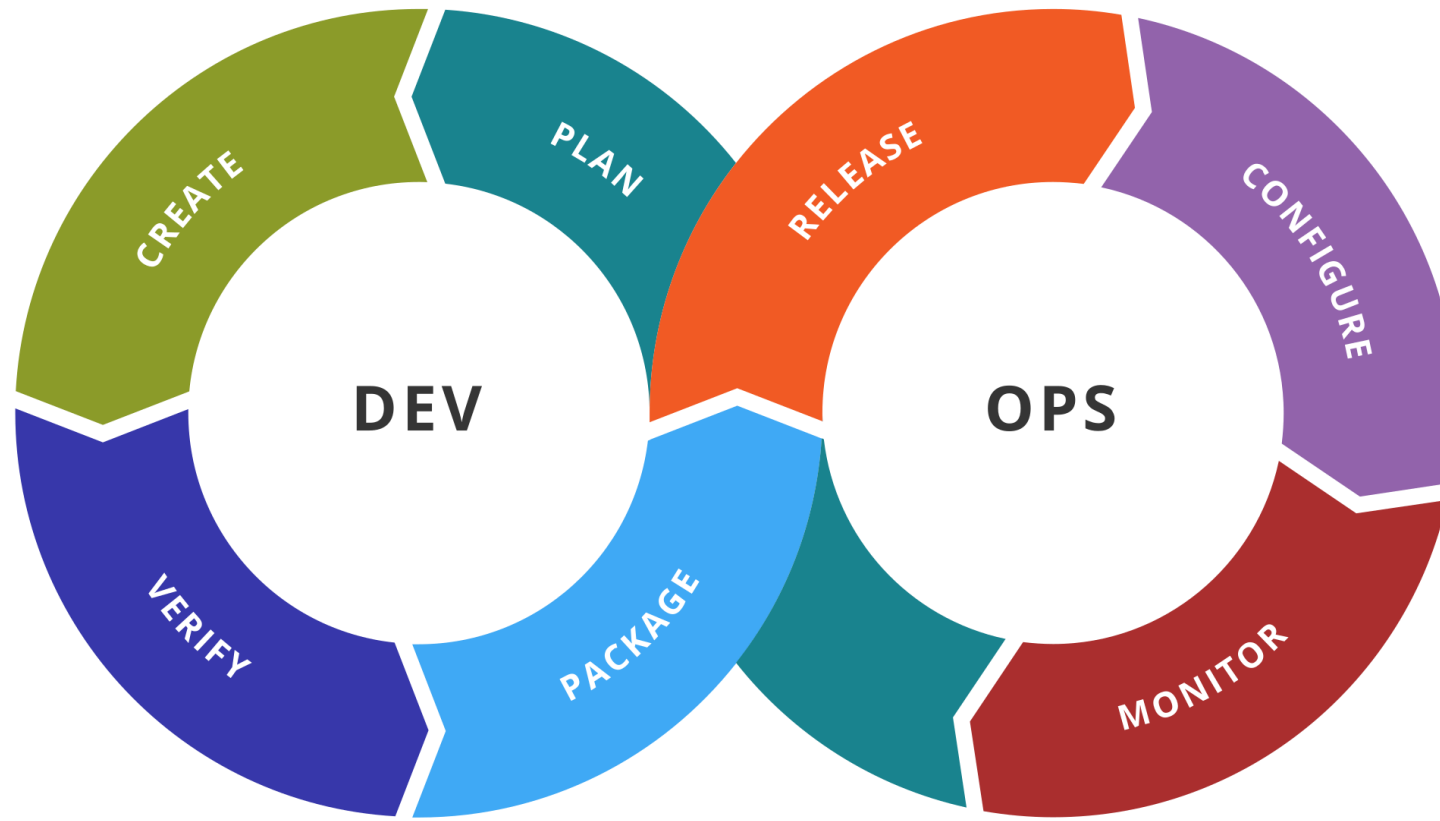


Source: <https://technology.berkeley.edu/tpo/agile>

DEVOPS

- DevOps is a combination of software development (dev) and operations (ops). It is defined as a software engineering methodology which aims to integrate the work of development teams and operations teams by facilitating a culture of collaboration and shared responsibility. (Source)

DEVOPS STAGES



Source: https://en.wikipedia.org/wiki/DevOps_toolchain

CORE DEVOPS PRINCIPLES

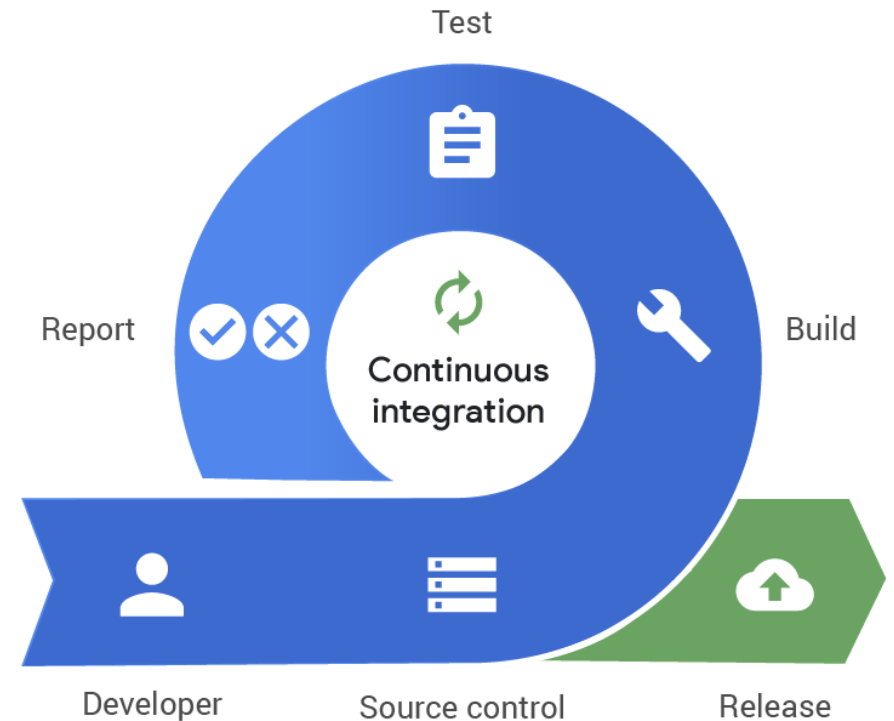
- Automation of the software development lifecycle
 - Automating the SDLC as much as possible increases productivity by reducing the introduction of human errors and freeing up humans from repetitive tasks.
 - Mainly consists of,
 - Continuous Integration (CI)
 - Continuous Delivery and Continuous Deployment (CD)
 - Infrastructure Automation (Automated provisioning of Environments and other resources)
 - Test Automation - **Shift Left** Testing ([Read this](#))

CORE DEVOPS PRINCIPLES

- Collaboration and communication
 - Developers, QA, stakeholders, IT Ops should communicate clearly to collaborate efficiently. At the end of the day, it is everyone's responsibility to deliver software which satisfies business requirements.
- Continuous improvement
 - It is the practice of focusing on experimentation, minimizing waste, and optimizing for speed, cost, and ease of delivery.
- Customer-centric action
 - DevOps teams use short feedback loops with customers and end users to develop products and services centered around user needs.

CONTINUOUS INTEGRATION (CI)

- Continuous integration is the practice of,
 - Frequently pushing all code changes into a designated main branch of a code repository,
 - automatically kicking off a build whenever there is a new change(s),
 - And running automatic tests against the build to verify the change.
 - Developers need to create the tests too if they don't exist already.



Source: <https://www.pagerduty.com/resources/learn/what-is-continuous-integration/>

CONTINUOUS INTEGRATION (CI)

- By merging changes frequently and triggering automatic testing and validation processes,
 - the issues can be discovered early and fixed then and there.
 - Integration challenges that can happen when waiting for release day to merge changes into the release branch can be avoided.
 - Quality of the product can be verified from the early days of development rather than near the release date (Shift-left testing).

CONTINUOUS DELIVERY (CD)

- **Continuous delivery** is an extension of continuous integration.
- It is the deployment of all built and automatically tested deliverables (basically executables) to a further testing and/or production environment after the CI stage via an automated process.
- This **takes away the stress on a team** to prepare for a delivery for days.

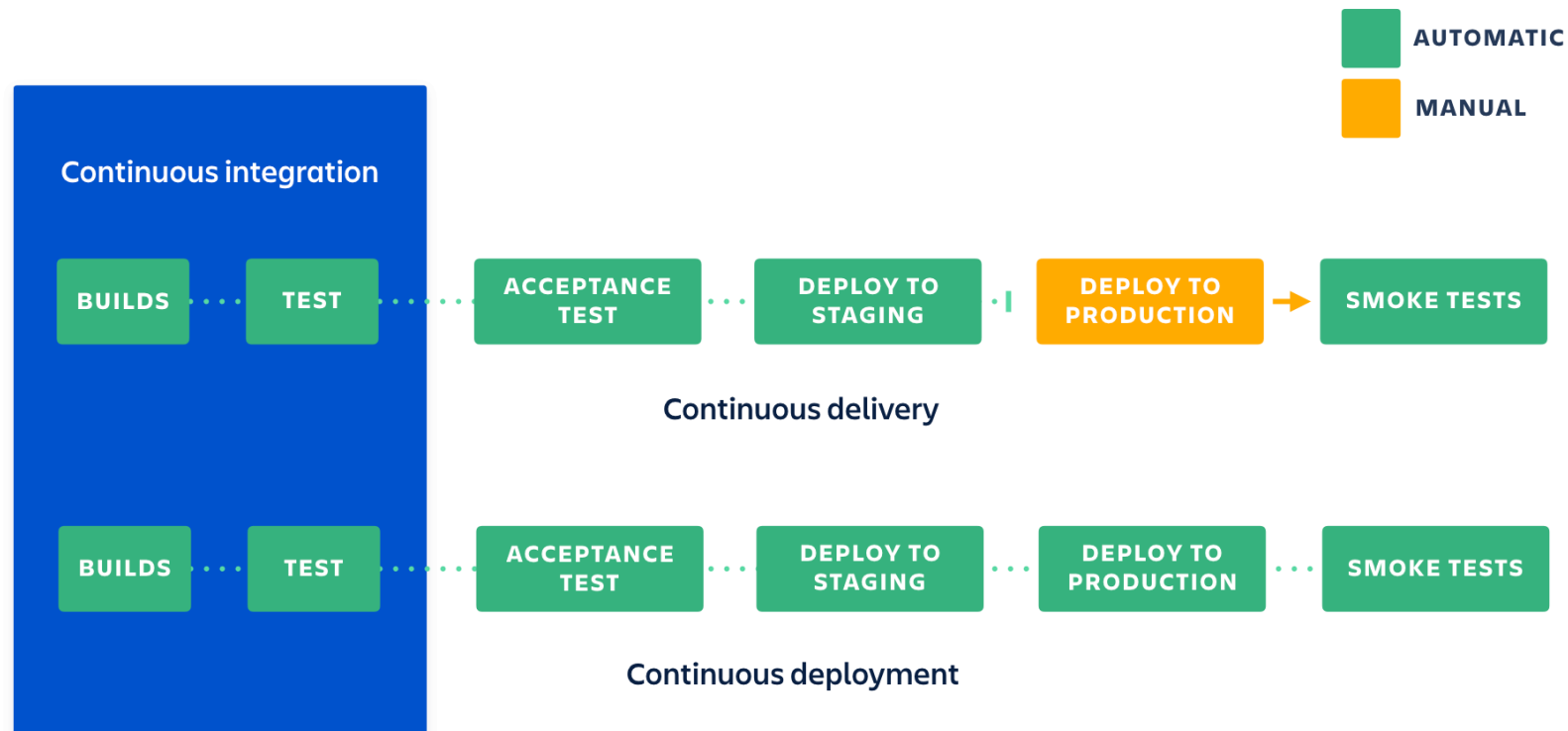
CONTINUOUS DELIVERY (CD)

- This deployment process is triggered manually. Once the process is triggered, the deployment happens automatically.
- The user has the control over when the automatic deployment should be triggered with just a click of a button.
- **Feature flags** are used to disable/ hide incomplete features from affecting customers in production.

CONTINUOUS DEPLOYMENT (CD)

- **Continuous Deployment** is a further extension of Continuous Delivery.
- With Continuous Deployment, every change that passes all stages of the production pipeline is released to the customers.
- There is **no human intervention**, and only a failed test will prevent a new change to be deployed to production.
- Feature flags are an inherent part in Continuous Deployments.

CONTINUOUS INTEGRATION AND CONTINUOUS DELIVERY VS. CONTINUOUS DEPLOYMENT



Source: <https://www.atlassian.com/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment>

SOME POPULAR TOOLS FOR CI/ CD

Open Source



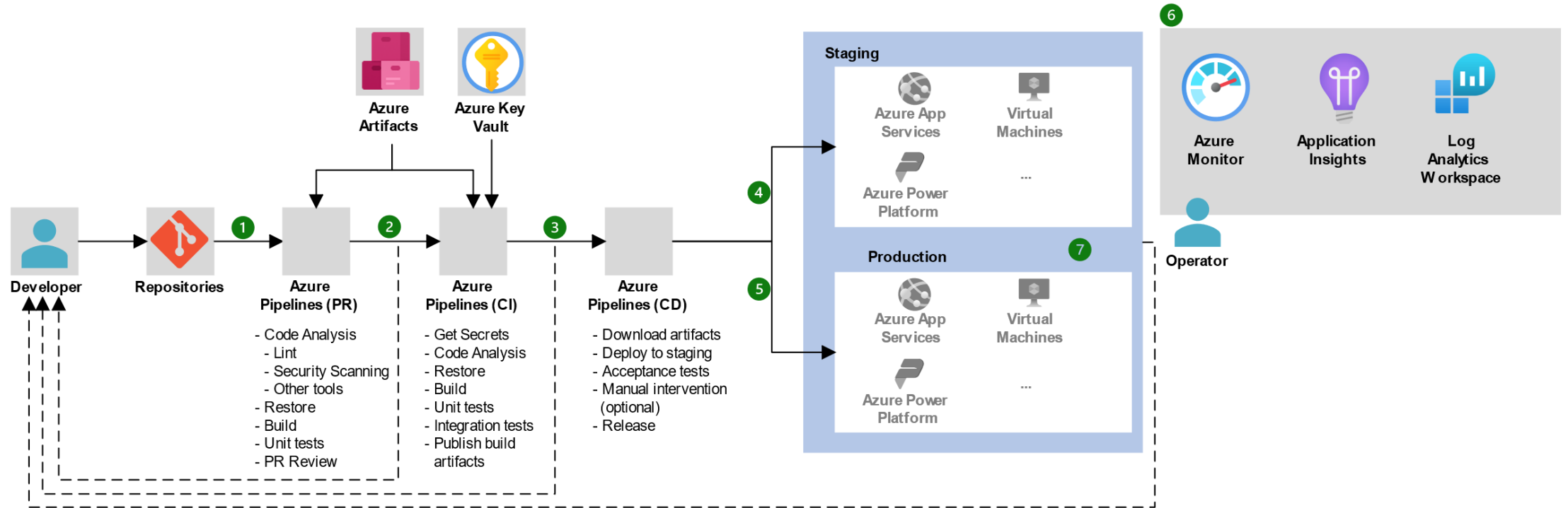
SaaS



Cloud Services



EXAMPLE CI/ CD WORKFLOW ON AZURE DEVOPS



Source: <https://learn.microsoft.com/en-us/azure/architecture/example-scenario/apps/devops-dotnet-baseline>

BENEFITS OF CONTINUOUS INTEGRATION

- Less bugs get shipped to production as regressions are captured early by the automated tests.
- Building the release is easy as all integration issues have been solved early.
- Less **context switching** as developers are alerted as soon as they **break the build** and can work on fixing it before they move to another task.
- Testing costs are reduced drastically – your CI server can run hundreds of tests in the matter of seconds.
- Your QA team spends less time testing and can focus on significant improvements to the quality culture.

BENEFITS OF CONTINUOUS DELIVERY

- The **complexity of deploying software has been taken away**. Your team doesn't have to spend days preparing for a release anymore.
- You can release more often, thus **accelerating the feedback loop** with your customers.

BENEFITS OF CONTINUOUS DEPLOYMENT

- Deployments pipelines are triggered automatically for every change.
- Releases are less risky and easier to fix in case of problem as you deploy small batches of changes.
- Customers see a continuous stream of improvements, and quality increases every day, instead of every month, quarter or year.

CI/ CD BEST PRACTICES

- Self-study activity: Read on best practices [here](#).

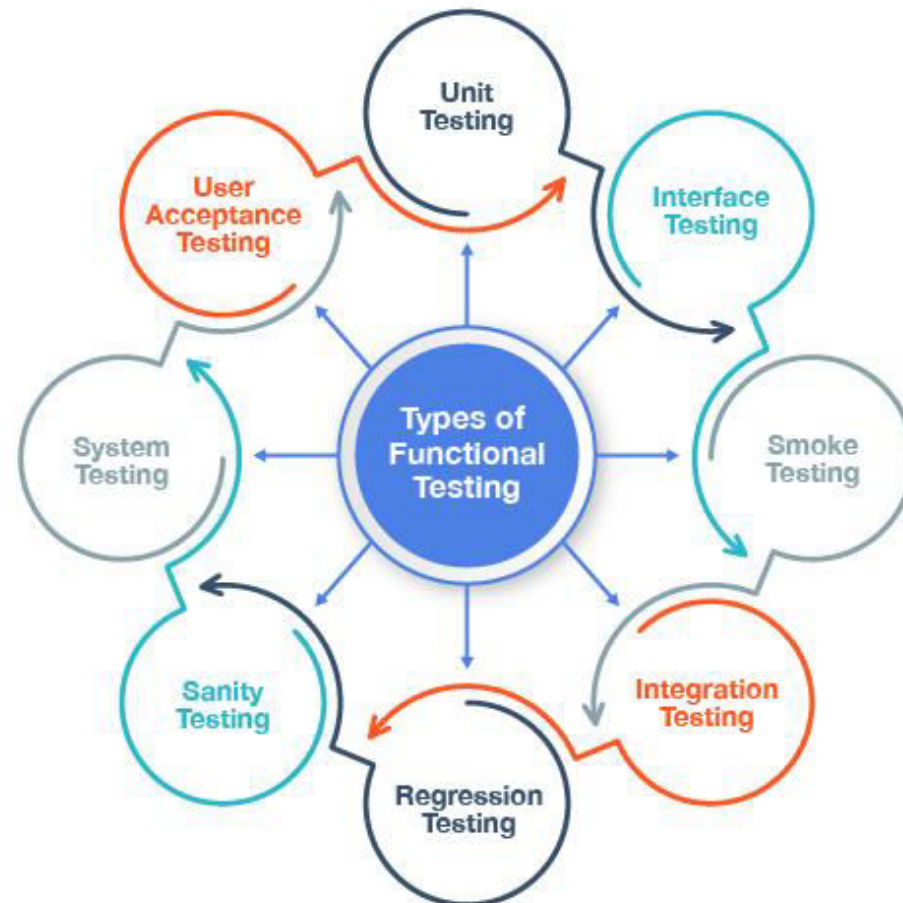
INFRASTRUCTURE AUTOMATION

- Infrastructure automation refers to the practice of using software and tools to automate the provisioning, configuration, management, and operation of IT infrastructure.
- It involves automating various tasks that would traditionally be performed manually, reducing the need for human intervention and increasing efficiency and reliability.
- [\[YouTube\] Infrastructure Automation with Red Hat Ansible](#)

SOFTWARE TESTING

- Software testing can be broadly categorized into two types.
 - Functional tests – verifies if functional requirements are met
 - Non-functional tests – verifies if non-functional requirements are met

FUNCTIONAL TEST TYPES



Source: <https://testsigma.com/blog/the-different-software-testing-types-explained/>

NON-FUNCTIONAL TEST TYPES



TEST AUTOMATION

- In continuous testing, various types of tests are performed within the CI/CD pipeline. These can include:
 - **Unit testing**, which checks that individual units of code work as expected.
 - **Integration testing**, which verifies how different modules or services within an application work together.
 - **Regression testing**, which is performed after a bug is fixed to ensure that specific bug won't occur again.
- Self study activity: Read this article on [Automated software testing](#).

WRAP UP

- [YouTube] DevOps CI/CD Explained in 100 Seconds
- Try out GitHub Actions: a CI/ CD Platform on GitHub.

SUMMARY

- Deploying an app
 - Typical Deployment Scenario
 - Failures
- DevOps
 - Core DevOps principles
 - Continuous Integration (CI)
 - Continuous Delivery and Continuous Deployment (CD)
 - Infrastructure Automation
 - Software Testing and Test Automation

REFERENCES

1. [What is CI/CD?](#)
2. [4 Must-know DevOps principles](#)
3. [Manifesto for Agile Software Development](#)
4. [How Is Netflix SO GOOD at DevOps?](#)
5. [What Is Shift Left Testing?](#)
6. [Continuous integration vs. delivery vs. deployment](#)
7. [The different types of software testing](#)



THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK



SOFTWARE QUALITY ASSURANCE (SQA) – AN OVERVIEW

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

LEARNING OUTCOMES

- After completing this lecture, you will be able to,
 - Define what Software Quality Assurance mean.
 - Differentiate between SQA and testing.
 - Discuss the importance of SQA in today's software industry.
 - Apply SQA practices in your development work.

CONTENTS

- What is Software Quality?
- Why is Software Quality Assurance (SQA) Important?
- The Role of SQA in the Software Development Life Cycle (SDLC)
- SQA Activities and Processes
 - The SQA Process
 - Building Quality into the Software Development Process
- SQA, Agile and DevOps
- Self-study activity
- Summary

WHAT IS SOFTWARE QUALITY?

- According to the IEEE Computer Society, software quality refers to,
 - how well a software product **conforms** to the given **requirements** and **meets** the **needs** of its **users**. It involves both the software **product itself** as well as the **processes** used to develop it ([source](#)).

WHAT IS SOFTWARE QUALITY?

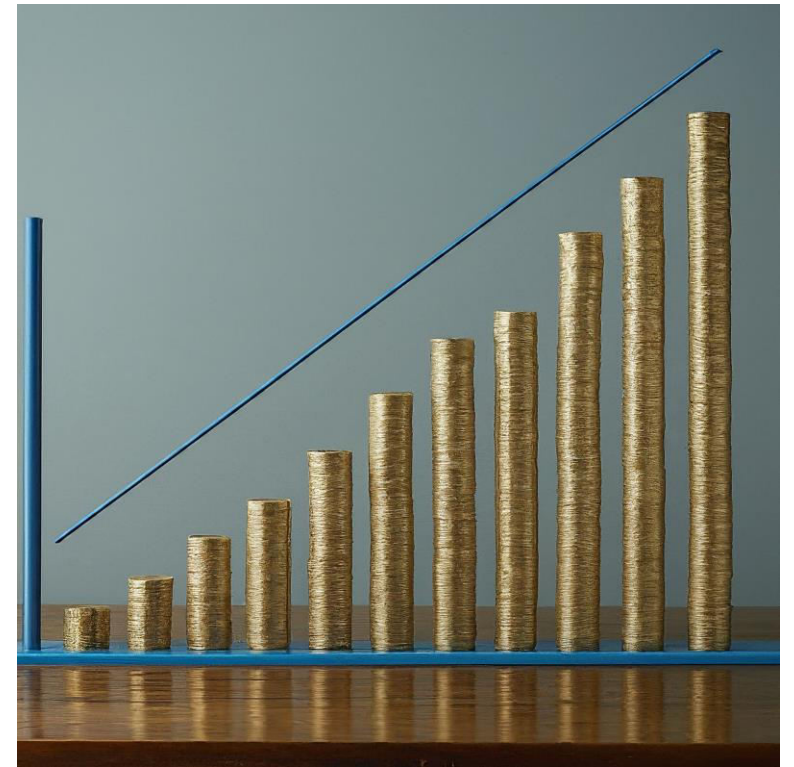
- The American Society for Quality defines Software Quality Assurance (SQA) as,
 - A **systematic approach** to evaluating the quality of and adherence to software product standards, processes and procedures. SQA includes **ensuring standards** and **procedures** are **established** and **followed** throughout the software acquisition life cycle ([source](#)).

WHY IS SOFTWARE QUALITY ASSURANCE (SQA) IMPORTANT?

- SQA is a crucial part of the software development process, **ensuring a final product** that meets **expectations**, that involves **people, processes, and technology**.
- SQA leads to a **positive user experience** by minimising failures and ensuring conformity with the user requirements. This translates to satisfied users who are more likely to recommend the product.

WHY IS SOFTWARE QUALITY ASSURANCE (SQA) IMPORTANT?

- SQA helps businesses **save money, time and reputation** by preventing or catching defects early. Fixing bugs later in the SDLC is significantly more expensive in all three aspects.
- SQA helps businesses **comply with industry standards and regulations**, reducing legal risks.



WHY IS SOFTWARE QUALITY ASSURANCE (SQA) IMPORTANT?

- **SQA emphasizes documentation**, ensuring that the requirements, processes, design decisions, technical information, test results and reports are well documented, which aids in understanding the base code and **makes maintenance easier**.

WHY IS SOFTWARE QUALITY ASSURANCE (SQA) IMPORTANT?

- SQA is **NOT** just testing!
 - SQA is a process that focuses on preventing defects through **proactive** measures throughout the SDLC and ensures overall quality.
 - Software Testing is a specific activity within SQA. It's a **reactive** approach that helps find defects, but SQA goes beyond testing to prevent them from occurring in the first place.
 - Read more on proactive vs reactive approaches to quality [here](#).



Source: <https://unsplash.com/photos/black-red-and-white-textile-hbb6GkG6p9M>

WHY IS SOFTWARE QUALITY ASSURANCE (SQA) IMPORTANT?

- Real-world examples of software failures
 - Therac-25 (6 deaths due to radiation overdose)
 - Boeing 737 Max Crashes (346 deaths, US\$20-80 Billion in costs)
 - Y2K Bug (US\$300 billion)/ Year 2038 problem (yet to occur)
 - Loss of Mars Climate Orbiter (US\$93 million)
 - Ariane 5 (US\$8 million)
 - ...how did your ITP project go? 😊



Source:

https://en.wikipedia.org/wiki/Year_2000_problem#/media/File:Bug_de_l'an_2000.jpg

THE SQA PROCESS

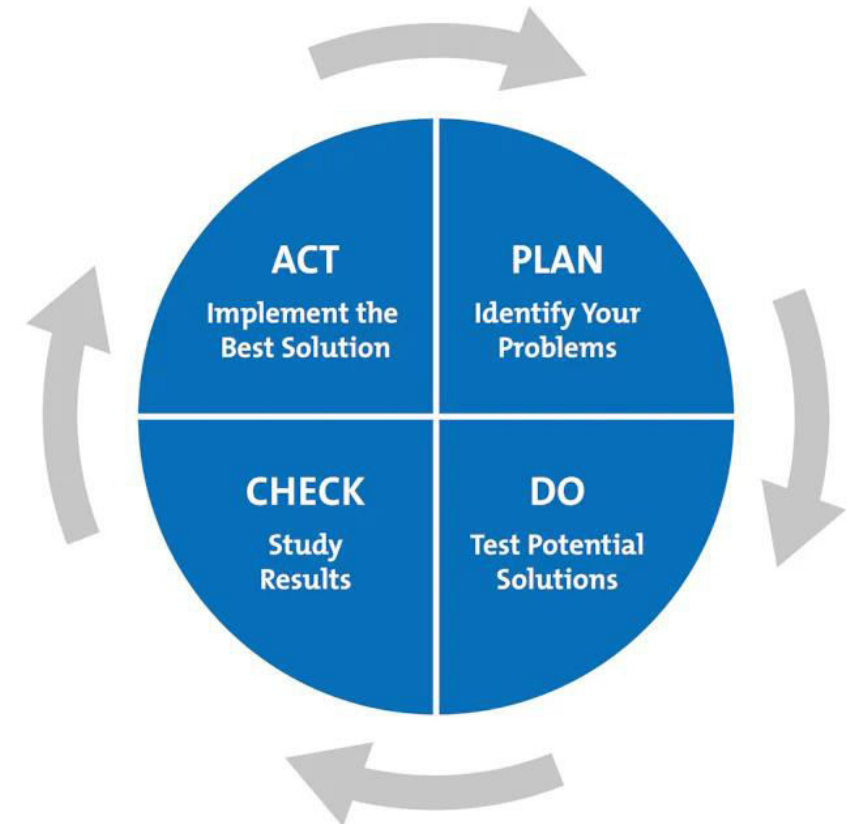
- Identify Requirements
- Establish quality standards based on the requirements
 - Clearly define the quality standards that the software product must meet. This includes defining requirements, acceptance criteria, and performance metrics.
- Plan SQA activities
 - Process for problem reporting and corrective action
 - Software artefact management
 - Process reviews/ audits
 - Testing methodology

THE SQA PROCESS

- Execute the SQA plan
 - Conduct reviews of software artefacts
 - Perform testing (repeatedly) – use automation to increase efficiency.
 - Conduct process reviews
- Monitor the quality of the product
 - This should be done throughout the SDLC.
 - Defects must be tracked
 - Analyze metrics such as code coverage and defect density
 - Conduct root cause analysis

THE SQA PROCESS

- Continuously improve the SQA process
 - Use the findings from the process reviews and monitoring data to identify areas for improvement and implement changes to the SQA process.
 - Follow the Plan-Do-Check-Act (PDCA) Cycle.



Source: <https://www.mindtools.com/as215i1/pdca-plan-do-check-act>

BUILDING QUALITY INTO THE SOFTWARE DEVELOPMENT PROCESS

■ Define Clear Requirements

- Start by clearly defining and documenting the requirements for the software. This ensures that everyone involved understands what needs to be built and what constitutes success. Effective requirement management is critical for delivering a successful software product.

■ Adopt Agile Methodologies

- Agile methodologies like Scrum or Kanban promote iterative development, frequent collaboration, and continuous improvement. This allows for early detection and resolution of issues, leading to higher-quality software.

BUILDING QUALITY INTO THE SOFTWARE DEVELOPMENT PROCESS

■ Design Reviews

- Design reviews involve a methodical examination of the software design. The goal is to identify and address any defects, inconsistencies, or areas for improvement in the design before coding begins. This proactive approach helps to prevent errors from being coded into the software and reduces the time and effort required for fixing problems later in the development process.

■ Implement Code Reviews

- Regular code reviews help catch issues early in the development process. They also facilitate knowledge sharing among team members and ensure adherence to coding standards and best practices.

BUILDING QUALITY INTO THE SOFTWARE DEVELOPMENT PROCESS

■ Automate Testing

- Automated testing, including unit tests, integration tests, and end-to-end tests, helps identify defects quickly and consistently. Continuous integration and continuous deployment (CI/CD) pipelines can automate the testing process, enabling faster feedback loops.

■ Use Version Control

- Version control systems like Git enable teams to track changes to the codebase, collaborate effectively, and revert to previous versions if needed. This promotes transparency and reduces the risk of introducing errors.

BUILDING QUALITY INTO THE SOFTWARE DEVELOPMENT PROCESS

■ Prioritize Security

- Incorporate security practices throughout the development process, including threat modelling, code scanning, penetration testing, and security reviews. Security should be considered at every stage, from design to deployment.

■ Focus on User Experience (UX)

- Design the software with the end user in mind. Conduct user research, gather feedback, and iterate based on user needs and preferences. A well-designed user interface enhances usability and overall satisfaction.

BUILDING QUALITY INTO THE SOFTWARE DEVELOPMENT PROCESS

- Document everything
 - In addition to the standard documentation (SRS, test plans etc.), document the thought process behind technical decisions. This should be a continuous process. This enables proactive identification of any issues during reviews as well as making it easy for anybody new to get up to speed quickly.
- Continuous Monitoring and Feedback
 - Implement monitoring and logging mechanisms to track system performance, errors, and user behaviour in real time. This enables proactive identification of issues and opportunities for optimization.

BUILDING QUALITY INTO THE SOFTWARE DEVELOPMENT PROCESS

- Invest in Training and Education

- Provide ongoing training and professional development opportunities for team members to stay updated on emerging technologies, best practices, and industry trends. Knowledgeable and skilled teams are better equipped to deliver high-quality software.

- Promote a Culture of Quality

- Foster a culture where quality is everyone's responsibility. Encourage open communication, collaboration, and accountability among team members. Recognize and reward efforts that contribute to maintaining and improving software quality.

SQA, AGILE AND DEVOPS

- Traditional approach (Waterfall SDLC)
 - It is a sequential process where each phase of the software development lifecycle is completed before moving on to the next phase.
 - SQA is performed at the end of each phase to ensure that the requirements have been met before moving to the next phase.
 - This approach involves requirement analysis, design, coding, testing, and maintenance to ensure that the software product is developed with minimal errors and defects and meets the desired quality standards.

SQA, AGILE AND DEVOPS

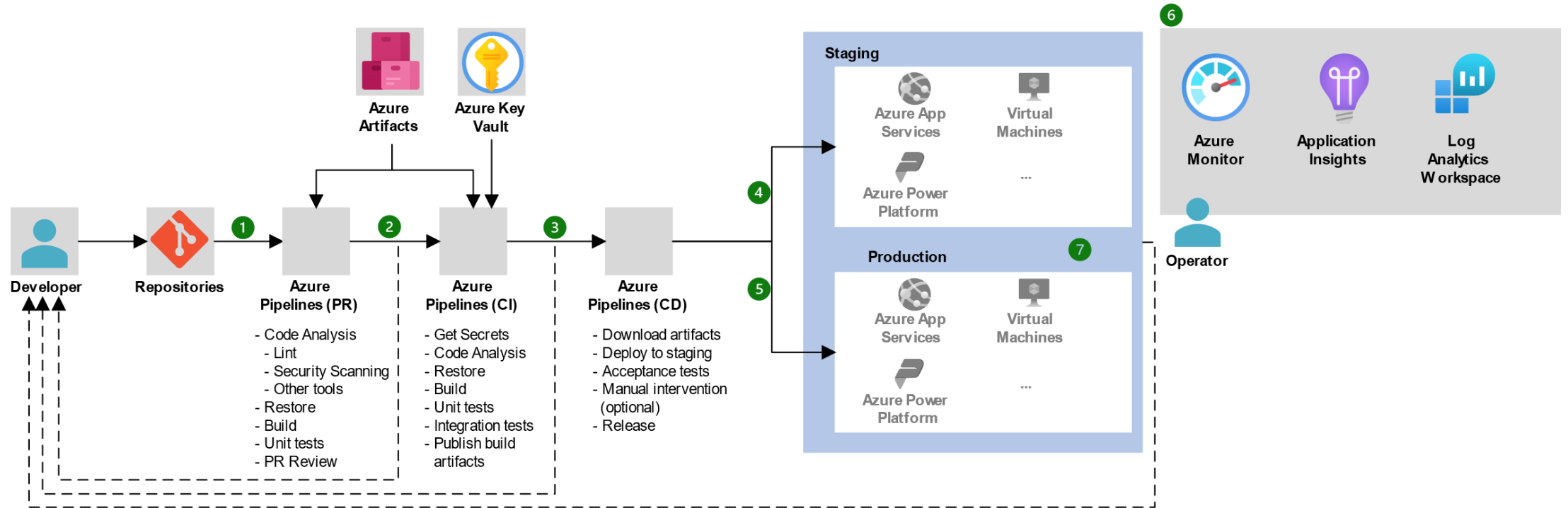
- Agile approach
 - The Agile approach to SQA is an iterative, incremental, and flexible approach that focuses on delivering software products in small increments.
 - This approach emphasizes **collaboration** between the development team and the stakeholders for a seamless and quick development process.
 - Agile SQA is quite popular and focuses on self-organizing teams, continuous integration and testing, continuous delivery, and continuous feedback to ensure a high-quality software product.

SQA, AGILE AND DEVOPS

■ DevOps approach

- This approach promotes **collaboration** between development and IT operations to ensure that the software product meets the requirements of the customers.
- DevOps is focused heavily on automation, such as continuous integration, continuous testing, and continuous deployment to deliver a high-quality product.
- This approach is great for projects that require frequent updates.

EXAMPLE CI/ CD WORKFLOW ON AZURE DEVOPS



Source: <https://learn.microsoft.com/en-us/azure/architecture/example-scenario/apps/devops-dotnet-baseline>

SELF-STUDY ACTIVITY

- Refer to the following material and identify how SQA could have prevented them.
 - CASE STUDY - THERAC-25
 - HOW THE BOEING 737 MAX DISASTER LOOKS TO A SOFTWARE DEVELOPER

SUMMARY

- What is Software Quality?
- Why is Software Quality Assurance (SQA) Important?
- The Role of SQA in the Software Development Life Cycle (SDLC)
- SQA Activities and Processes
 - The SQA Process
 - Building Quality into the Software Development Process
- SQA, Agile and DevOps

REFERENCES

1. What is Software Quality?
2. What Is Software Quality Assurance? A Step-By-Step Guide And Importance
3. What Is Software Quality Assurance, and Why Is It Important?



THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK

SASINI.H@SLIIT.LK

ENTERPRISE SOFTWARE ENGINEERING

NIPUN USWATTE

Software Engineer & Technical Trainer
IFS Academy - IFS R&D International (Pvt) Ltd

Welcome

- Timings
- Ground rules
- Objectives



Agenda

- 01 Introduction to Enterprise Software Engineering
- 02 Software Development Life Cycle (SDLC)
- 03 Architectural Considerations in Enterprise Software Engineering
- 04 Scalability and Performance
- 05 DevOps and CI/CD
- 06 Quality Assurance and Testing
- 07 Collaboration and Communication

- The process of developing, deploying, and maintaining software systems that are used to support the operations of large organizations or enterprises.
- These systems are typically
 - Complex
 - Mission-critical
 - Often span multiple departments or functions within an organization

- Enterprise Resource Planning (ERP)
- Customer Relationship Management (CRM)
- Supply Chain Management (SCM)
- Human Capital Management (HCM)
- Business Intelligence (BI) and Analytics
- Enterprise Content Management (ECM)
- Project Portfolio Management (PPM)
- Enterprise Asset Management (EAM)

- Scale
- Complexity
- Integration
- Security
- Scalability
- Reliability and Availability

- Efficiency
- Innovation
- Customer Experience
- Compliance and Governance

- Technological Complexity
- Legacy Systems
- Security Threats
- Agility and Innovation
- Cloud Computing

Software Development Life Cycle (SDLC)

- “A structured process used by software development teams to design, develop, test, deploy, and maintain high-quality software systems.”
- **It provides** a framework for managing the entire software development process, from initial planning to final delivery and ongoing maintenance.
- **Primary goal** is to ensure that software projects are completed on time, within budget, and meet the specified requirements and quality standards.

Software Development Life Cycle (SDLC)



SDLC Tutorial (YouTube): [WATCH ME](#)

- Formalized Processes
- Cross-functional Collaboration
- Iterative and Agile Approaches
- Governance and Oversight

Agile Tutorial (YouTube): [WATCH ME](#)

- Improved Quality
- Predictable Delivery
- Reduced Risks
- Enhanced Collaboration
- Continuous Improvement

- Refer to the principles, patterns, and decisions that guide the design and implementation of software systems for large organizations or enterprises.
- These considerations address the
 - Structural organization
 - Interaction patterns
 - Deployment strategies of enterprise software.

Ensure scalability, maintainability, security, and other critical quality attributes.

1. Enterprise Architecture

2. Architectural Patterns

- **Monolithic Architecture**

- A single, unified system where all components are tightly coupled and deployed together.

- **Microservices Architecture**

- Decomposes the application into small, loosely coupled services that can be developed, deployed, and scaled independently.

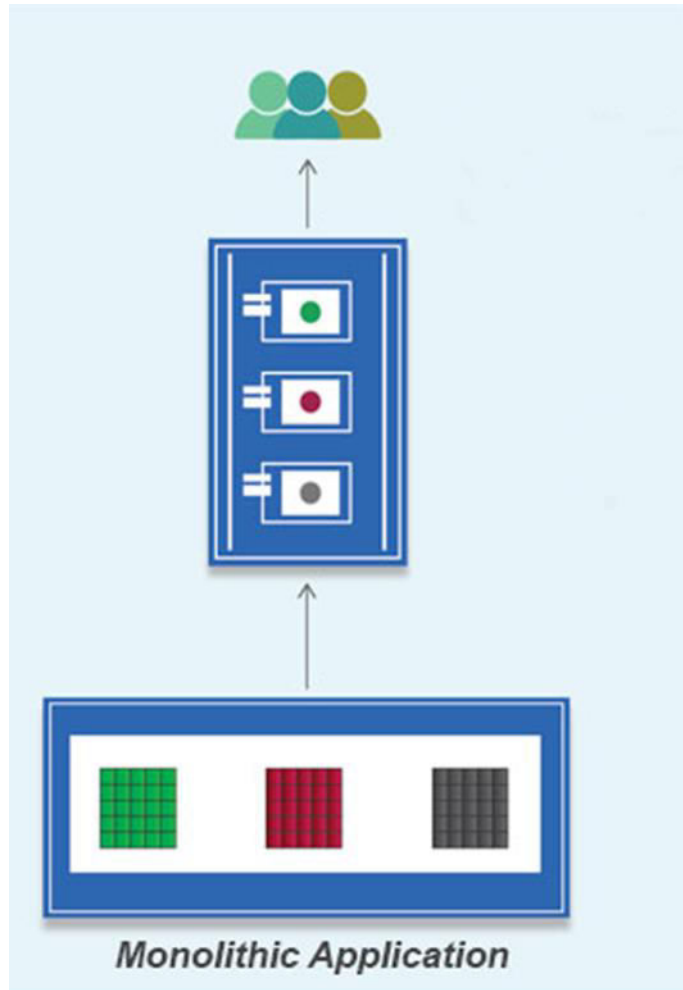
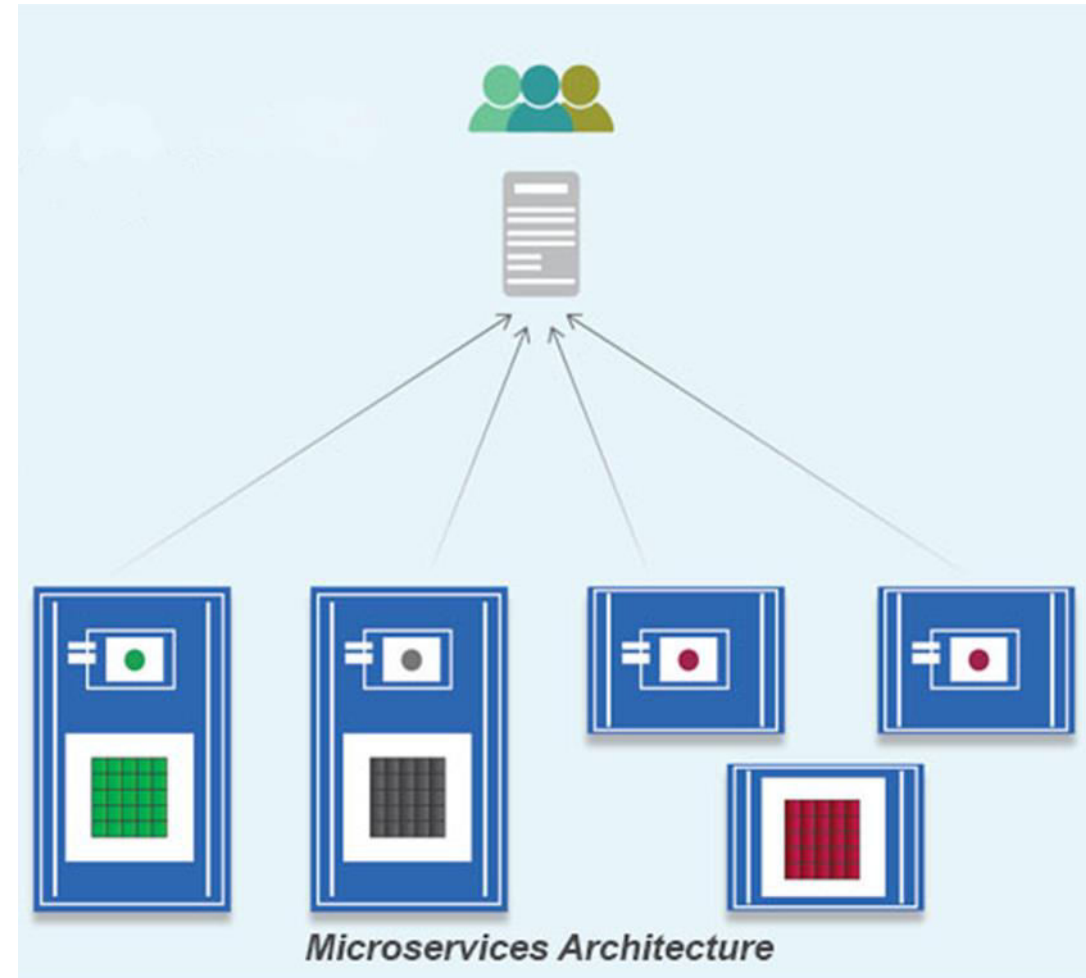
- **Service-Oriented Architecture (SOA)**

- Organizes software components as services that communicate via standardized protocols, promoting interoperability and reusability.

03

Architectural Considerations in ESE

Monolithic Architecture vs Microservices Architecture

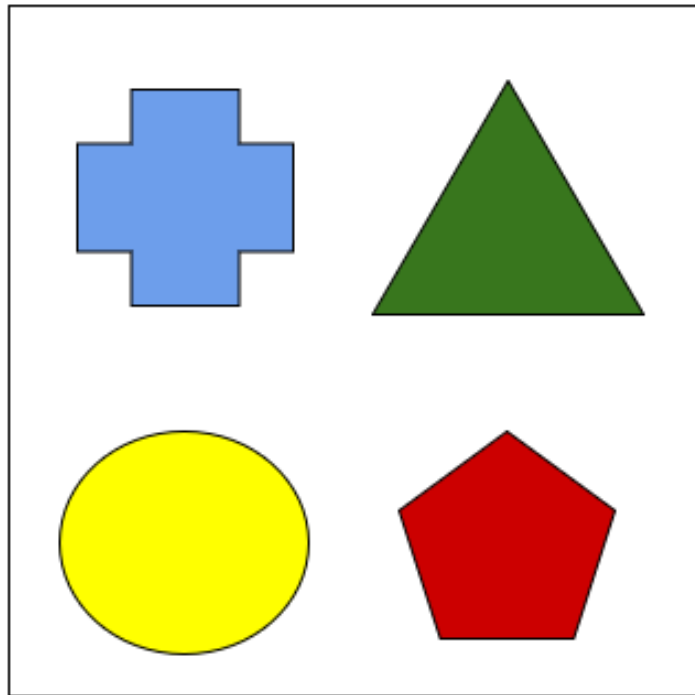
**vs**

03

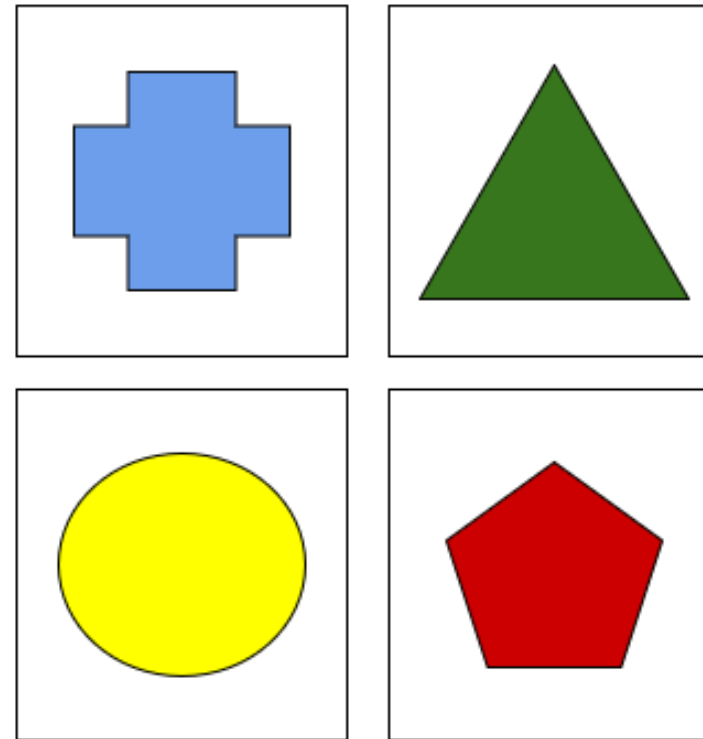
Architectural Considerations in ESE

Monolithic Architecture vs Microservices Architecture Cont.

A monolith application puts all its functionality into a single process



A microservices architecture puts each element of functionality into a separate process

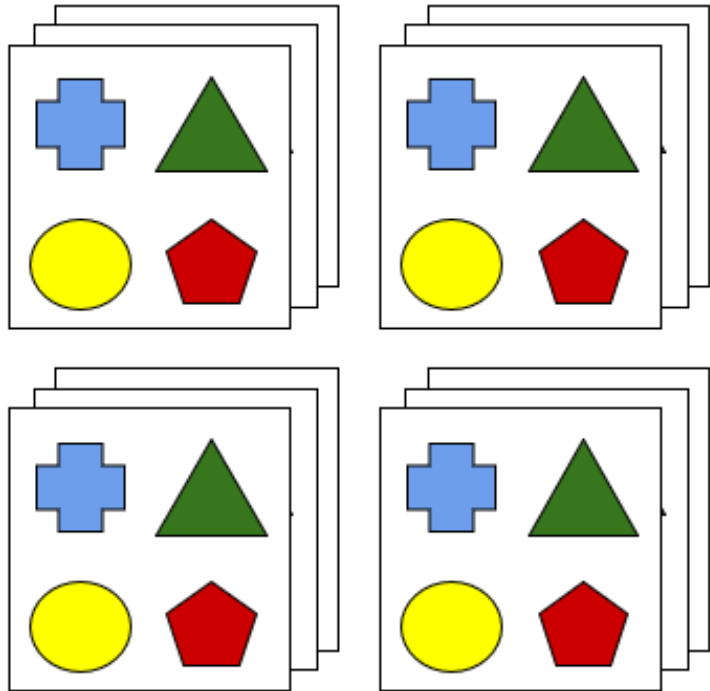


03

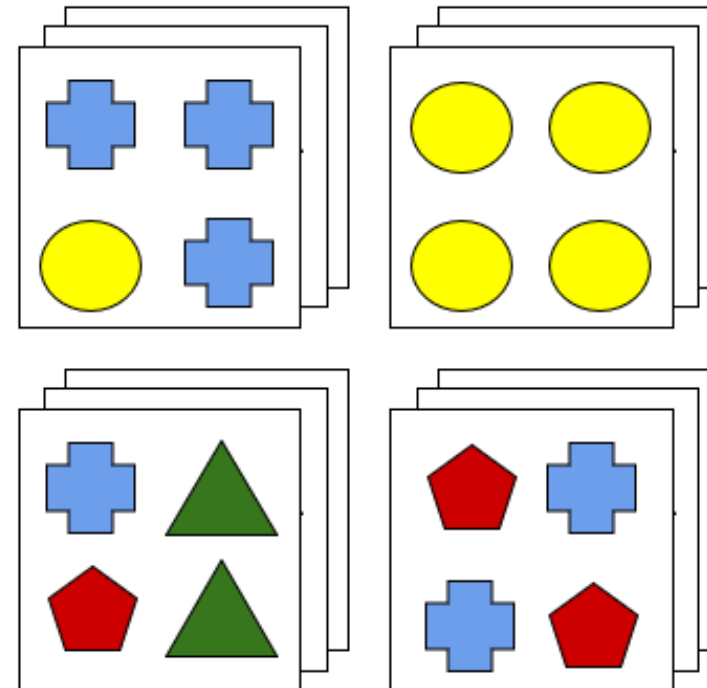
Architectural Considerations in ESE

Monolithic Architecture vs Microservices Architecture Cont.

Monolith scales by replicating the whole thing on multiple servers

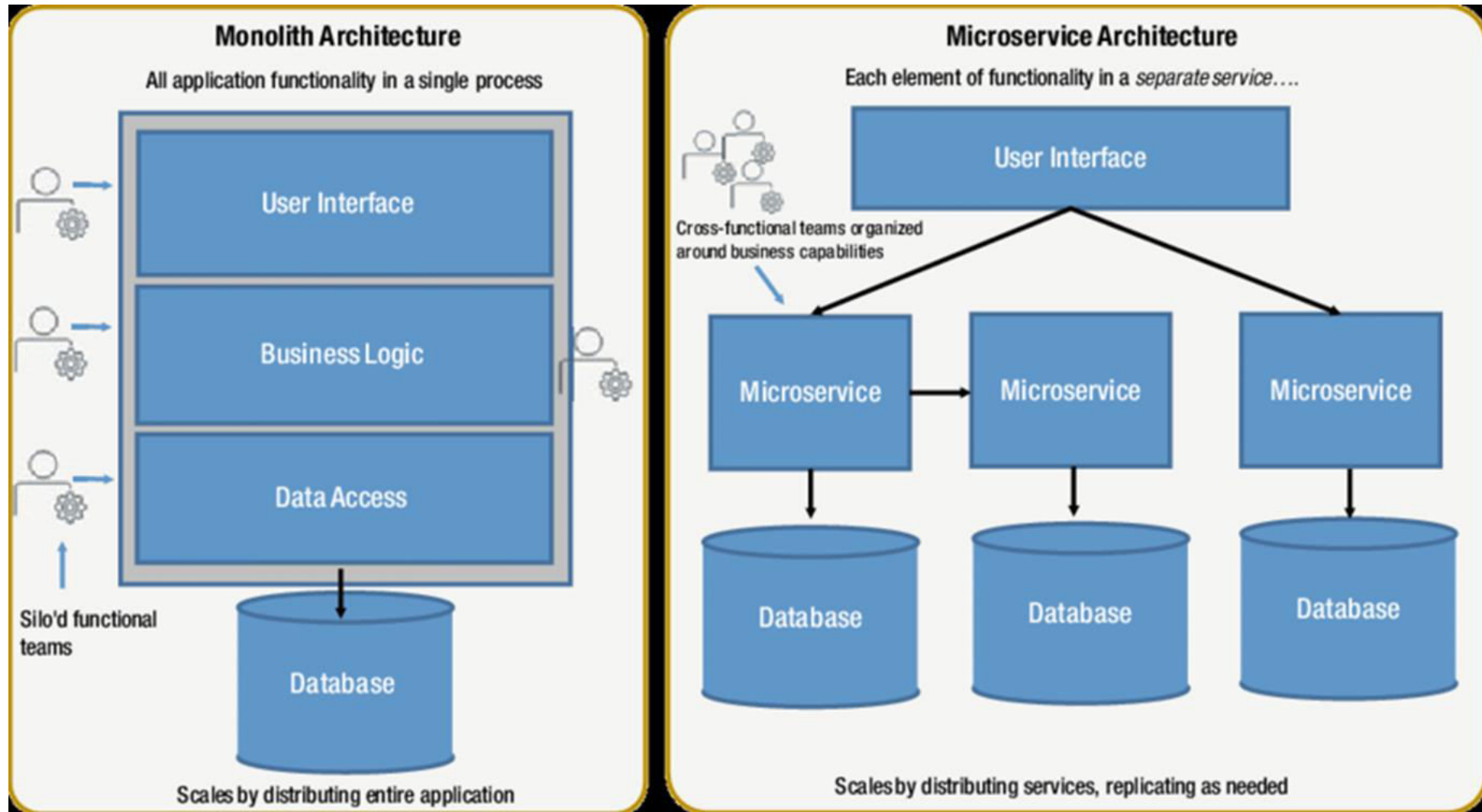


Microservices scales by distributing these services across servers, replicating as needed.

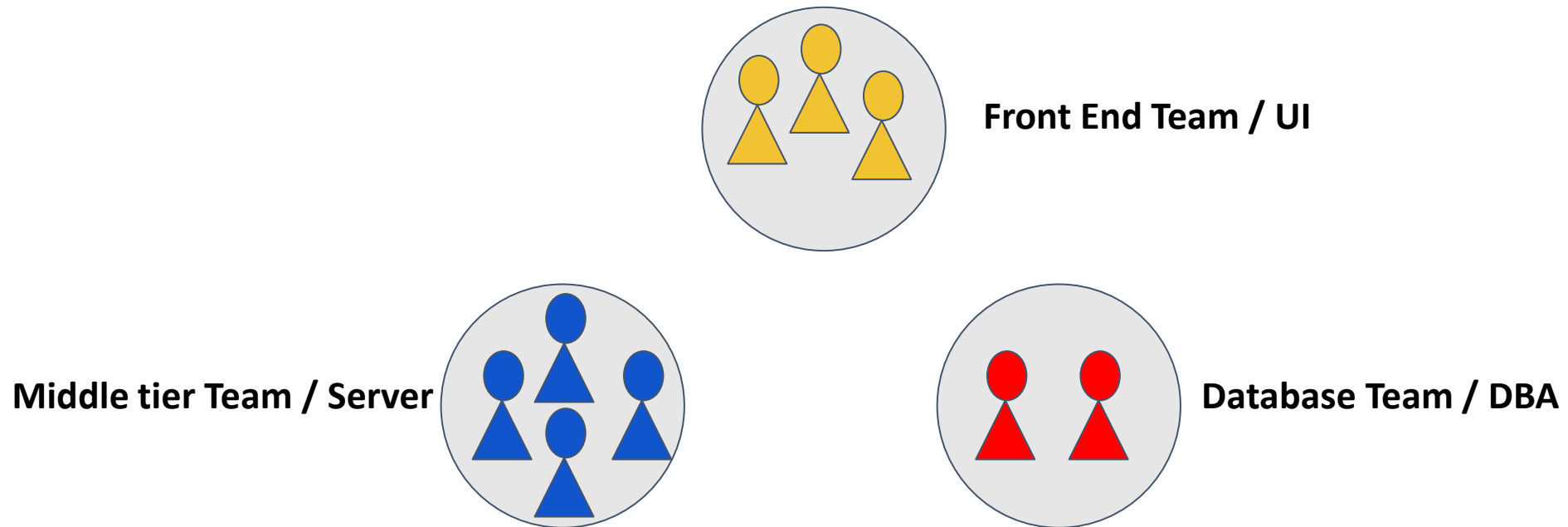


Architectural Considerations in ESE

Monolithic Architecture vs Microservices Architecture Cont.



- Most of the software development organizations organize the team according to the technology.

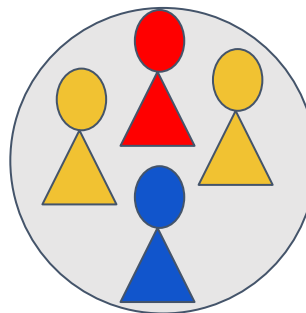


- But when implementing a solution according to the Microservices approach, you should organize the team according to the business capabilities.

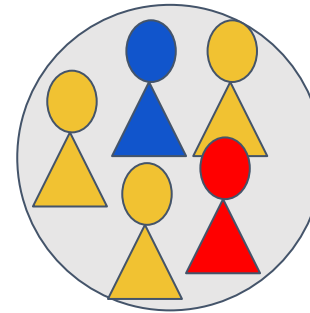
Order Catalog



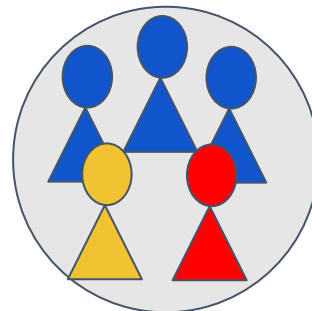
Shipping



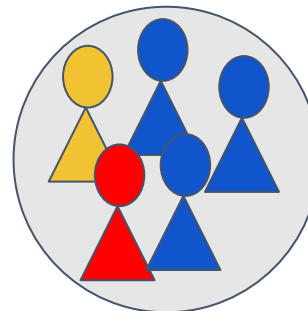
Finance



Inventory



Recommendation Engine



3. Scalability

- Scalability refers to the ability of a system to handle increasing workloads and user demands without sacrificing performance or reliability.
 - **Horizontal Scaling**: Adding more instances of components to distribute the load.
 - **Vertical Scaling**: Increasing the resources (e.g., CPU, memory) of individual components.
 - **Partitioning and Sharing**: Splitting data and workload across multiple nodes or databases.

4. Security

- Defense-in-Depth
 - Implementing multiple layers of security controls (e.g., firewalls, encryption, access controls) to protect against various attack vectors.
- Secure Authentication and Authorization
 - Implementing strong authentication mechanisms and fine-grained access controls to ensure only authorized users can access resources.
- Secure Communication
 - Using encryption and secure communication protocols (e.g., HTTPS, SSL/TLS) to protect data in transit.

5. Maintainability

- Modular Design
 - Breaking down the system into smaller, cohesive modules with well-defined interfaces.
- Separation of Concerns
 - Ensuring each component or module has a clear and distinct responsibility, making it easier to understand and modify.
- Loose Coupling
 - Minimizing dependencies between components to reduce the impact of changes and facilitate independent development and testing.

- Ability of a system to handle increasing workloads and user demands without sacrificing performance, reliability, or availability.

Business Growth

Must be able to accommodate the growth of the organization, including increasing user base, transaction volume, and data size.



Seasonal or Peak Loads

Many enterprise applications experience fluctuating workloads due to seasonal demand, marketing campaigns, or events.



04

Scalability and Performance

Importance of Scalability in Enterprise Applications Cont.

Cost Efficiency



Scalability allows organizations to optimize resource utilization and avoid overprovisioning of hardware or infrastructure.



Competitive Advantage

In today's fast-paced business environment, agility and responsiveness are essential for maintaining a competitive edge.

Scalability and Performance

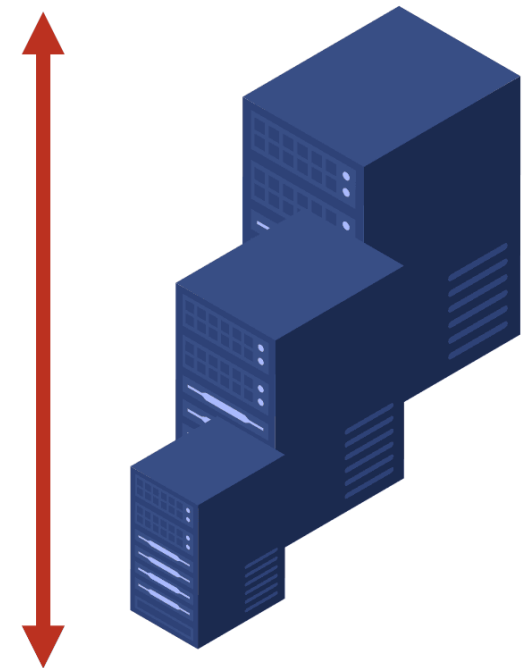
Techniques for Designing Scalable Systems

1. Vertical Scaling – Scaling Up

- Process of increasing the capacity or capabilities of an individual hardware or software component within a system.
- Aims to improve the performance and capacity of the system to handle higher loads or more complex tasks without changing the fundamental architecture or adding additional servers.

Advantages of Vertical Scaling

- Increased capacity
- Easier management



2. Horizontal Scaling – Scaling Out

- Process of increasing the capacity or performance of a system by adding more machines or servers to distribute the workload across a larger number of individual units.
- There is no downtime while adding more servers to the network

Advantages of Vertical Scaling

- Increased capacity
- Improved performance
- Increased fault tolerance



3. Decomposition into Microservices

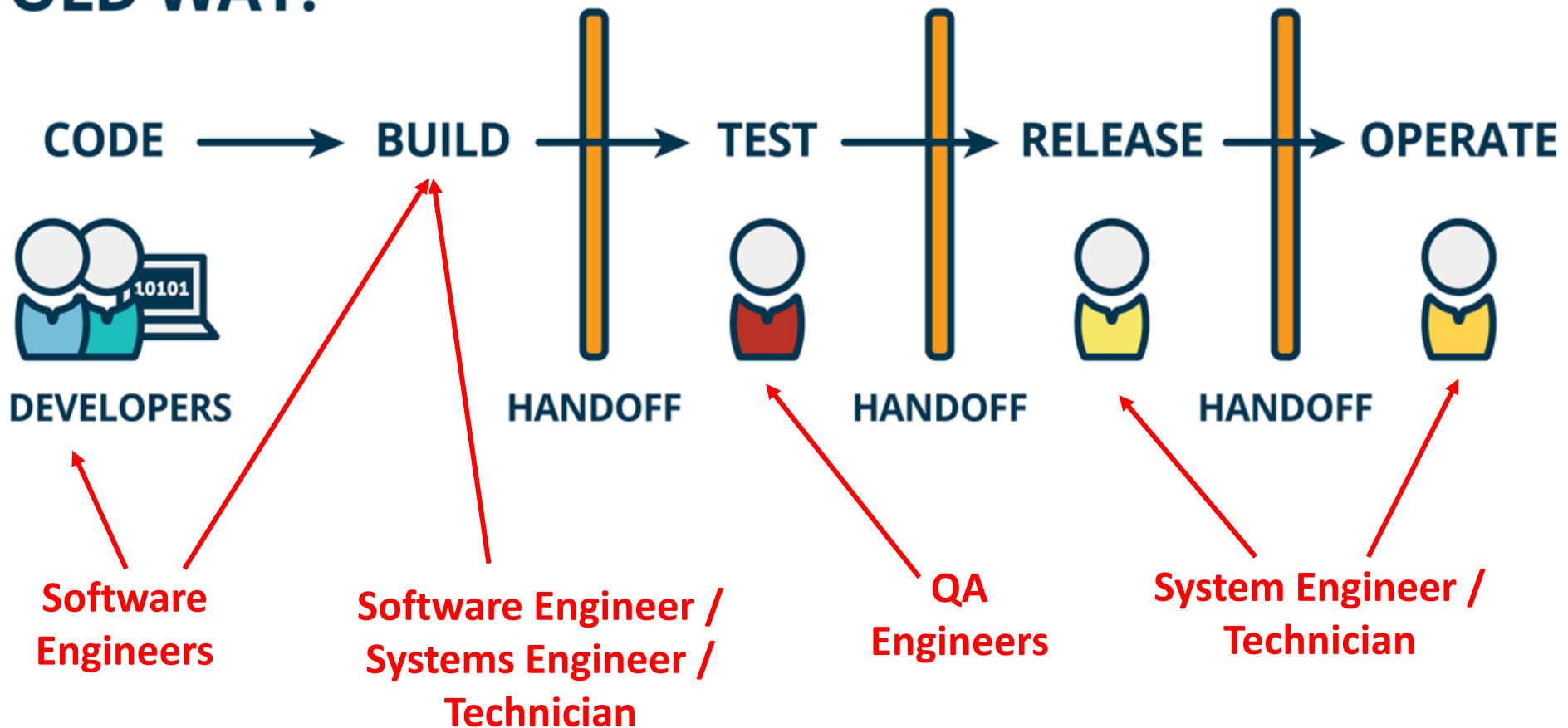
- Decomposing monolithic applications into smaller, loosely coupled microservices allows for independent scaling of individual components.
- Microservices can be deployed and scaled independently, enabling better resource utilization and flexibility.



4. Use of Distributed Architectures

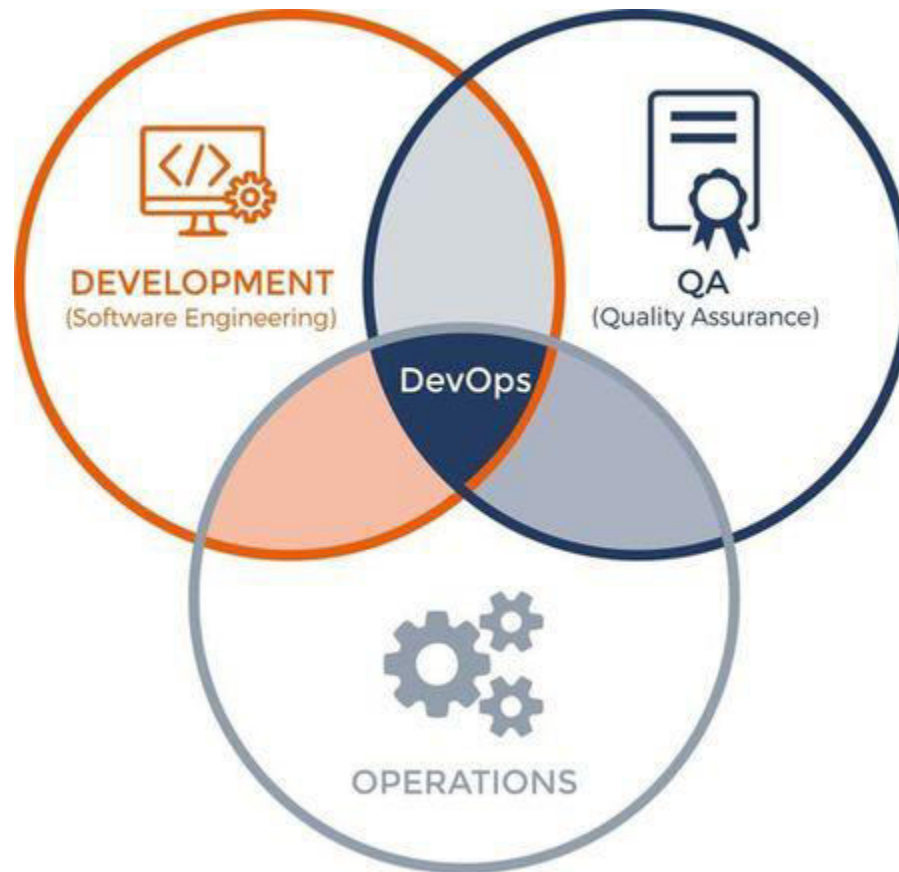
- Distributed architectures, such as distributed databases, messaging systems, and caching layers, enable data and processing to be distributed across multiple nodes or clusters.
- This reduces bottlenecks and single points of failure, improving scalability and fault tolerance.



OLD WAY:

- Deliveries are usually very large due to big chunks of code.
- Usually, lot of manual process to be involved and slow progress.
- Takes long time to deliver a new features and bug fixes.
- Developer environments, build environments and test and production environments are not identical.
- Due to mismatches of environments unexpected errors can occur in each hand off phase.
- Developers may not be aware of build lifecycle and testing aspects.

- DevOps is a set of practices that combines software development and IT operations.
- Its main goal is to unify software development and software operations by bridging the gap between them.
- QA activities are also integrated with this model so that one can argue it helps to unify the processes of software development, quality assurance and operations as well.
- This helps to increase frequency of deliveries and services offered.



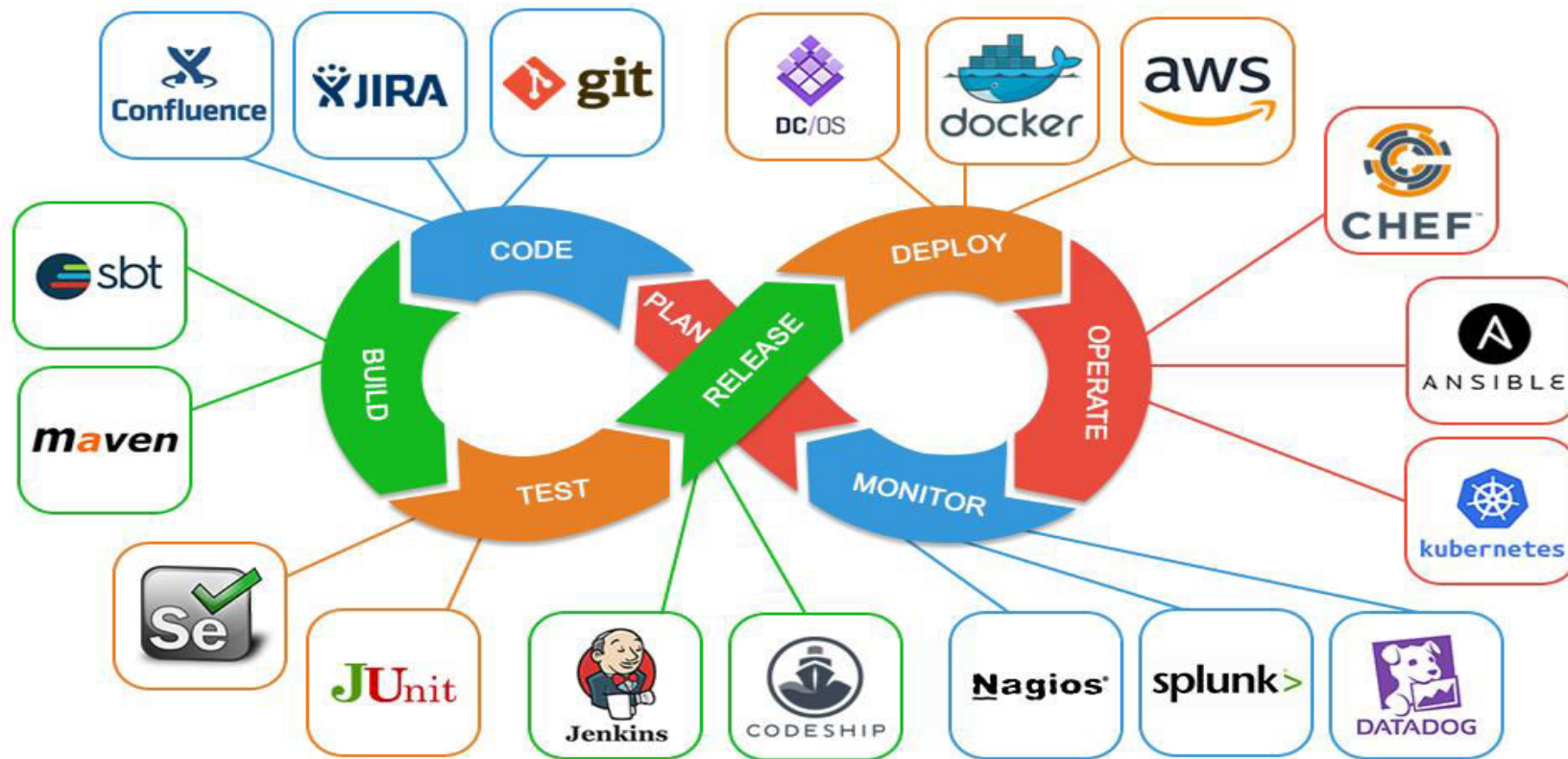
- Deliveries are usually very small and frequent.
- Almost all the steps are automated using various tools.
- Takes shorter time to deliver a functionality and fixes.
- Developer environments, build environments and test and production environments are identical.
- Therefore, delivery process hardly fails due to environment specific issues.
- Developers are aware and capable of build lifecycle and testing aspects.

- DevOps is about removing the barriers between development and operations.
- With DevOps multiple teams can work together to optimize productivity of developers and reliability of operations.
- Transitioning to DevOps requires a change in culture and mindset.
- In addition to that it needs several tools to automate most of the development and operational phases which were traditionally done as a repetitive manual processes.

05

DevOps and CI/CD

Phases of DevOps Lifecycle & Tools

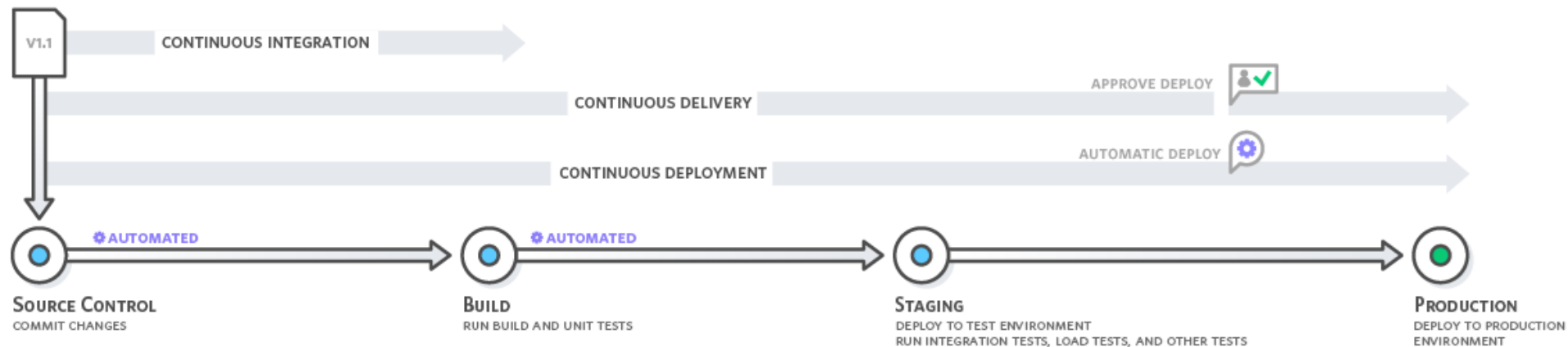


- Continuous Integration
- Continuous Delivery
- Continuous Deployment
- Monitoring and Logging
- Automated Testing
- Infrastructure as code

05

DevOps and CI/CD

Continuous Integration



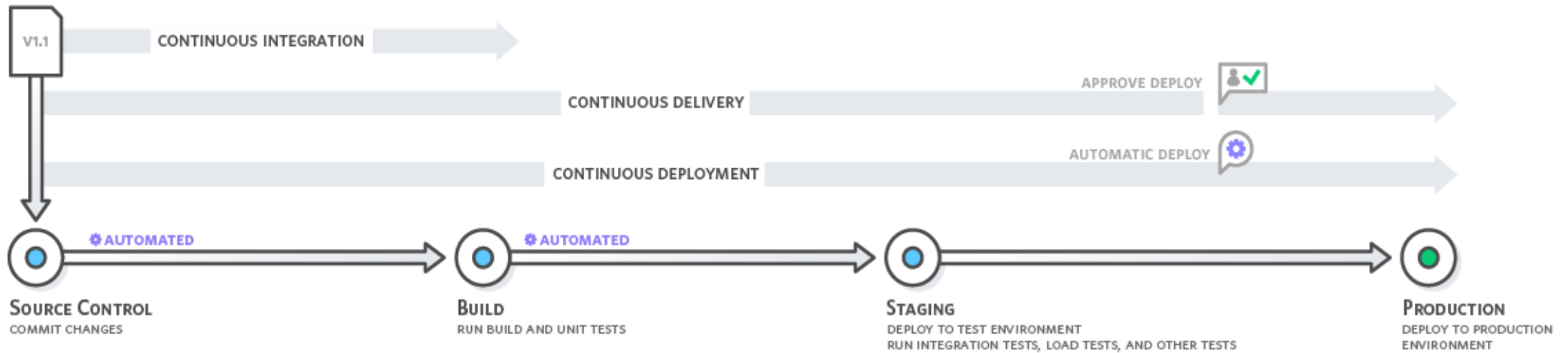
- Developers regularly merge code changes into a central repository and every such revision committed triggers an automated build and test.
- Number of tools are available which can pull the checked in code to prepare a build.

- The continuous delivery process deploy the deliveries to a series of test environments where last environment being installed should be a mirror of a production environment.
- Ability to perform the User Acceptance Testing.
- Builds up the team's confidence to apply it in the production environment.
- Benefits
 - Faster delivery of new features and bug fixes
 - Faster software release process
 - Increase the productivity of the developers
 - Faster Mean Time To Resolution (MTTR)
 - Reduce costs

05

DevOps and CI/CD

Continuous Deployment



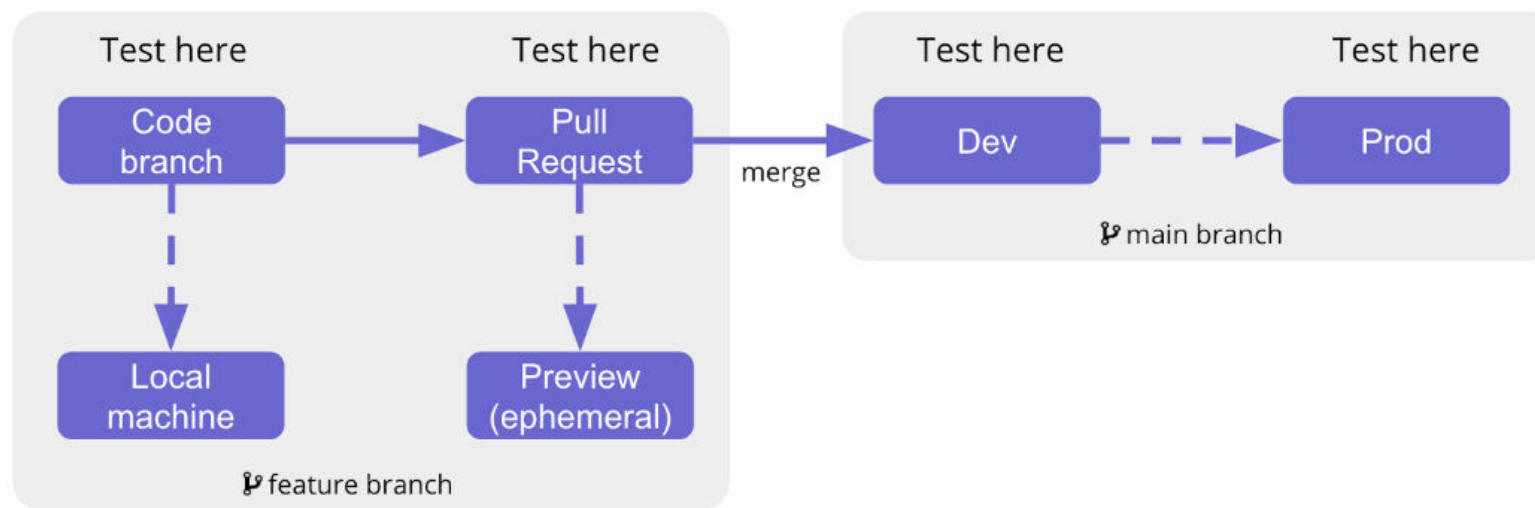
- Perform the same tasks of Continuous Delivery and go further by automatically deploying the release into the actual production environment.
- Here the delivering to production environment happens automatically without explicit approval.

- DevOps monitoring entails overseeing the entire development process from planning, development, integration and testing, deployment, and operations.
- It involves a complete and real-time view of the status of applications, services, and infrastructure in the production environment.
- Features such as real-time streaming, historical replay, and visualizations are critical components of application and service monitoring.
- Shift-left testing
- Alert and incident management
- Monitoring and Logging

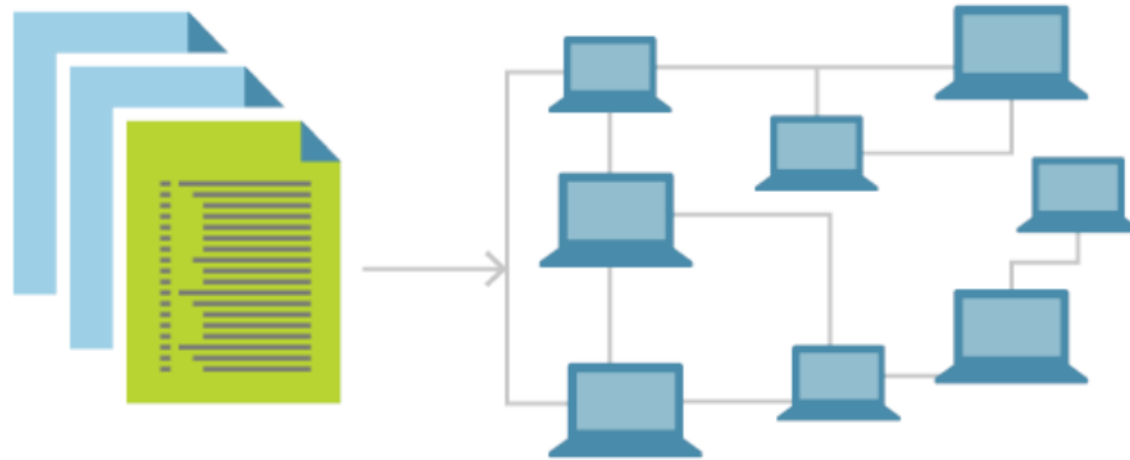
05

DevOps and CI/CD

Automated Testing



- A DevOps best practice is to run automated tests as early and as often as possible within the CI/CD pipeline.
- Run automated UI tests in production to proactively monitor for user experience issues.
- Get the ability to early detection of issues.



- Infrastructure as Code (IaC) is the managing and provisioning of infrastructure through code instead of through manual processes
 - Keep infrastructure definitions as source code files
 - Version controlled
 - Easily create and delete infrastructure resources
 - No manual work

- QA ensures that software meets quality, reliability, and usability standards.
- **Business Impact:** Quality software supports core business processes and decision-making.
- **Customer Satisfaction:** High-quality software enhances user satisfaction and trust.
- **Compliance and Regulation:** QA ensures compliance with regulatory requirements and standards.
- **Risk Mitigation:** QA identifies and mitigates risks associated with defects and failures.
- **Continuous Improvement:** QA drives ongoing improvement in processes and practices.

- **Unit Testing**

- Involves testing individual components or modules of the software in isolation to ensure they function correctly.

- **Integration Testing**

- Verifies that individual components or modules work together as expected when integrated into the larger system.

- **System Testing**

- Tests the entire software system as a whole to ensure it meets specified requirements and behaves as intended.

- **Performance Testing**

- Evaluates the performance, scalability, and reliability of the software under various load conditions.

- **Security Testing**

- Assesses the security posture of the software system to identify vulnerabilities, weaknesses, and compliance issues.

- **Regression Testing**

- Ensures that new changes or updates to the software do not introduce unintended side effects or break existing functionality.

- **Test Planning and Management:** Define objectives, scope, strategies, and manage testing activities.
- **Test Execution and Reporting:** Run tests, analyze results, and report findings to stakeholders.
- **Test Automation:** Automate testing tasks to improve efficiency and coverage.
- **Defect Management:** Identify, track, and resolve defects discovered during testing.

- **Early Testing:** Start testing as early as possible in the development lifecycle.
- **Continuous Testing:** Integrate testing throughout the software delivery pipeline.
- **Test Data Management:** Ensure availability, quality, and relevance of test data.
- **Collaboration and Communication:** Foster teamwork and alignment between teams.
- **Feedback and Metrics:** Collect feedback and monitor key metrics to track progress.
- **Continuous Improvement:** Embrace a culture of ongoing optimization and innovation.

- Distributed teams are becoming increasingly common in enterprise software engineering, with team members working remotely from different locations.
- Challenges:
 - Communication barriers due to distance, time zones, and cultural differences.
 - Lack of face-to-face interaction leading to misunderstandings and misalignment.
 - Difficulty in building trust and rapport among team members who may never meet in person.

- Use of Collaboration Tools.
- Establish Clear Communication Channels.
- Encourage Transparent Communication.
- Promote Documentation and Knowledge Sharing.
- Build Trust and Camaraderie.

- Project Management Tools:
 - Examples: Jira, Asana, Trello, Monday.com
 - Functionality: Task tracking, project planning, sprint management, issue tracking, and progress reporting.
 - Benefits: Centralized project management, visibility into project status, collaboration among team members, and alignment with Agile methodologies.

Jira Tutorial (YouTube): [WATCH ME](#)

- Version Control Systems (VCS):
 - Examples: Git, Subversion (SVN), Mercurial
 - Functionality: Versioning, branching, merging, collaboration, and change tracking for source code and other artifacts.
 - Benefits: Collaboration among distributed teams, code review processes, traceability of changes, and rollback capabilities.

- Team Collaboration Tools:
 - Examples: Slack, Microsoft Teams, Discord
 - Functionality: Real-time messaging, file sharing, video conferencing, channel-based communication, and integrations with other tools.
 - Benefits: Seamless communication, virtual collaboration, team alignment, and integration with project management and version control tools.



Case Studies and Real-world Examples

- **Netflix** relies heavily on ESE principles to manage its vast library of content, personalize recommendations, and ensure a seamless streaming experience for users.
- **Salesforce** relies on ESE principles to deliver scalable, secure, and customizable Customer Relationship Management (CRM) solutions to businesses of all sizes.
- **AWS** leverages enterprise software engineering principles to provide scalable, reliable, and secure cloud services to enterprises worldwide.

Session

WRAP UP

ANY
QUESTIONS?







Thank You!

